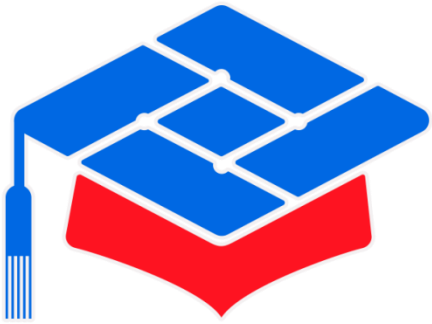


第十届

全国大学生集成电路创新创业大赛

报告类型：技术文档

参赛杯赛：“紫光同创”企业命题

作品名称：基于紫光 FPGA+RK3568 的视频流智能交通识别系统

队伍编号：CICC1001862

团队名称：ZLC

目录

1 作品预览简介 3
初赛方案 5
2 系统组成及总体设计方案 5
2.1 整体介绍 5
2.2 硬件系统介绍 6
2.2.1 硬件整体介绍 6
2.3 算法介绍 6
2.3.1 算法整体介绍 6
2.3.2 RK3568 算法模块介绍 7
2.3.3 FPGA 端算法模块介绍 9
分赛区决赛方案 11
1 系统组成及总体设计方案 11
1.1 整体介绍 11
1.2 硬件系统介绍 12
1.2.1 硬件整体介绍 12
1.3 算法介绍 14
1.3.1 算法整体介绍 14
1.3.2 RK3568 算法模块介绍 15
1.3.3 车牌检测模型的训练与底层优化 34
1.3.4 FPGA 端算法介绍 40
2 性能参数 76
2.1 RK3568 侧误差测算 76
2.1.1 模型训练概述 76
2.1.2 实验配置 77
2.1.3 模型的量化与部署 78
2.2 FPGA 资源开销 79
2.3 FPGA 车牌识别误差分析 81
3 仿真与测试报告 81
3.1 系统模块功能仿真测试 81
3.2 仿真环境与参数设置 82
3.3 模块仿真结果分析 82
4 总结 91
4.1 可扩展之处 91
4.2 心得体会与小组分工 92
5 参考文献 93
6 AI 使用说明 94

1. 作品预览简介

本项目设计了一套基于FPGA与ARM异构架构的智能交通视频流分析系统。系统采用盘古 PG2L200H-6IFBG676 与 RK3568 作为核心硬件平台，主要实现两大核心功能：基于视频流的实时车牌检测与识别、基于视频流的行人闯红灯检测。FPGA 负责 1920×1080@60 Hz HDMI 图像视频流采集，利用部署的 320×320 输入的 23 层 CNN 模型完成车牌检测框选，RK3568 负责后端高精度深度学习推理与结果渲染输出，从而构建出高效的并行处理视频分析架构。

关键技术：

- **FPGA 端 CNN 车牌四角点检测与硬件加速：**针对 FPGA 资源特点，设计由 Standard、Depthwise 和 Pointwise 卷积组成的 INT8CNN，通过 32 路并行 MAC 阵列和统一卷积引擎复用降低硬件开销；融合 40×40 浅层定位特征与 20×20 深层语义特征，利用 Center、Coarse、Corner 和 Offset 四个检测分支实现倾斜车牌的四角点精细定位。
- **FPGA 跨时钟域帧级同步与实时叠加：**采用 DDR3 帧环缓存、跨时钟域 Mailbox 和基于 frame_id 的结果表，将 CNN 输出与对应显示帧精确匹配并完成倾斜四边形检测框叠加；在 148.5 MHz 时钟下，CNN 最大单帧计算时间约 **13.65 ms**，1080p60 连续仿真达到 **60 帧/秒**稳态吞吐且无丢帧。
- **RK3568 边缘 AI 部署与推理：**RK3568 端部署改进的 YOLOv5s 检测模型与 CTC 端到端文字识别模型。采用多线程并发架构，利用队列分离 I/O 密集型的视频采集子线程与计算密集型的主推理线程。视频解码和图片格式转换下沉至硬件端，采用 MPP 和 RGA 完成，释放 CPU 的性能。
- **全栈远程管理系统：**采用前后端分离架构搭建 Web 远程管理平台。通过 SSH 和 WebSocket 协议，不仅支持对嵌入式端多种推理模式的远程启停控制，还能实现实时终端日志的低延迟推送与监控。

重要指标：

- **输入源与接口指标：**视频接口采用 MS7200 与 MS7210 构成的 HDMI 收发链路，兼容 HDMI 标准接口。工作于 **1920×1080@60 Hz** 模式，像素时钟为 148.5 MHz。支持 1920×1080@60 Hz 电脑、1080p@60Hz 摄像头输入。

- **系统吞吐与接口能力：** FPGA 端到端延时约 **30.20ms**；单帧 CNN 计算时间为 **13.65ms**；CNN 核心吞吐率为 **73.24 次/s**。RK3568 单独完成检测识别全流程端到端延时约 **70.40ms**；由 FPGA 发送串口角点坐标，RK 解析车牌区域进行字符识别端到端时延约 **15ms**。
- **模型量化精度：**算法模型经 RKNN-Toolkit2 进行 INT8 量化后，检测模型 mAP@0.5 达 0.935；识别模型整牌准确率达 0.936。
- **推理加速性能：**部署至端侧后，检测推理单帧耗时降至 42ms，识别推理单帧耗时降至 10ms，充分满足智能交通场景的实时性要求。
- **支持识别功能与环境适应性：**FPGA 利用 CNN 模型，支持蓝、黄、绿、黑、白色单车牌检测；容忍一定光线条件差、雨雪天气环境，容忍最大 25°倾斜。FPGA 协同 RK3568 支持蓝、黄、绿、黑、白色单个或多个车牌识别（最多 5 个），可进行特殊军警、领事馆车牌识别；容忍光线条件差、一定范围过曝、雨雪天气环境，容忍 30°以内倾斜。
- **车牌识别实时帧率：**通过串口发送坐标、RK3568 定时解析坐标并进行当前帧识别，通过异步架构（显示识别分离框架）实时视频流显示帧率达 **60FPS**。

作品亮点

- **创新的“串口辅助”软硬异构协同：**系统提供三种灵活的运行模式。在核心的“串口坐标快速识别”模式中，FPGA 在硬件流水线完成车牌物理框选后，将四角坐标通过 UART 串口直传至 RK3568。RK 端接收坐标后直接裁剪目标 ROI 区域并送入 CTC 识别模型，跳过高算力消耗的 YOLO 检测环节，释放了 NPU 算力并提升了系统响应速率。
- **FPGA 侧 CNN 部署与硬件加速：**将 INT8 CNN 完整部署于 FPGA，构建了覆盖 HDMI 视频采集、DDR3 帧缓存、图像预处理、CNN 推理、四角点解码、几何校验和检测框叠加的视频处理系统。网络针对 FPGA 资源特点采用深度可分离卷积、多尺度特征融合、32 路并行 MAC 和统一卷积引擎复用，并通过 Center、Coarse、Corner 和 Offset 四分支实现倾斜车牌的四角点精细定位。在检测结果处理方面，作品将神经网络置信度与边界、角点顺序、凸性和面积等几何规则结合，同时通过跨时钟域 Mailbox、frame_id 结果表和 DDR3 帧环缓存实现检测结果与显示帧的精确绑定。

2. 系统组成及总体设计方案

2.1 整体介绍

系统采用 FPGA+Arm 异构思路，采用盘古 PG2L200H-6IFBG676+RK3568 的硬件平台，核心实现两大功能：**车牌识别**和**行人闯红灯检测**。基于模块化设计思想，由 FPGA 负责高速图像预处理与实时车牌检测及行人闯红灯检测；RK3568 可支持完整的车牌检测与识别流程；同时可通过串口传输 FPGA 已定位的车牌坐标实现 RK 车牌识别。通过并行处理架构实现高效的视频流分析。核心部分包括：FPGA 端图像采集与预处理模块及车牌框选、RK3568 车牌检测与识别部分、行人闯红灯两种方法实现、DDR3 内存模块、输出显示模块、以太网和 USB 的视频输出接口。

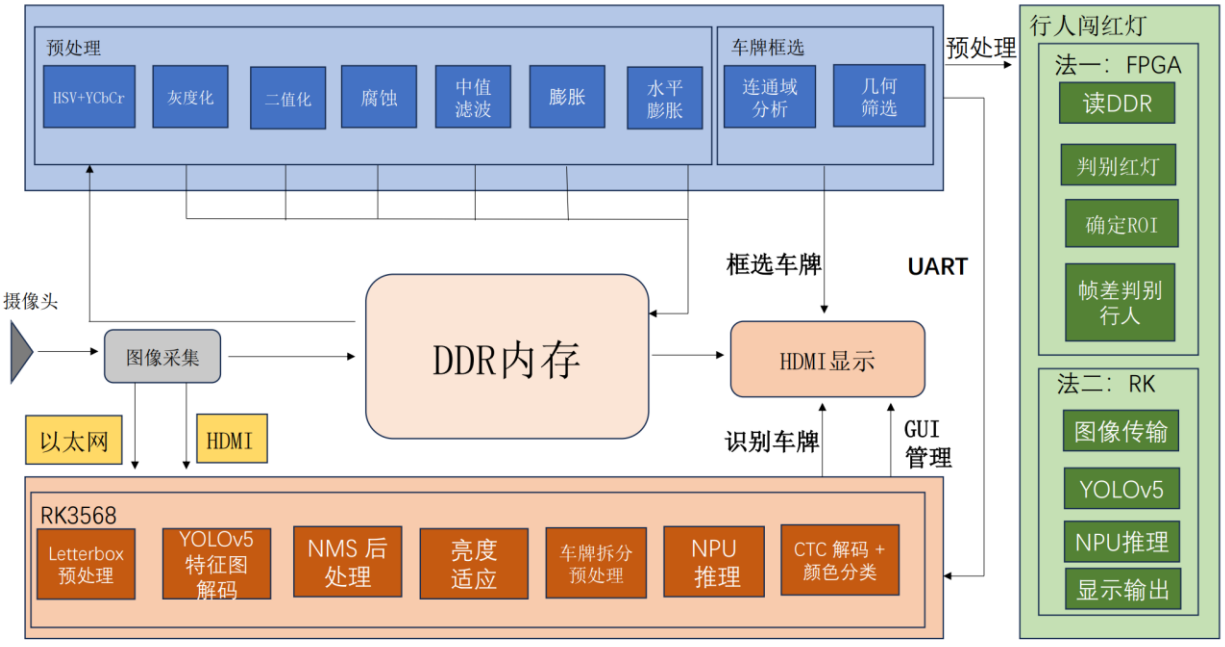


图 1 整体架构流程图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

系统使用盘古 PG2L200H-6IFBG676[1]开发板，摄像头 OV5640[2]通过 FMC[3]扩展板与 FPGA 相连，摄像头采集车牌照片、行人闯红灯视频支持 DDR 读取在 FPGA 硬件[4]流水线处理完成车牌框选、行人闯红灯框定并输出 HDMI 显示；或经过以太网、HDMI-USB 转接口传入 RK3568 通过 NPU 单独完成高精度字符识别与行人检测；同时支持通过串口从 FPGA 向 RK 传输实时车牌坐标，帮助 RK 跳过定位算法加速 RK 识别。

硬件接口示意如下：

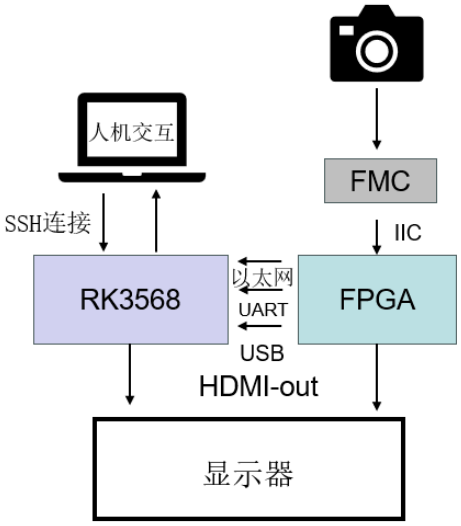


图 2 硬件接口示意图

2.3 算法介绍

2.3.1. 算法整体介绍

FPGA 端：负责视频采集，通过色彩空间转换、图像滤波、形态学处理、连通域分析、车牌角点检测等完成车牌框选；通过图像预处理、颜色判别、帧差检测完成行人闯红灯的违法识别。

RK3568 端：运行 YOLOv5、CTC 等深度学习推理算法，完成车牌检测与识别、行人闯红灯行为分析。

2.3.2. RK3568 算法模块介绍

系统概述：

面向智能交通场景，实现两大核心功能：

- 基于视频流的实时车牌检测与识别。
- 基于视频流的行人闯红灯检测。

硬件平台由开发板、板卡、采集卡和显示器组成。系统代码分为部署在 RK3568 的嵌入式应用和部署在上位机的远程管理平台。

术语	说明
RKNN	Rockchip NPU 推理引擎及开发工具包
FPGA	现场可编程门阵列
NPU	神经网络处理单元
RGB565	16 位色深像素格式
CTC	连接时序分类算法
NMS	非极大值抑制
V4L2	Linux 视频采集框架
Framebuffer	Linux 内核帧缓冲设备
JWT	鉴权令牌
SSH	安全外壳协议

表 1 术语及说明

设计功能：

车牌检测与识别：通过 HDMI 采集卡或以太网 UDP 接收 FPGA 视频流，使用 RKNN NPU 运行 YOLOv5 检测模型和 CTC 识别模型，完成车牌定位、文字识别和颜色分类，结果通过 HDMI 输出。

串口坐标快速识别：接收 FPGA 通过串口发送的车牌四角坐标，跳过检测模型直接进行识别，提升处理速度。

行人闯红灯检测：使用 YOLOv5 模型检测行人、红绿灯及闯红灯行为，支持 FPGA 以太网、HDMI 采集卡和本地视频三种输入源。

Web 远程管理：提供基于浏览器的管理平台，通过 SSH 远程启动，停止嵌入式端功能，实时查看运行日志。

系统组成：

嵌入式 AI 应用：部署于 RK3568 开发板，负责视频采集、AI 推理和结果输出。

Web 远程管理平台：部署于上位机，提供 Web 界面进行远程控制和监控。

环境分类	组建/类别	规格与说明
硬件环境	主控板	RK3568
	FPGA 板卡	PG2L200H-6IFBG676
	HDMI 采集卡	MACROSILICON 芯片
	HDMI 显示器	通过帧缓存驱动输出图像
软件环境	操作系统	Linux
	编程语言	Python3
	算法与视觉处理库	RKNN-Toolkit2、OpenCV、Pillow、NumPy
	硬件通信库	Pyserial
	前后端框架	Flask、Vue3、Element Plus

表 2 运行环境说明

总体架构：

系统采用三层架构，自下而上分别为感知层、处理层和应用层。

感知层：由摄像头、FPGA 板卡和 HDMI 采集卡组成。摄像头采集视频信号并传输给 FPGA 板卡，FPGA 负责视频编码与转发，通过以太网 UDP 协议将 RGB565 格式的 640×480 视频帧持续发送至 RK3568 开发板，同时通过 UART 串口发送车牌四角坐标数据。HDMI 采集卡作为备选输入源，通过 V4L2 接口提供 1280×720@30fps 的 MJPG 视频流。

处理层：部署在 RK3568 开发板上。该层运行嵌入式 AI 应用，采用多进程+多线程并发架构：独立子进程负责视频采集，通过单槽 multiprocessing.Queue 与主进程通信；主线程执行视频解码、NPU 推理和结果输出的完整流水线。RK3568 内置的 NPU 运行 RKNN int8 量化模型，完成 YOLOv5 目标检测和 CTC 文字识别等 AI 推理任务。处理结果通过 HDMI Framebuffer 输出至显示器。

应用层：包含 HDMI 视频输出终端和 Web 远程管理平台。Web 平台采用前后端分离架构，后端基于 Flask 框架，通过 Paramiko SSH 协议远程连接 RK3568 设备，控制推理模式的启停；前端为 Vue3 单页应用，通过 WebSocket 实时接收运行日志和状态推送。用户通过浏览器即可完成远程监控和管理。

各层之间通过以下协议通信：FPGA 与 RK3568 之间使用 UDP 以太网或 HDMI

传输视频帧和 UART 串口传输坐标数据；Web 管理平台与 RK3568 之间使用 SSH 协议进行命令控制，WebSocket 协议进行实时日志推送，HTTP REST API 进行模式管理和状态查询。

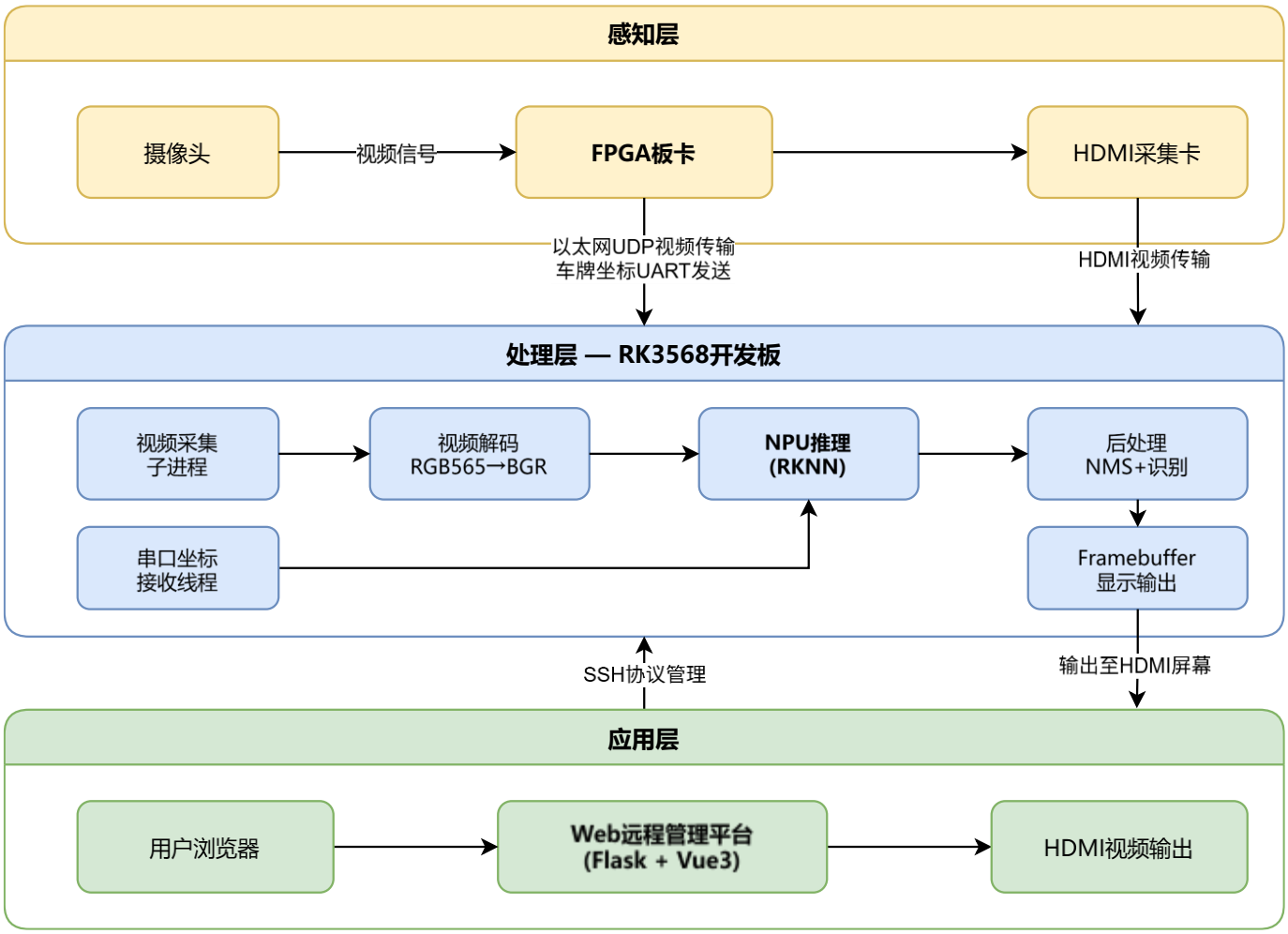


图 3 系统总体架构

2.3.3 FPGA 端算法模块介绍

车牌识别框选模块设计

摄像头连续采集，经过“颜色过滤+形态学+边缘检测”提取出轮廓，计算出两个目标四个角的精确坐标：一路将坐标通过串口上报，另一路将坐标画在图像上并存入 DDR，最终以 720P 高清格式输出至 HDMI 屏幕。

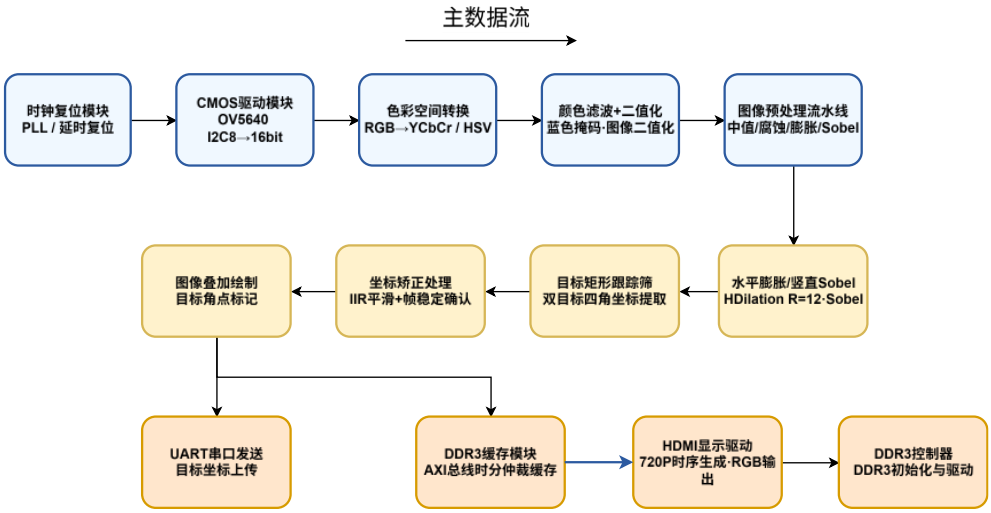


图 4 车牌识别形态学方法顶层模块流程图

行人闯红灯预警检测模块设计

色彩空间预处理模块

复用车牌识别的色彩预处理模块。

dynamic_pedestrian_detector

设计功能：dynamic_pedestrian_detector 模块通过逐帧特定区域的颜色判定进行红绿灯状态的确定；运用帧差法，判定运动行人，将输入亮度的高 4 位存入 BRAM，延迟一帧，计算当前 4 位亮度与上一帧同位置亮度差的绝对值 abs_diff，累加整帧所有 abs_diff，帧结束时取平均的高 4 位作为 global_bias，对每个像素的 abs_diff 减去 global_bias 并对结果进行非负截断，得到补偿后的差分 abs_diff_comp。判定 abs_diff_comp>3 且有效数据有效时生成最终的运动信号。模块内部嵌入了腐蚀、膨胀，同时动态记录最小/最大 x、y 坐标，最终在帧结束时得到原始运动矩形框。为避免错框多框，模块叠加宽高比限制、连续帧确认及 IIR 防抖逻辑。

分赛区决赛方案

1. 系统组成及总体设计方案

1.1. 整体介绍

系统仍采用 FPGA+Arm 异构思路，基于盘古 PG2L200H-6IFBG676+RK3568 的硬件平台，核心实现两大功能：**车牌识别**和**行人闯红灯检测**。在初赛方案的基础中，我们发现形态学准确率欠佳，受环境光线等因素影响较大，我们学习并部署成功了 FPGA 侧输入尺寸为 **128×128 的 6 层 CNN**、**320×320 的 23 层 CNN**。在初赛基础上扩展输入源，融合 **1920×1080，60Hz 的 HDMI 输入**，支持电脑屏幕、1080p 摄像头。全流程由 FPGA 利用 CNN 卷积计算负责车牌检测框选，RK3568 可支持完整的车牌检测与识别流程；同时可通过串口传输 FPGA 已定位的车牌坐标实现 RK 车牌识别，通过并行处理架构实现高效的视频流分析。行人闯红灯复用初赛方案，分别在 FPGA 和 RK3568 保留两种不同实现。

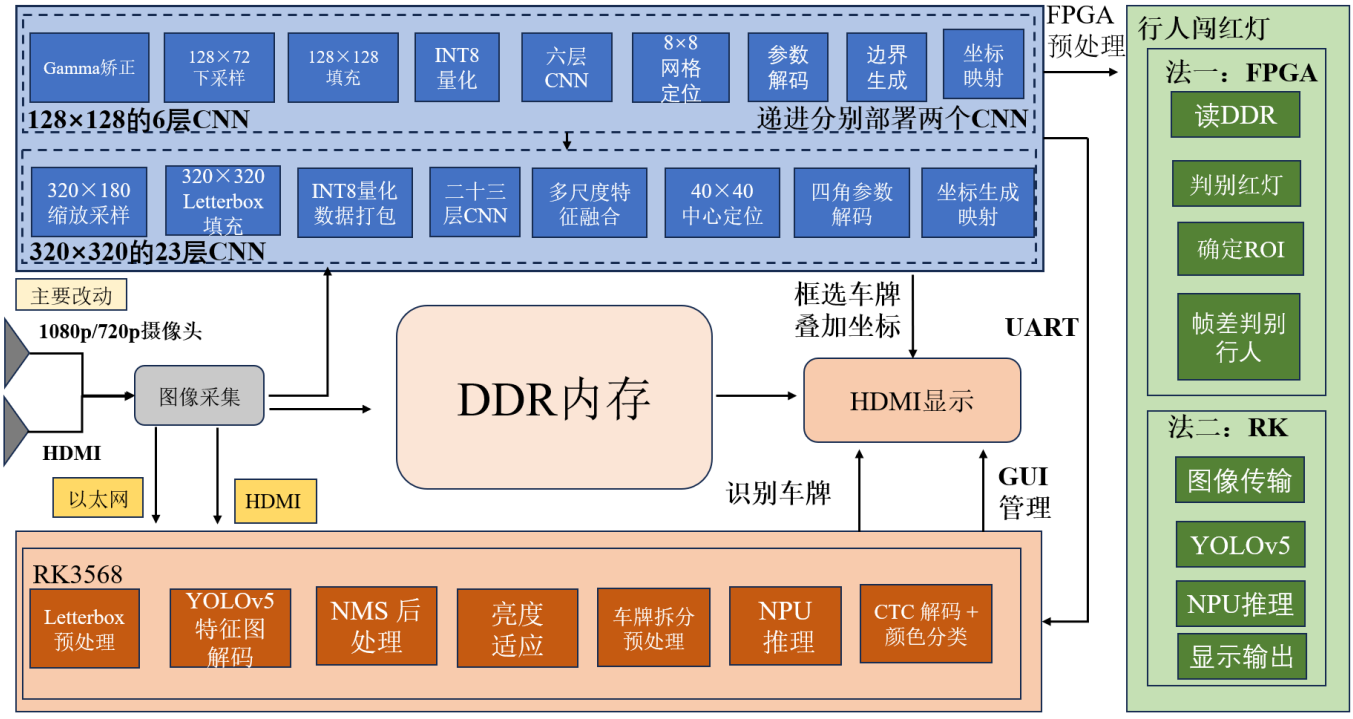


图 5 改进系统方案流程图

1.2. 硬件系统介绍

1.2.1 硬件整体介绍

系统使用盘古 PG2L200H-6IFBG676[1]开发板，在初赛硬件上扩展 1080p 输入，支持 1080p 摄像头和电脑屏幕 1080p 作为输入源。输入源采集车牌照片、行人闯红灯视频支持 DDR 读取并在 FPGA 硬件[4]流水线处理完成车牌框选、行人闯红灯框定并输出 HDMI 显示；或经过 HDMI-USB 转接口传入 RK3568 通过 NPU 单独完成高精度字符识别与行人检测；同时支持通过串口从 FPGA 向 RK 传输实时车牌坐标，帮助 RK 舍弃检测定位算法加速 RK 识别。

硬件接口示意如下：

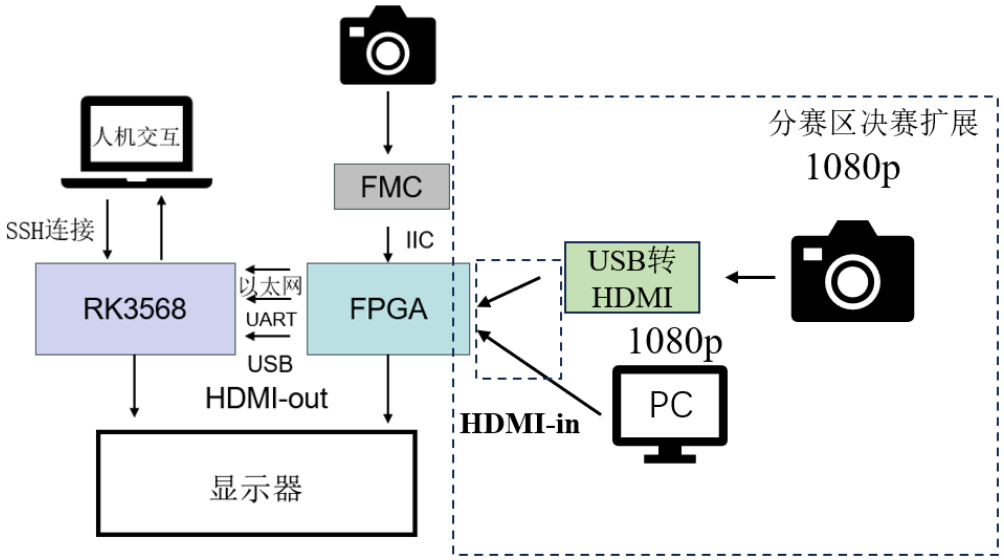


图 6 硬件系统接口

1.2.2 电路各模块介绍

1.UART 电路：

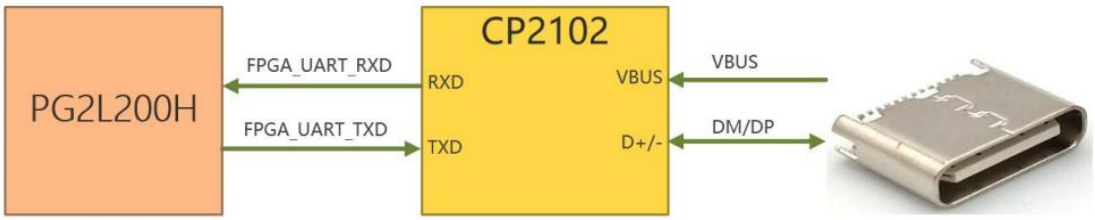


图 7 UART 电路图

2.HDMI 电路：

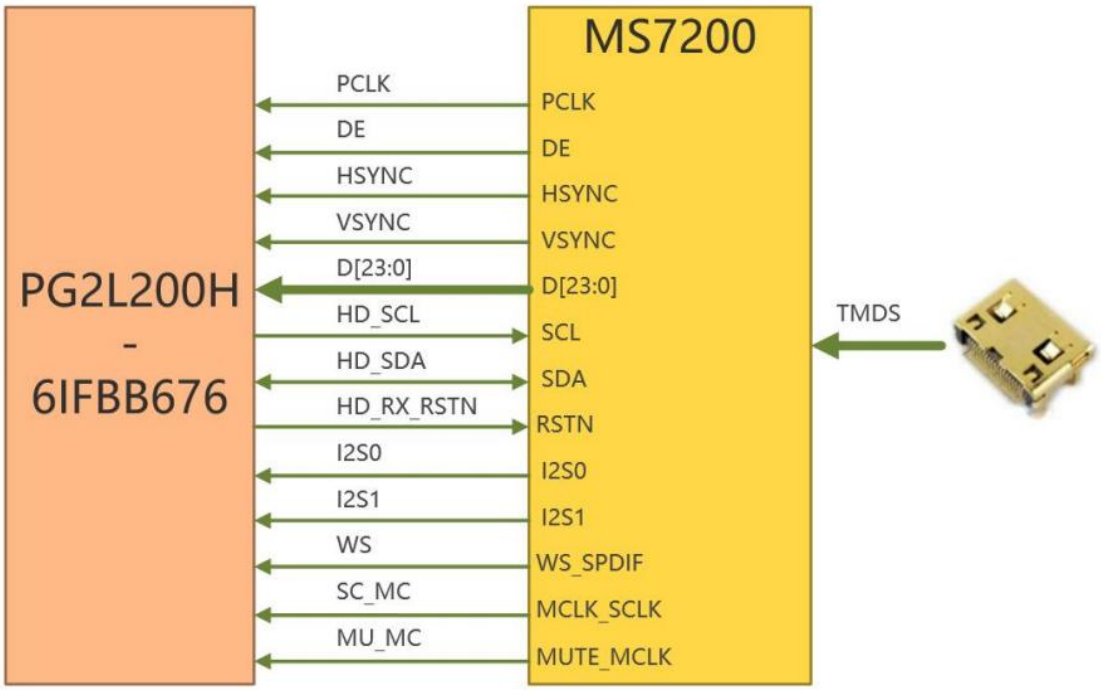


图 8 HDMI 电路图

3.HDMI 转 USB 转接模块电路图：

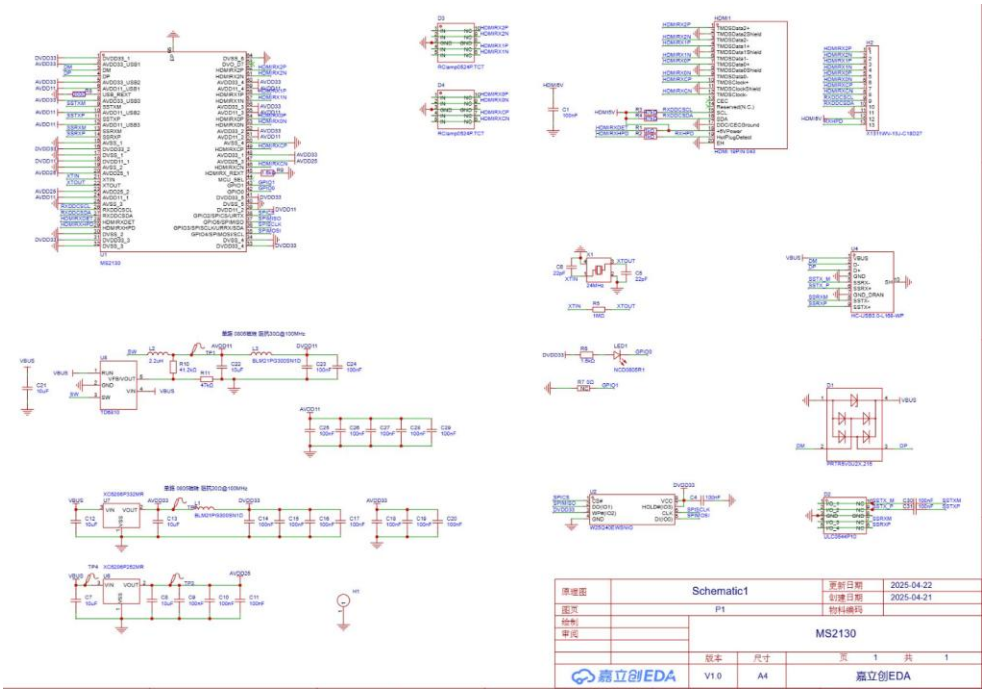


图 9 HDMI 转 USB 电路图

4.USB 转 HDMI 转换模块电路图：（部分）

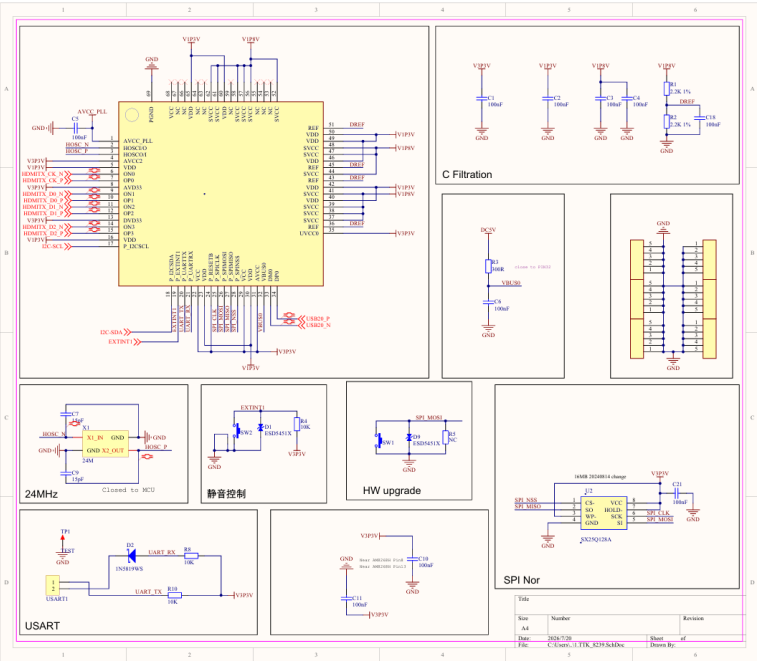


图 10 USB 转 HDMI 模块电路图

1.3 算法介绍

1.3.1. 算法整体介绍

FPGA 端：第一版设计部署 128×128 六层 INT8 CNN，第二版将输入提高到 320×320，采用 23 层深度可分离卷积网络和多尺度特征融合结构进一步优化。采用 128×128 输入的六层 INT8 CNN，将 1920×1080 图像缩放为 128×72 并通过 Letterbox 填充到 128×128，经过卷积推理后在 8×8 特征图中搜索最大目标响应，解码车牌中心、宽度和高度，生成水平矩形框并映射回原始图像进行显示。将输入提高到 320×320，采用 23 层深度可分离卷积网络和多尺度特征融合结构，通过中心响应、四角粗坐标、角点置信度和精细偏移等多个检测分支，直接预测车牌真实的左上、右上、右下和左下四个角点，并对结果执行边界、点序、凸性、面积和置信度校验；检测结果通过帧编号与 DDR 读出的原始视频帧准确匹配，最终在 HDMI 输出

RK3568 端：初赛 RK3568 端采用 OpenCV 软解码 720p@30fps 视频、cv2 预处理、写 Framebuffer 显示的串行流水线。区域决赛阶段我们对 RK3568 端做了全链路硬件加速与异步流水线重构，在保持识别精度不变的前提下大幅提升实时性，主要改进如下：

1. 视频输入升级到 1080p@60fps：HDMI 采集从 1280×720@30fps 提升到 1920×1080@60fps，匹配 FPGA 侧新增的 1080p 输入源。

2. MPP 硬件解码替代软件解码：MJPEG 解码由 Rockchip MPP 硬件解码器完成，取代初赛的 OpenCV 软件解码。

3. RGA 硬件预处理：NV16 到 BGR 的色彩空间转换与 letterbox 检测前处理由 RGA 硬件完成，输出 NHWC 格式，省去 CPU 的转置运算。

4. DRM、RGA 双缓冲显示：显示后端由直写帧缓存方案，升级为基于 DRM/KMS 叠加平面、配合 RGA 硬件合成的双缓冲输出模式，实现输出画面稳定 1080p@60fps。

5. 显示识别完全异步：采集、检测、识别、显示分线程运行，显示线程恒定 1080p@60fps 输出，不被 NPU 推理阻塞。

6. 原生 C 库分层调度架构：采集、解码、显示、前处理等性能关键链路，全部下沉至原生 C 语言动态库中实现；Python 层仅承担整体流程的编排与调度职能，通过标准外部调用接口与底层库交互。

7. 识别结果优化：初赛识别模型单帧直接输出，区域决赛新增多帧投票稳定与结果缓存复用，减少误识别抖动并提升识别效率。

1.3.2 RK3568 算法模块介绍

术语	说明
RKNN	Rockchip NPU 推理引擎及开发工具包
NPU	神经网络处理单元
MPP	Rockchip 媒体硬件编解码平台
RGA	Rockchip 2D 图形加速器，负责缩放与色彩空间转换
DRM/KMS	Linux 直接渲染管理与内核模式设置显示框架
CTC	连接时序分类算法
NMS	非极大值抑制
V4L2	Linux 视频采集框架
MJPEG	Motion JPEG 视频编码格式
CTC 字典	78 字符车牌字符映射表

表 3 RK3568 关键术语介绍

分类	组件、类别	规格与说明
硬件环境	主控板	RK3568
	FPGA 板卡	PG2L200H-6IFBG676
	HDMI 采集卡	MACROSILICON 芯片，V4L2 接口
	HDMI 显示器	DRM/KMS 加 RGA 双缓冲输出
软件环境	操作系统	Linux
	编程语言	Python，C++
	算法和视觉库	RKNN-Toolkit2、OpenCV、NumPy
	硬件加速	MPP 硬件解码、RGA 预处理与色彩转换、DRM/KMS 显示
	硬件通信库	Pyserial
	前后端框架	Flask、Vue3、Element Plus

表 4 作品依赖环境

RK3568 架构

软件框架

```
final
├─ core                          核心基础层
│   ├─ camera_fps.py             摄像头帧率曝光配置
│   ├─ display.py                Framebuffer 显示后端
│   ├─ display_drm_rga.py        DRM/RGA 双缓冲显示
│   ├─ display_factory.py        显示后端工厂
│   ├─ fpga_io.py                UDP 视频流接收解码
│   ├─ hdmi_io.py                HDMI 采集常量与工具
│   ├─ hdmi_native.py            MPP 硬解采集封装
│   ├─ image_utils.py            图像预处理工具
│   ├─ native                    native C 加速库
│   │   ├─ librga_preprocess.so  RGA 前处理库
│   │   ├─ librkdrm_display.so   DRM 显示库
│   │   ├─ librkhdmi_capture.so  MPP 采集解码库
│   │   ├─ Makefile              native 库编译脚本
```

```
│   │   ├── rga_preprocess.c          RGA 前处理源码
│   │   ├── rkdrm_display.c           DRM 显示源码
│   │   └── rkhdmi_capture.c          采集解码源码
│   ├── postprocess.py                YOLO 解码与 NMS 后处理
│   ├── rga_preprocess.py             RGA 前处理封装
│   ├── serial_io.py                  串口坐标协议解析
│   └── __init__.py                   core 包导出
├── fpga_ai.py                        车牌检测识别入口
├── fpga_serial.py                    串口坐标识别入口
├── people_det                        行人检测模块
│   ├── people_detector.py            行人红灯检测器
│   ├── people_run.py                 多模式运行封装
│   └── __init__.py                   包初始化
├── people_redlight.py                行人闯红灯入口
├── plate_det                         车牌检测识别模块
│   ├── detection_filter.py           检测候选过滤
│   ├── plate_detector.py             车牌检测器
│   ├── plate_draw.py                 绘制框与标签
│   ├── plate_pipeline.py             透视校正预处理
│   ├── plate_recognition.py          字符表与颜色表
│   ├── plate_recognizer.py           CTC 识别颜色分类
│   ├── realtime_cache.py             识别结果缓存复用
│   ├── result_stabilizer.py          多帧投票稳定
│   └── __init__.py                   包初始化
├── rknn_infer.py                     CLI 离线推理工具
├── rknn_models                       RKNN 模型目录
│   ├── plate_detect_320_int8.rknn    车牌检测模型
│   ├── plate_rec_color_int8.rknn     车牌识别模型
│   └── runlight_yolov5_640x360_int8.rknn  行人红灯模型
```

└─ __init__.py	顶层包初始化

表 5 目录结构

嵌入式端 AI 应用部署在 RK3568 开发板/home/cat/RK3568/final 目录，采用 Python 语言开发，基于 RKNN-Toolkit2 进行 NPU 推理。在初赛的应用入口层、AI 检测识别层、核心基础层三层结构的基础上新增硬件加速层，即用 C 语言编写并编译为动态库的加速模块，将性能关键路径下沉到底层。四层自上而下为：

应用入口层：提供四个独立的运行入口，每个入口对应一种使用场景。

fpga_ai.py：　执行完整的车牌检测和识别流程，接收 HDMI 采集卡或 FPGA 以太网视频流，依次调用检测模型和识别模型，识别结果经多帧稳定优化后输出至 HDMI；

fpga_serial.py：　跳过检测环节，接收 FPGA 串口发送的车牌坐标后直接调用识别模型，适用于 FPGA 已完成目标检测的场景；

people_redlight.py：运行行人闯红灯检测，支持 HDMI 采集卡、FPGA 以太网和本地视频文件三种输入源。

rknn_infer.py：　CLI 批量离线推理工具，遍历图片目录执行检测+识别并保存结果，用于前期调试。

AI 检测识别层：封装核心推理逻辑。

plate_detector.py：　运行 RKNN 车牌检测模型检测车牌。

plate_pipeline.py：　车牌透视校正、双层车牌分割预处理。

plate_recognizer.py：车牌识别 CTC 文字解码、颜色分类。

plate_draw.py：　绘制识别结果和中文标签。

detection_filter.py：检测候选的几何与置信度过滤。

result_stabilizer.py：识别结果多帧投票稳定。

realtime_cache.py：识别结果按交并比缓存复用，跳过重复识别。

people_detector.py：运行 RKNN 检测模型检测行人、红绿灯、斑马线等目标。

people_run.py：　提供行人闯红灯检测的多模式运行封装。

核心基础层：提供硬件通信和通用处理能力，被上层所有模块共同依赖。

hdmi_native.py：　通过底层库完成 V4L2 采集、MPP 硬件解码及 RGA 色彩转换，取代初赛的 OpenCV 软件解码采集。

rga_preprocess.py：调用底层库用 RGA 硬件完成检测前处理，包括等比缩放与

BGR 转 RGB 并输出紧凑格式。

display_factory.py： 显示后端工厂，按参数创建 DRM/RGA 或 Framebuffer 后端。

display_drm_rga.py： 调用底层库以 DRM/KMS 配合 RGA 双缓冲输出 HDMI。

display.py： 通过内存映射直接写入 Framebuffer 实现 HDMI 输出，调试时作为备用回退后端保留。

camera_fps.py： USB 摄像头帧率与曝光配置。

postprocess.py： NumPy 实现的 YOLO 特征图解码和 NMS 后处理。

serial_io.py： 解析 UART 串口坐标协议。

fpga_io.py： 实现 FPGA UDP 视频帧接收与 RGB565 解码。

hdmi_io.py： 保存 HDMI 采集设备与分辨率常量及队列工具。

硬件加速层：用 C 语言编写并编译为动态库的硬件加速模块，供核心基础层调用。

rkhdmi_capture.c→librkhdmi_capture.so：负责视频采集、硬件解码与 BGR 格式转换。

rga_preprocess.c→librga_preprocess.so：RGA 硬件实现车牌检测前处理。

rkdrm_display.c→librkdrm_display.so：双缓冲 HDMI 显示，DRM 搭配 RGA 合成。

Makefile：原生库编译脚本，自动配置依赖并生成对应动态库。

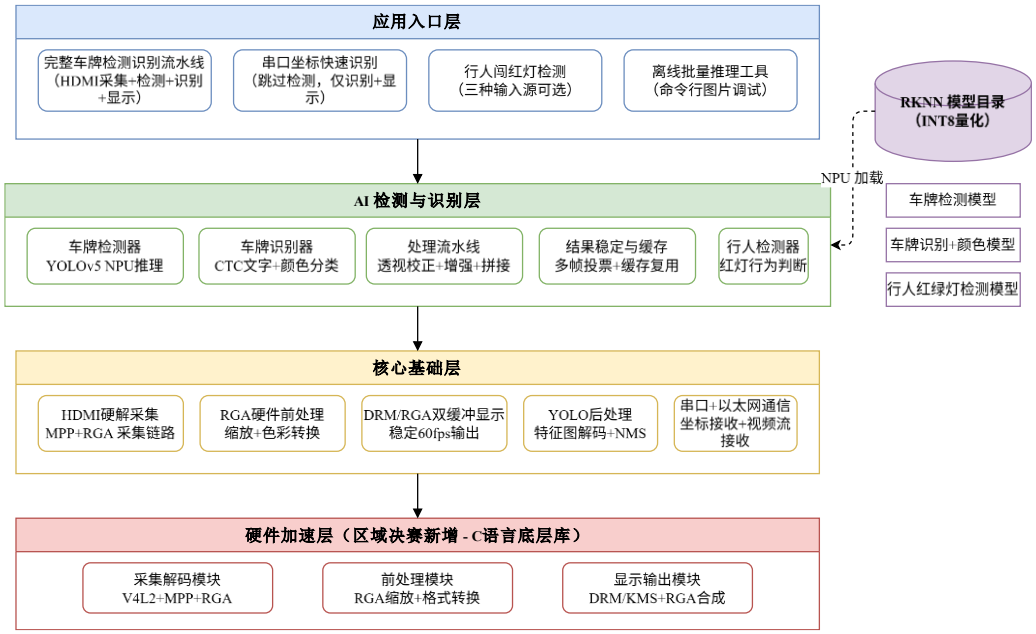


图 11 嵌入式端软件架构

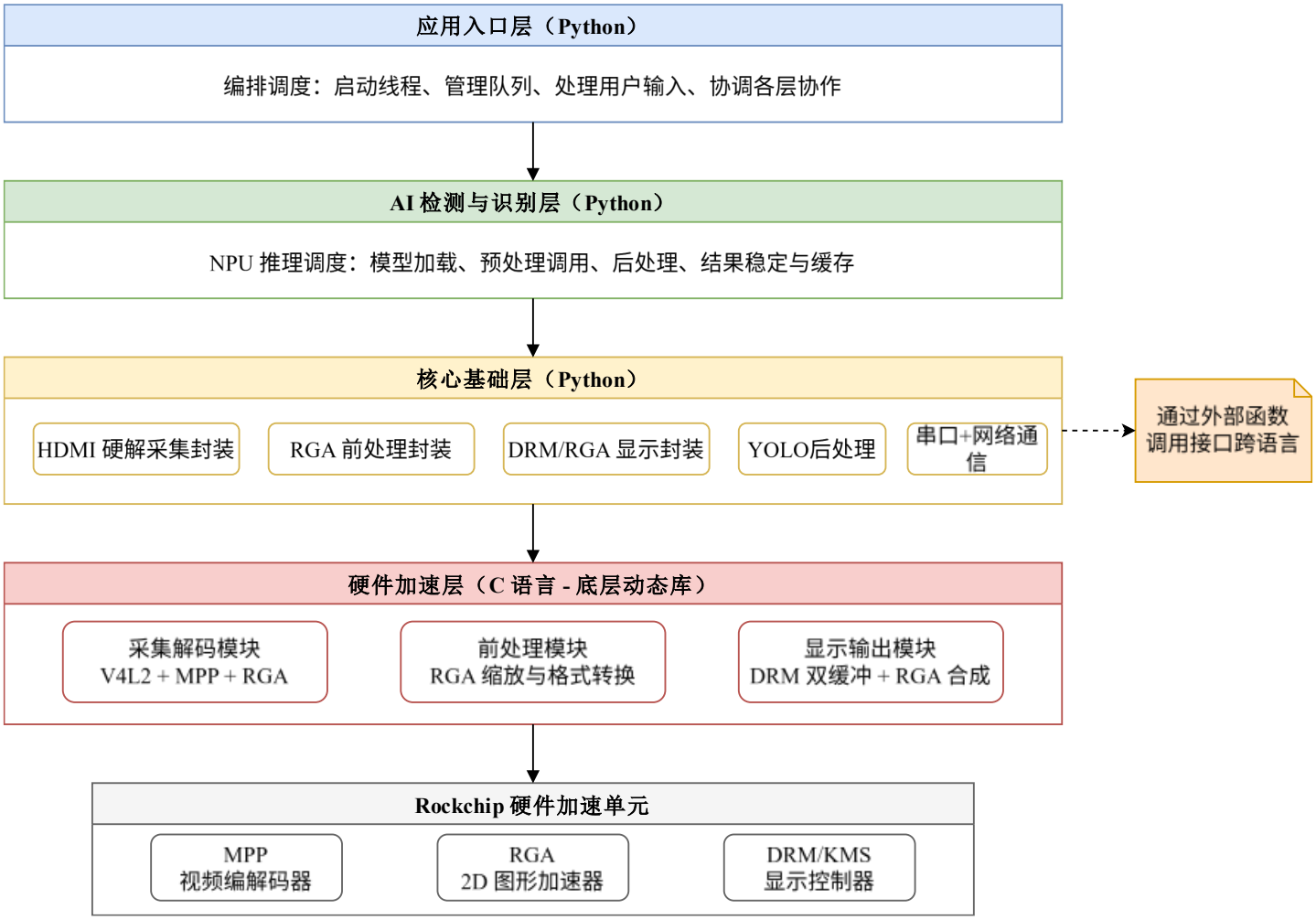


图 12 各层调用关系

进程与线程模型：初赛采用多进程加消息队列的并发方案，独立子进程负责视频采集，通过消息队列与主进程通信。决赛针对初赛方案中进程间序列化开销大、显示与推理互相阻塞的问题，重构为多线程异步流水线架构，针对输入输出密集型的视频采集与计算密集型的 NPU 推理分别选用合适的并发机制。以主入口 fpga_ai.py 为例，系统共运行四个线程：

线程 1：视频采集线程

该线程封装在底层 C 库内部，负责完整的硬件采集与解码链路。线程通过 V4L2 接口从采集卡以内存映射方式抓取 MJPEG 码流，调用 MPP 硬件解码器解码为原始图像，再经 RGA 硬件完成色彩空间转换，最终输出 BGR 帧写入单槽队列。队列满时丢弃旧帧，确保下游始终拿到最新画面。

Thread 1: Capture Thread
Input: HDMI capture device, MJPEG 1920×1080@60fps **Output:** frame_queue (single-slot, BGR frame)

v4l2_dev ← open V4L2 device, configure MJPEG 1920×1080@60fps
mpp_dec ← MPP decoder init (MJPEG → NV12/NV16)
rga_ctx ← RGA context init (NV12/NV16 → BGR888)
mmap_buffers ← V4L2 MMAP buffer pool
while not stop do
  buf ← v4l2_dequeue(mmap_buffers) // 等待采集卡填充一帧
  pkt ← wrap_mpp_packet(buf) // 封装为 MPP 数据包
  frm ← mpp_decode(mpp_dec, pkt) // 硬件解码
  bgr ← rga_convert(rga_ctx, frm) // 硬件色彩转换
  try frame_queue.put_nowait(bgr)
  catch Full:
  frame_queue.get_nowait() // 丢弃旧帧
  frame_queue.put_nowait(bgr) // 放入新帧
  v4l2_queue(buf) // 归还缓冲区

线程 2：检测线程

该线程从帧队列取出最新帧，调用 RGA 硬件完成检测前处理——等比缩放至 320×320 与 BGR 转 RGB，输出紧凑排列格式。随后送入 NPU 运行 YOLOv5 检测模型，对输出的三个特征图执行解码和 NMS 过滤，还原为原图尺度坐标。每个检测结果包含边界框、置信度、四个角点坐标和类别标签。检测结果送入识别队列供识别线程消费。

Thread 2: Detection Thread
Input: frame_queue, det_rknn **Output**: det_queue (bbox, landmarks, confidence, class)

post_processor ← YOLOPostProcessor(grid_cache, anchors)
while not stop do
  bgr ← frame_queue.get() // 阻塞等待最新帧
  nhwc_320 ← rga_letterbox_rgb(bgr, 320) // RGA 硬件: 缩放
  if RGA failed then
  nhwc_320 ← cv2_letterbox(bgr, 320) // 回退 cv2 预处理
  outputs ← det_rknn.inference(nhwc_320) // NPU 推理 (3 特征图)
  boxes, confs, landmarks, classes ← post_processor.decode(outputs)
  indices ← NMS(boxes, confs, iou_thresh) // OpenCV C++ 后端
  det_results ← scale_to_original(indices, 320, bgr.shape)
  det_queue.put(det_results)

线程 3：识别线程

该线程从检测队列取出结果，对每个检测框依次执行透视校正、自适应亮度增强，双层车牌则水平拼接为单行，然后送入 NPU 运行 CTC 识别模型。识别结果经多帧投票稳定器处理后写入共享结果字典，供显示线程叠加绘制。

Thread 3: Recognition Thread

Input: det_queue, rec_rknn, stabilizer, cache
Output: shared_result (plate text, color, confidence)

```
stabilizer ← ResultStabilizer(mode="conservative", window=5, required=3)
cache ← RealtimeCache(ttl=0.5s)
while not stop do
    det_results, frame ← det_queue.get()
    for each det in det_results do
        if cache.match(det.bbox) then                    // 命中缓存则跳过 NPU
            result ← cache.get(det.bbox)
            continue
        roi ← perspective_correct(frame, det.landmarks)
        roi ← enhance_plate(roi)                         //自适应曝光调节
        if det.class == double_layer then
            roi ← split_and_merge(roi)                   //双层拼接为单行
        outputs ← rec_rknn.inference(roi)                //NPU 推理
        plate_text ← ctc_decode(outputs)                 //CTC 解码,查字典
        plate_color ← softmax_argmax(outputs.color_head)
        result ← {plate_text, plate_color, conf}
        stabilizer.update(det.bbox, result)              //送入投票稳定器
        cache.set(det.bbox, result)
    stable_result ← stabilizer.get_stable()              //达到投票阈值才返回
    if stable_result.ready then
        Lock.acquire()
        shared_result ← stable_result
        Lock.release()
```

线程 4：显示线程

该线程恒定以 60fps 速率运行，通过 DRM/KMS 叠加平面与 RGA 硬件合成实现双缓冲翻页输出。每帧从共享结果字典读取最新识别结果，绘制车牌框与中文标签叠加到当前 BGR 帧上，经 DRM 输出至 HDMI 显示器。显示与 AI 推理完全解耦，NPU 推理耗时不影响画面流畅度。

Thread 4: Display Thread

Input: frame_queue, shared_result, drm_ctx
Output: HDMI display 1920×1080@60fps

```
drm_ctx ← DRMDisplay(1920, 1080)
font ← load_chinese_font("platech.ttf", 24)
while not stop do
    bgr ← frame_queue.get()                          // 取最新帧用于显示
    Lock.acquire()
    result ← shared_result                           // 读取最新识别结果
    Lock.release()
    if result is not None then
        display_img ← draw_plate_box(bgr, result.bbox)
        display_img ← draw_chinese_text(display_img, result.text, font)
    else
        display_img ← bgr
    drm_ctx.page_flip(display_img)                   // RGA 合成+ DRM 输出
```

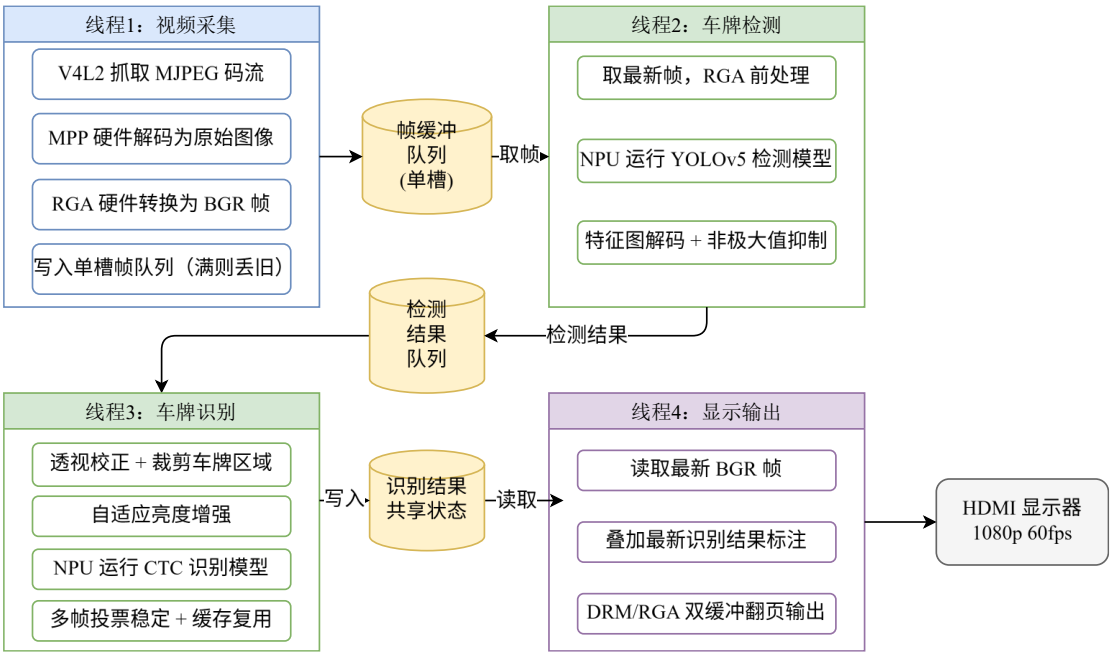


图 13 多线程异步流水线示意图

区域决赛核心优化产物

硬件加速层设计

硬件加速层是区域决赛相对初赛在架构上的关键增量。它由 C 语言源码及其编译产物动态库组成，通过 ctypes 供上层 Python 调用，把采集、解码、前处理、显示等性能关键路径从 Python 下沉到 C 层，直接调用 Rockchip 硬件加速单元，有效减轻 CPU 负荷。

模块名称	编译产物	功能	使用的硬件单元
采集模块	采集动态库	V4L2 采集、MJPEG 硬件解码、色彩转 BGR	MPP、RGA、V4L2
前处理模块	前处理动态库	车牌检测前处理，等比缩放与 BGR 转 RGB	RGA
显示模块	显示动态库	HDMI 双缓冲显示合成	DRM/KMS、RGA

图 14 硬件模块简介

1.采集模块：在底层完成从采集卡抓取 MJPEG 码流、调用 MPP 硬件解码器解码、再经 RGA 转换为 BGR 的完整链路，输出可直接供上层使用的 BGR 帧。

2.前处理模块：用 RGA 硬件完成检测模型所需的等比缩放与 BGR 转 RGB，直接输出紧凑排列格式，省去 CPU 上的缩放与数据重排开销。

3.显示模块：通过 DRM/KMS 叠加平面与 RGA 合成实现双缓冲显示输出，稳定输出 60fps 视频流

编译由统一的编译脚本管理，通过系统包管理工具链接 MPP、RGA、libdrm、libv4l2 等系统库，生成三个所需动态库。

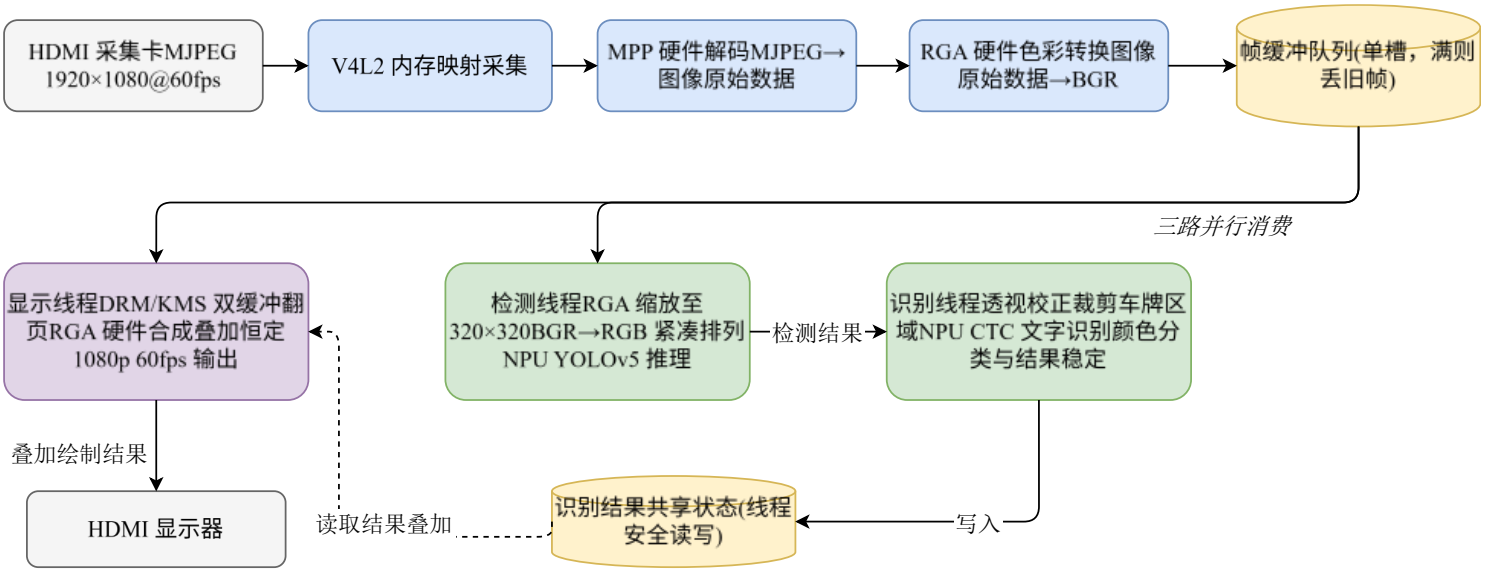


图 15 硬件加速链路流程图

识别结果优化

初赛的识别结果由识别模型单帧直接输出：一帧图像送入 CTC 识别模型，输出即作为最终车牌号与颜色，未做任何跨帧处理。这种方式对单帧误识别、光照抖动、遮挡瞬间较为敏感，同一辆车相邻帧可能输出不同结果，画面标注抖动明显。区域决赛在识别模型输出之后新增两级后处理，显著提升了结果稳定性与效率。

1.多帧投票稳定机制

目标关联：按检测框的重叠度将相邻帧的同一车牌关联为一条轨迹，每条轨迹维护一个固定长度的识别结果滑动窗口。关联阈值为重叠度不低于 0.5。

众数投票：车牌号、颜色分别在窗口内做众数统计，同一取值累计达到设定阈值次数才确认输出，过滤偶发的单帧误识别。

快速确认：若连续若干帧车牌号与颜色完全一致且各项置信度均超过阈值，则提前锁定结果，兼顾稳定与低延迟。

过期清除：轨迹按有效期自动清除，车辆离开视野后不残留旧结果。

2.识别结果缓存复用

已识别车牌的结果按检测框缓存。在有效时间窗内，若新检测框与缓存框重叠度达标，则直接复用已有识别结果，跳过重复的 NPU 识别推理。对静止或缓慢移动的车牌可大幅减少识别模型调用次数，释放 NPU 算力，降低整体时延。

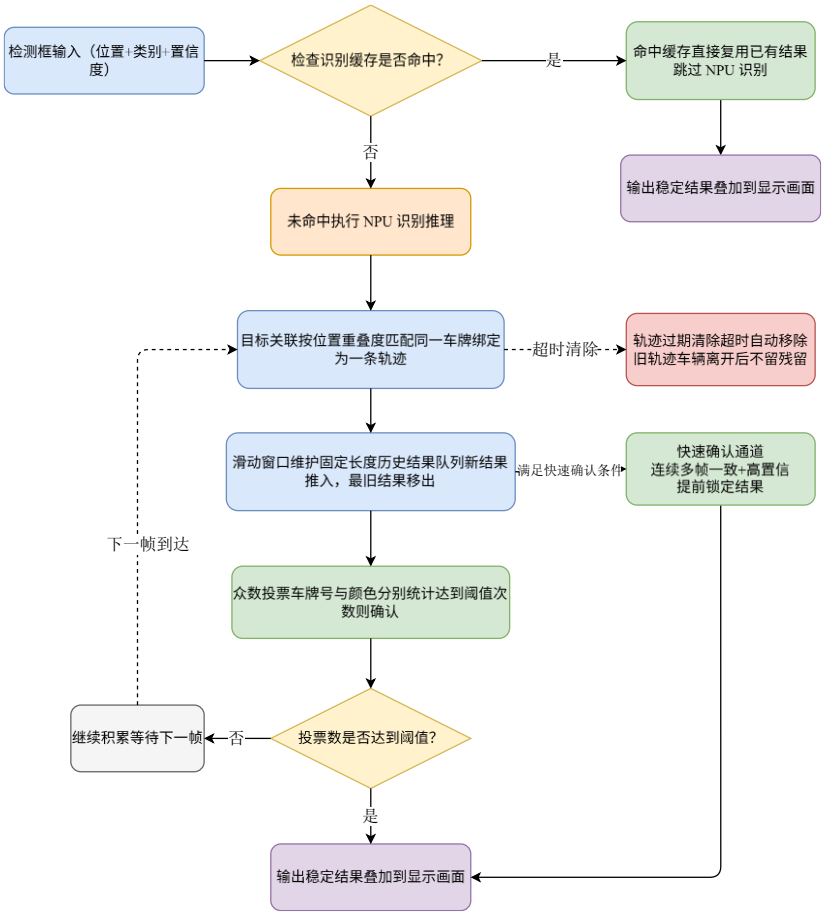


图 16 识别结果多帧投票稳定流程

Web 管理平台架构

软件框架

```
rk3568_manager/
├── app.py                    Flask 主应用，注册路由和 WebSocket
├── config.py                 全局配置
├── requirements.txt          Python 环境依赖
├── backend/                  后端核心逻辑
│   ├── auth.py                   JWT 认证
│   ├── ssh_manager.py            SSH 远程指令管理器
│   └── socket_handlers.py        WebSocket 实时通信处理
├── db/                       SQLite 数据库存储目录
└── frontend/                 前端 Vue 项目目录
    └── index.html            Vue3 单页面应用入口
```

Web 远程管理平台 rk3568_manager 部署在上位机，为用户提供基于浏览器的远程控制界面，支持远程启停 RK3568 上的推理功能并实时查看运行日志。

平台管理三种推理模式，每种模式对应 RK3568 上的一个入口脚本：

车牌检测和识别，执行 fpga_ai.py

串口坐标识别，执行 fpga_serial.py

行人闯红灯检测，执行 people_redlight.py

同一时刻只允许运行一种模式。

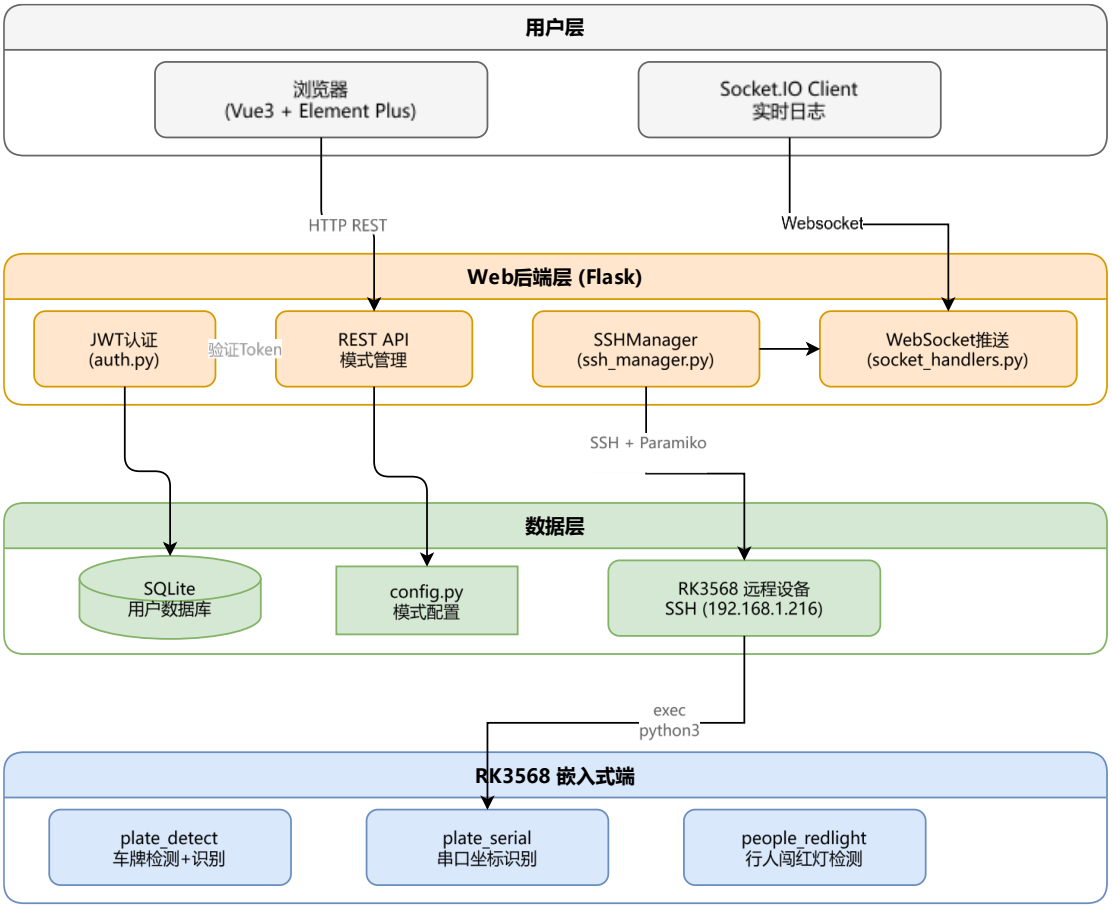


图 17 Web 端架构

后端架构：由四个核心模块组成。

app.py：Flask 主应用入口，负责注册路由蓝图、配置 JWT 认证中间件、挂载 CORS 跨域支持，并启动 Flask-SocketIO 服务。

auth.py：实现 JWT 认证蓝图，提供用户注册、登录和用户信息查询三个接口，用户数据以 bcrypt 哈希存储在 SQLite 数据库中。

ssh_manager.py：建立 SSH 连接到 RK3568 设备为每个推理模式分配 PTY 终端并执行对应的 Python 脚本，同时在后台线程持续读取输出，通过 SocketIO 推送到前端。

socket_handlers.py：注册 WebSocket 事件处理器，连接时验证 JWT Token，支持模式状态变更通知和实时日志流推送两个事件。

前端架构：单文件 SPA，使用 Vue 3 Composition API 和 Element Plus 2.7.0 构建。包含登录页和管理控制台两个页面。

控制台顶部显示导航栏，左侧展示三个模式卡片，右侧为实时终端日志面板。



图 18 登录页展示

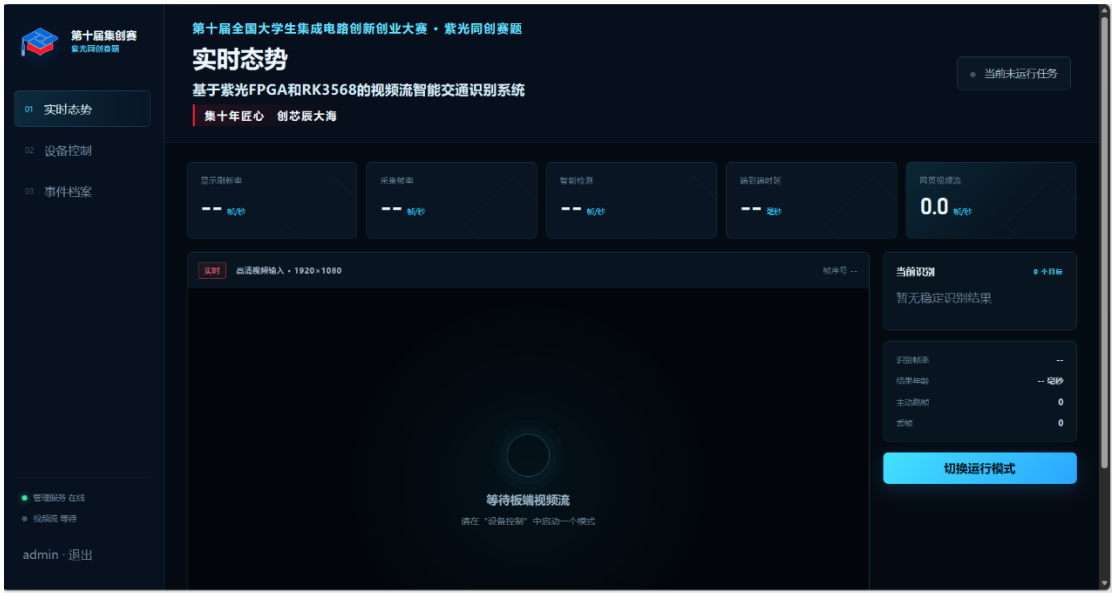


图 19 设备管理界面

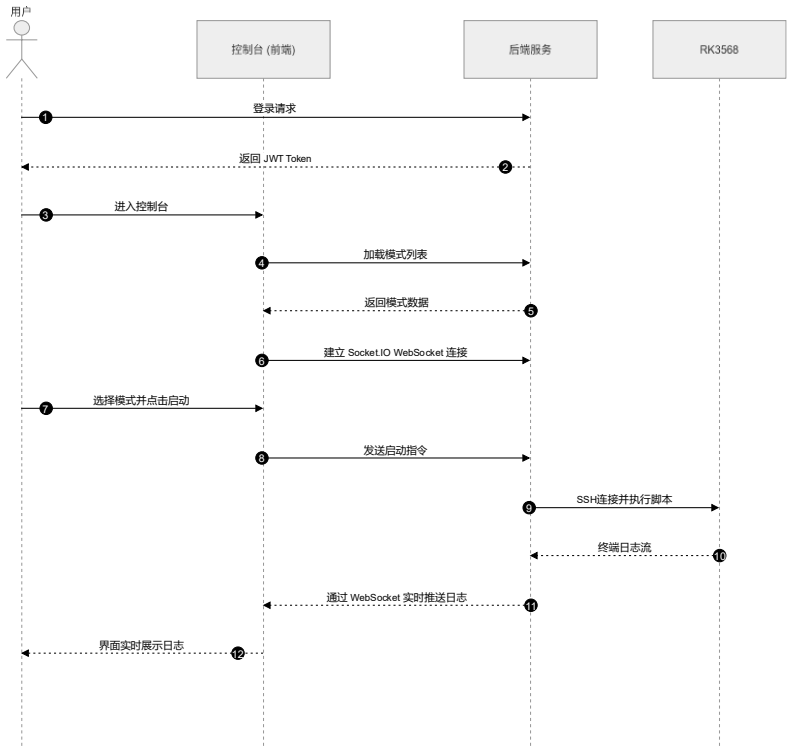


图 20 交互流程

底层核心模块设计

core/hdmi_native.py：HDMI 硬解采集

功能：通过底层 C 库在 native 层完成 V4L2 采集、MPP 硬件 MJPEG 解码与 RGA 色彩空间转换，输出 BGR 帧写入单槽队列，取代初赛基于 OpenCV VideoCapture 的软解采集。运行时自动预加载官方 MPP 运行时库。

NativeMppHdmiReceiver 类封装完整采集链路：初始化时加载 native 动态库，配置采集设备、分辨率、帧率和像素格式。start()方法创建 daemon 线程，内部调用 native 库的采集循环，在 C 层完成抓取、解码、色彩转换全流程，仅在输出 BGR 帧时回到 Python 侧入队。队列采用单槽设计，满时丢弃旧帧确保下游始终拿到最新画面。调试模式下支持统计 MPP 解码耗时和 RGA 转换耗时的周期快照。

core/rga_preprocess.py：RGA 硬件前处理

功能：封装 native RGA 库，用 RGA 硬件一步完成检测模型所需的等比缩放与 BGR 转 RGB，直接输出紧凑排列格式，省去 CPU 上的缩放与数据重排开销。

前处理函数接收任意尺寸的 BGR 图像和目标尺寸，内部调用 native 库的 RGA 接口执行硬件加速的等比缩放与色彩空间转换。与 cv2 方案相比，RGA 路径同时完成缩放和格式转换，且输出为紧凑排列布局，避免了 Python 侧的转置调用耗时。

core/display_drm_rga.py：DRM/RGA 双缓冲显示

功能：通过 native 显示库使用 DRM/KMS 叠加平面与 RGA 合成实现双缓冲显示输出，稳定 60fps 无撕裂，为区域决赛默认显示后端。

DRMDisplay 类初始化时打开 DRM 设备、查询显示分辨率和色深，配置两个帧缓冲区交替翻页。show()方法将待显示的 BGR 图像交给 native 库，由 RGA 硬件完成缩放与格式转换后写入当前非活跃缓冲区，再通过 DRM 翻页机制原子切换到活跃状态。由于翻页操作等待垂直同步且底层调用期间释放 GIL，显示线程之外的采集和推理线程得以并行执行，确保 1080p 60fps 稳定输出。

core/postprocess.py：YOLO 后处理

功能：NumPy 实现的 YOLOv5 特征图解码与 NMS 后处理。

YOLOPostProcessor 类接收模型三个输出特征图，通过 sigmoid 处理横纵坐标偏移和置信度，应用锚框解码公式还原为实际坐标。解码输出包含边界框、置信度、四个角点坐标和类别概率。使用网格坐标缓存避免重复创建。NMS 通过 OpenCV 和 C++ 后端加速执行。坐标还原函数将检测结果从预处理坐标系映射回原始图像坐标系。

core/serial_io.py：串口通信

功能：通过 UART 串口接收 FPGA 发送的车牌四角坐标数据。解析过程为逐字节扫描帧头，读取 16 字节数据后解析为 8 个无符号 16 位整数值，重组为 4×2 的坐标矩阵供识别线程裁剪使用。

core/fpga_io.py：FPGA UDP 视频流接收

功能：独立进程接收 FPGA 通过 UDP 发送的 RGB565 视频帧，处理粘包问题，将最新帧放入共享队列。

参数	数值	说明
FPGA 地址	192.168.1.10:61440	FPGA 设备 IP 和端口
RK3568 地址	192.168.1.102:61440	本地接收地址和端口
帧头标记	b\xf0\x5a\xa5\x0f	4 字节帧头
帧分辨率	640×480	每帧像素尺寸
帧数据大小	614400 字节	RGB565 数据载荷
头部偏移	8 字节	帧头+分辨率信息

触发机制：进程启动后向 FPGA 发送 1 字节触发信号，FPGA 收到后开始持续发送视频帧。

粘包处理算法：由于 UDP 是无连接协议，多个数据包可能合并到达。模块使用字节缓冲区，通过搜索帧头标记定位帧边界，当缓冲区中数据长度足够时截取完整帧数据。缓冲区过大时自动清空，防止内存泄漏。

RGB565 解码：将大端 RGB565 字节流通过 NumPy 视图重组后调用 OpenCV 转换为 BGR 图像。

车牌检测识别与行人检测模块

plate_det/plate_detector.py：车牌检测器

功能：使用 YOLOv5 模型通过 NPU 推理实现车牌检测。模型文件 plate_detect_320_int8.rknn。

detect()函数为检测入口，内部依次调用 preprocess_detect()完成 RGA 硬件等比缩放至 320×320 与 BGR 转 RGB，输出紧凑排列格式后送入 NPU 推理。推理输出的三个特征图由 detect_preprocessed()调用 YOLOPostProcessor 解码并经 NMS 过滤，最后坐标还原到原图尺度。每个检测结果包含边界框坐标、置信度、用于透视校正的四个角点坐标和类别标签。

detection_filter.py 提供 DetectionCandidateFilter 类，对检测候选按几何约束与置信度分级过滤：高置信候选直接放行，低置信候选需连续多帧命中才确认，有效抑制虚检。

plate_det/plate_recognizer.py：车牌识别器

功能：对裁剪后的车牌区域进行文字识别与颜色分类。模型文件为 plate_rec_color_int8.rknn。

recognize_rknn()为识别入口，接收 RKNN 模型和车牌裁剪图，执行 NPU 推理后分别走 CTC 解码和颜色分类两路输出。ctc_decode()取每步最大值，去除重复和空白符，对照 78 字符字典映射为最终文字，涵盖 31 个省份简称、字母 A 至 Z、数字 0 至 9，以及"学、警、港、澳、挂、使、领、民、航"等特殊字符。颜色分类经 softmax 取最大概率类别，共黑、蓝、绿、白、黄五类。

plate_det/plate_pipeline.py：处理流水线

功能：整合完整车牌处理流程。

process_plate()函数串联四个步骤：首先 four_point_transform()将车牌透视校正为正视图；然后调用 enhance_plate()自适应亮度增强；当检测类别为双层车牌时，调用 get_split_merge()将上下两层水平拼接为单行；最后送入 recognize_rknn()获得文字和颜色。

plate_det/plate_draw.py：可视化绘制

功能：在原图上绘制矩形框标记车牌位置，并在框旁叠加中文文字标签。

draw_result()函数接收原图和识别结果列表，遍历每个结果绘制红色矩形框。由于 OpenCV 原生不支持中文渲染，通过 cv2.freetype.createFreeType2()加载系统中文字体文件。

people_det/people_detector.py：行人闯红灯检测器

功能：基于 RKNN 的 YOLOv5 模型，检测行人、红绿灯、斑马线等多个目标。RKNNDetector 类封装完整推理流程：infer()方法对输入图像依次执行等比缩放、RGB 转换、NPU 推理，按类别分别执行 NMS，最后坐标还原。draw()方法对闯红灯目标用红色矩形框、普通行人用绿色矩形框绘制标注，返回标注图像与统计计数。

people_det/people_run.py：多模式运行器

功能：提供三种运行模式，均支持跳帧优化。

模式	函数	输入源	说明
FPGA 以太网	run_fpga()	UDP 视频流	RGB565 解码后检测，HDMI 输出
HDMI 采集卡	run_hdmi()	HDMI 采集卡	BGR 帧直接检测，HDMI 输出
本地视频	run_video()	视频文件	支持 HDMI 输出和窗口预览

性能与优化说明

决赛通过全链路硬件加速与异步流水线重构，在输入分辨率从 720p 提升至 1080p 的前提下，各项指标相比初赛均有大幅改善。下表为初赛与决赛的关键性能对比：

指标	初赛	区域决赛	提升幅度	优化来源
输入分辨率	1280×720@30fps	1920×1080@60fps	275%	MPP 硬件解码替代 OpenCV 软解
视频解码耗时	15.6ms	3.2ms	79.6%	MPP 硬件解码替代 OpenCV 软解
色彩转换耗时	7.8ms	1.8ms	76.7%	RGA 硬件色彩空间转换
检测前处理耗时	6.5ms	1.5ms	76.7%	RGA 完成缩放与格式转换
检测推理耗时	47.6 ms	42.3 ms	9.1%	NHWC 紧凑格式减少 NPU 数据搬运
端到端时延	93.8ms	63.1ms	33.3%	全链路硬件加速叠加
显示帧率	25～30fps	60fps	110%	显示与推理解耦，独立显示线程
RK 检测识别帧率	10.5fps	23.6fps	120%	采集、检测、识别、显示四线程异步
串口模式帧率	20.3fps	58.2fps	190%	跳过检测，显示/识别异步，硬解采集

初赛瓶颈分析：初赛的"解码→检测→识别→绘制→显示"为单线程串行流水线，OpenCV 软解 720p MJPEG 消耗约 15.6msCPU 时间，cv2 预处理额外消耗约 14.3ms，加上 NPU 推理约 58.8ms，单帧总耗时接近 94ms。显示写 Framebuffer 直接受推理节奏影响，实际输出仅 25～30fps 且存在画面撕裂。

区域决赛优化效果：区域决赛将解码和预处理全部下沉到 MPP/RGA 硬件，NPU 推理在独立线程异步执行，显示线程恒定 60fps，不受推理阻塞。串口模式下跳过检测环节，识别帧率接近显示帧率，充分释放了硬件加速管线的吞吐潜力。

测试条件：初赛数据源为 HDMI 采集 MJPEG 码流，分辨率 1280×720、帧率 30fps，采用 OpenCV 软件解码搭配 cv2 图像预处理，最终通过帧缓存输出画面；决赛数据源为 HDMI 采集 MJPEG 码流，分辨率 1920×1080、帧率 60fps，依托 MPP 硬件解码与 RGA 完成图像预处理，采用 DRM 结合 RGA 双缓冲方案实现显示。两组测试均配置独立线程执行检测与识别任务，调试模式下持续运行 60 秒，采集数据求取平均值。

1.3.3. 车牌检测模型的训练与底层优化

FPGA 端部署的是一套从零训练的角点检测网络，输入 320×320 的 RGB 图像，输出 40×40 网格上的 21 通道检测结果。之所以不直接采用 YOLO 等通用视觉检测模型，是基于 FPGA 端的硬件约束做出的权衡：YOLO 系列模型广泛使用的最大池化、FPN/PAN 多尺度特征融合、SiLU/Swish 非线性激活以及 NMS 后处理等操作，在 FPGA 上不仅缺乏直接的硬件映射路径，需消耗大量 LUT 资源模拟实现，而且涉及动态形状与条件分支，在 INT8 量化路径下精度损失严重；此外 YOLOv5s 级别的参数量与中间特征图规模很可能会超出可用存储上限。因此我们选择从零构建仅包含卷积、批归一化和 ReLU 激活层的检测网络，且不引入池化、全连接或跳跃连接等结构。其中批归一化可在模型导出阶段折叠进卷积权重，不占用额外计算资源；ReLU 仅通过符号位判断即可实现，几乎无硬件开销；所有卷积运算均可映射到同一套 INT8 MAC 阵列上时分复用，能够充分适配 FPGA 的硬件特性与资源约束。

网络结构

整体下采样率为 8 倍，分为四个阶段：

网络阶段	模块名称	通道变换	分辨率	模块核心说明
降采样	Stem 基础卷积块	3 → 16	160×160	初始卷积提取特征，压缩图像尺寸
降采样	DS1 深度可分离卷积块	16 → 24	80×80	轻量化卷积，降尺寸、升特征通道
降采样	DS2 深度可分离卷积块	24 → 32	40×40	继续下采样，进一步提高特征维度
局部特征提取	Local1~3 连续特征块	32 → 32	40×40	堆叠卷积块，挖掘精细化局部特征
上下文特征增强	Context1 上下文卷积块	32 → 48	20×20	下采样扩感受野，捕捉上下文信息
上下文特征增强	Context2a、2b 连续上下文块	48 → 48	20×20	强化深层特征，增强全局上下文关联
特征融合	特征融合模块	48 → 32	40×40	上采样还原尺寸，融合深浅层特征
预测输出	共享特征头	32 → 32	40×40	共享特征头，统一并精炼特征维度
预测输出	解耦预测头	32→1/8/4/8	40×40	四分支卷积，实现多任务解耦输出

解耦头输出 21 个通道分为四组：

通道范围	名称	含义
[0]	中心热图	车牌几何中心的高斯响应，峰值处对应检测位置
[1:9]	coarse 坐标	四个角点相对于选中中心格点的像素偏移
[9:13]	角点热图	四个角点各自的高斯响应，用于精化 coarse 预估
[13:21]	offset 偏移	四个角点在各自格点内的亚像素偏移，经 sigmoid 映射到(0,1)区间

网络总计算量约 27.9MMAC，在 148.5 MHz 时钟下，单帧推理耗时可满足 1080p@60fps 实时处理要求。网络内全部卷积为标准卷积或深度可分离卷积；BN 层可在量化流程中完成折叠融合，上采样采用无参数最近邻插值，分支融合运算采用定点饱和加法，上述算子均可在 FPGA 数据通路中映射为面积与时延特性可预测的硬件逻辑。

训练配置

训练基于 PyTorch 框架，使用 CUDA 加速。主要超参数如下：

参数	取值
训练轮数	100（FP32）/10（QAT）
批大小	16
优化器	AdamW
初始学习率	1×10^{-3}
权重衰减	1×10^{-4}
EMA 衰减率	0.998
梯度裁剪	10.0
混合精度	启用 AMP，初始 scale=1024

数据集使用 CCPD2020 数据集，按原始划分取训练集和验证集。每张图像通过文件名解析四个角点坐标，规范化为 TL→TR→BR→BL 顺时针顺序后，变换到 320×320 模型坐标空间。训练时对每张图像进行几何增强和光度增强以及 15%概率的负样本生成。为匹配 FPGA 端 HDMI 输入预处理后的数据分布，图像以 128 为中心值的 INT8 格式送入网络。

检测总损失由以下项构成：

损失项	权重	说明
中心点热力图	1.0	采用焦点损失，依据有效车牌目标数量做归一化处理
角点热力图	2.0	采用焦点损失，依据角点峰值数量独立完成归一化
粗粒度坐标	0.5	仅在中心点峰值位置计算平滑 L1 损失，除以步长 8，保证尺度与偏移量一致
角点偏移量	4.0	仅在角点峰值位置计算平滑 L1 损失

辅助几何损失

四项主损失中的 coarse 和 offset 均只在单点计算 Smooth-L1，缺乏对预测四边形整体形状的约束。极端情况下，四个角点各自的局部回归即使都准确，拼合后的四边形仍可能出现不合理的扭曲、倾斜或尺度偏差。为此在训练 5 轮后引入三种几何损失，仅在主损失已将角点拉至真值附近后才生效，避免早期梯度冲突。

1. 单应性网格损失：将预测的四个角点与真值角点分别拟合单应性变换矩阵，在车牌区域内采样 5×3 的网格点，计算两组网格点之间的 Smooth-L1 距离。该损失直接度量了两个四边形在透视投影下的空间对准程度，当预测的角点位置使车牌区域发生透视畸变时，内部网格点会偏离真值位置，损失增大。

2. 边缘对齐损失：计算预测四边形的每条边到真值对应边的点到直线距离，同时计算真值四边形的每条边到预测对应边的距离，取双向均值。这迫使预测的四条边紧贴真实车牌轮廓，抑制角点沿边切线方向的漂移，这种漂移在 Smooth-L1 距离上几乎不可见，但会导致车牌裁剪区域被拉伸或压缩。

3. 面积比损失：计算预测四边形与真值四边形的面积之比的绝对值对数。与逐点距离损失不同，它直接约束预测框的整体尺度，防止四个角点协同外扩或内缩导致车牌裁剪过大或缺失边缘信息。

三种几何损失通过一个从 0 到 1 的线性 ramp 系数加权后与主损失相加：ramp 在第 5 轮之前为 0，第 5 至 20 轮线性增长至 1，之后保持为 1。这样保证了前期训练由主损失主导角点的粗略定位，几何约束仅在角点收敛到合理区间后才介入精细化。

OCR 一致性损失

车牌检测的最终目标是保证裁剪出的车牌区域能够被识别模块正确读取。Smooth-L1 和几何损失均以像素空间的坐标误差为优化目标，但像素级对齐并不等价于最优的识别效果，例如角点略向内偏移可能导致首字符边缘被裁切，而这种裁切在坐标误差上可能仅有 2-3 像素的损失差异，但会导致识别模型识别错误。

为解决这一问题，训练时引入一个冻结的 CRNN-CTC 车牌识别网络作为教师。在每个包含车牌的批次中，将预测的四个角点通过单应性变换把原图的车牌区域矫正为 168×48 的标准矩形，同时用真值角点做同样的矫正。两路矫正结果分别送入教师网络，计算其 CTC 解码前 logits 分布的 KL 散度。该信号引导检测网络朝向裁切出的图像更利于字符识别的方向收敛。

三类几何损失和 OCR 损失的权重分别为 1.0、0.25、0.1、0.1，由 ramp 系数统一缩放后加入总损失。

模型的量化与部署

FPGA 端模型的部署流程分为三个阶段：FP32 训练、PTQ 标定、QAT 微调。

第一阶段：FP32 训练

如上文所述，在浮点精度下完成训练。保存 EMA 影子权重用于后续量化。

第二阶段：离线量化标定（PTQ）

将训练完成的模型切换为验证推理模式，完成批归一化层折叠处理后，选取 2048 张训练集图像作为标定数据，通过 99.9 分位数统计方式，计算模型每一层激活值的数值动态范围。对网络权重与激活值采用逐通道对称量化策略，统一转换为 8 位整型数据，模型偏置参数则量化为 32 位整型数据。各层的量化乘法系数与 6 位移位参数，均通过输入缩放系数×权重缩放系数/输出缩放系数的固定公式计算生成。离线量化标定阶段得到的所有缩放系数，将在后续量化微调阶段固定不变，作为量化训练的初始标定基准参数。

第三阶段：量化微调训练（QAT）

基于离线量化得到的固定缩放系数，在浮点训练框架中仿真 FPGA 芯片的整数推理链路。针对每一个卷积-批归一化-激活组合模块，前向推理结果统一量化为 8 位整型：权重采用逐通道对称 8 位整型量化方式，激活值采用四舍五入取整规则并截断至 8 位整型有效数值范围。

量化运算完全贴合硬件逻辑，采用与寄存器传输级代码一致的 64 位整数乘积运算和有符号移位计算逻辑。反向传播过程中，通过直通估计器完成梯度回传，更新浮点模型参数。

模型粗预测分支在量化微调过程中，采用固定 0.75 的缩放系数。该系数可完整覆盖 CCPD2020 增强训练集中粗偏移量的全部数值动态范围，预留充足数值余量，有效规避 8 位整型量化溢出问题。

浮点训练阶段采用相对粗坐标语义规则，即粗坐标始终表征相对于选定中心格点的偏移量，将粗坐标数值约束在有限区间内，避免坐标绝对数值过大引发的量化截断误差。训练过程中，通过激活饱和率指标实时监控激活值饱和占比，保证量化噪声处于可控范围。

第四阶段：固件导出

待量化微调模型收敛后，统一导出网络各层的 8 位整型权重、32 位整型偏置、量化乘法系数、移位参数，以及 4 组 256 项 Sigmoid 查找表数据，生成十六进制固件文件。同时生成 100 组确定性标准校验样本，包含推理链路中所有中间张量的 8 位整型标准数值。

所有导出文件通过清单校验工具完成比特级精准校验：重新运行 Python 整数参考推理流程，逐张量比对标准校验样本数据，确保固件参数在 FPGA 硬件端的推理输出结果与训练仿真端的数值结果一致，实现软硬件推理精度对齐。

模型训练效果

FP32 模型训练效果：训练在 CUDA 环境下完成，使用单张 GPU。以下为 FP32 模型在 CCPD2020 验证集上的最优轮次指标：

指标	数值
精准率	99.3%
召回率	98.1%
平均 IoU	0.885
角点像素误差	1.38px
面积比第 95%位数	1.114

模型整体检测性能合格。检测精准率 99.3%、召回率 98.1%，模型误检数量少，车牌目标检出能力较强；平均 IoU 达到 0.885，检测框与真实车牌轮廓贴合度高。车牌角点像素误差 1.38px，四角定位精度高，能够为后续透视矫正、字符识别提供可靠输入。面积比 95%分位数为 1.114，绝大部分样本检测框尺度与真实目标偏差可控。

量化误差分析

检测头	余弦相似度	MAE	相对 L2	INT8 范围利用率
center_head	0.993	1.07	0.127	[-127, 17]
coarse_head	0.980	1.53	0.236	[-58, 74]
corner_head	0.993	0.81	0.127	[-114, 17]
offset_head	0.967	0.27	0.26	[-98, 86]

余弦相似度衡量 FP32 与 INT8 输出在方向上的相似程度，越接近 1 表示量化损失越小。四个检测头均在 **0.96** 以上，center 和 corner 两个检测头达到 **0.993**，说明量化对关键点定位的影响很小。coarse_head 的 MAE 最大，主要因为其固定量化步长 0.75 px/LSB，每个 LSB 的舍入即为 0.75 像素误差。但在实际解码中 coarse 仅作为粗估，后续角点热图的 7×7 局部搜索和 offset 亚像素补偿可以有效修正。offset_head 的余弦相似度最低，但因其输出值域很小，MAE 仅 0.27，对最终坐标合成的绝对贡献有限，误差可以接受。

饱和率监控在训练过程中跟踪每层输出超出 INT8 范围的比例。阈值最高的层为 shared_head，有约 5.4%的激活值触及饱和边界，其余各层均在 1%以下，表明 INT8 动态范围分配合理。

固件导出与比特级精准校验

QAT 收敛后导出 FPGA 部署包，包含权重、偏置、requant 参数、sigmoid 查找表及 RTL 配置文件。导出阶段生成 100 组 golden 样本做比特级校验：两轮独立整数推理和两轮独立解码结果逐 tensor 一致，与训练端 QAT 前向输出 SHA-256 匹配。后续 Verilator 仿真中 RTL 输出与 golden 样本完全一致，确认了从训练到 FPGA 部署的数值一致性。

1.3.4 FPGA 端算法介绍

1. 128×128 的 6 层 CNN

功能介绍：在 1920×1080 HDMI 实时视频中检测车牌区域。系统完成 HDMI 图像采集、Gamma 校正、图像缩放、INT8 CNN 推理、车牌矩形框解码、坐标恢复、绿色框叠加、DDR3 视频缓存和 HDMI 显示，同时可以通过 UART 输出车牌矩形的四个角点坐标，实现车牌位置检测和实时可视化。

实现逻辑：HDMI 输入图像经过 Gamma 校正并转换为 RGB565，随后分为实时视频通路和 CNN 检测通路。视频通路始终连续传输当前摄像头画面；CNN 通路在允许采集时将一帧图像缩放为 128×72，通过上下 Letterbox 填充形成 128×128 的 INT8 输入，然后由六层共享卷积引擎依次完成特征提取，生成 8×8×5 检测特征图。系统遍历 64 个网格寻找最大目标响应，解码最佳网格的中心偏移和宽高参数，生成 128 坐标空间中的矩形边界，去除 Letterbox 填充并按 15 倍比例映射回 1920×1080。CNN 计算期间，视频通路仍将未画框的原始画面连续写入 DDR；检测完成后，解码结果作为当前最新矩形坐标，用于判断随后到达的实时视频像素是否位于矩形边缘，边缘像素被替换为绿色，其余像素保持原图，然后写入 DDR 并通过 HDMI 显示。当检测结果无效时，系统不叠加矩形框，显示原始视频；检测结果更新时，矩形坐标同时通过 UART 发送。

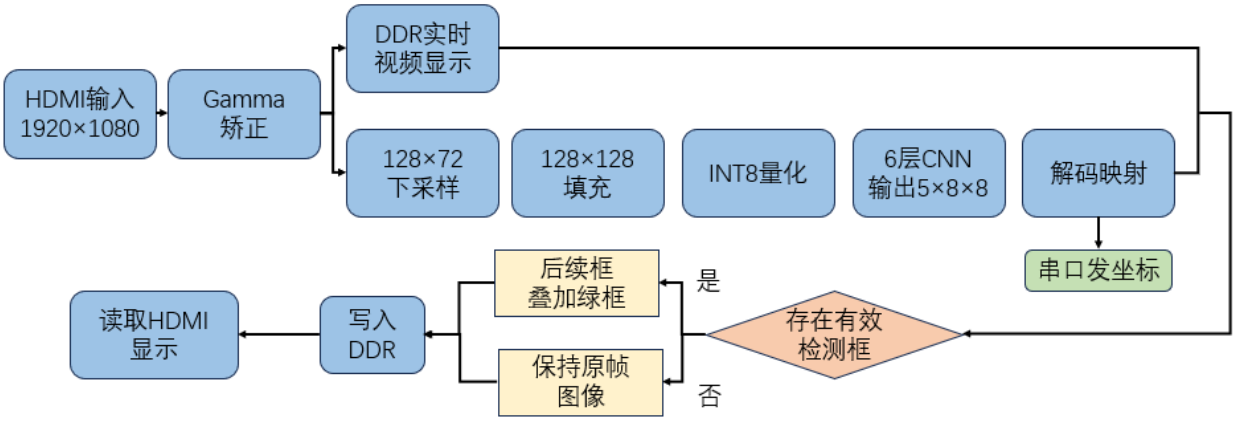


图 21 车牌识别顶层流程图

网络结构：

层	输入	卷积参数	输出	作用
输入	3×128×128	—	3×128×128	RGB 三通道 INT8 图像
Conv0	3×128×128	3×3, s=2, p=1	16×64×64	提取基础特征并下采样
Conv1	16×64×64	3×3, s=2, p=1	32×32×32	提取更高层特征并下采样
Conv2	32×32×32	3×3, s=1, p=1	32×32×32	深化特征，尺寸不变
Conv3	32×32×32	3×3, s=2, p=1	64×16×16	增加通道并下采样
Conv4	64×16×16	3×3, s=2, p=1	64×8×8	生成最终空间特征
Conv5	64×8×8	1×1, s=1, p=0	5×8×8	检测头

表 6 神经网络完整结构

FPGA 工作流程：

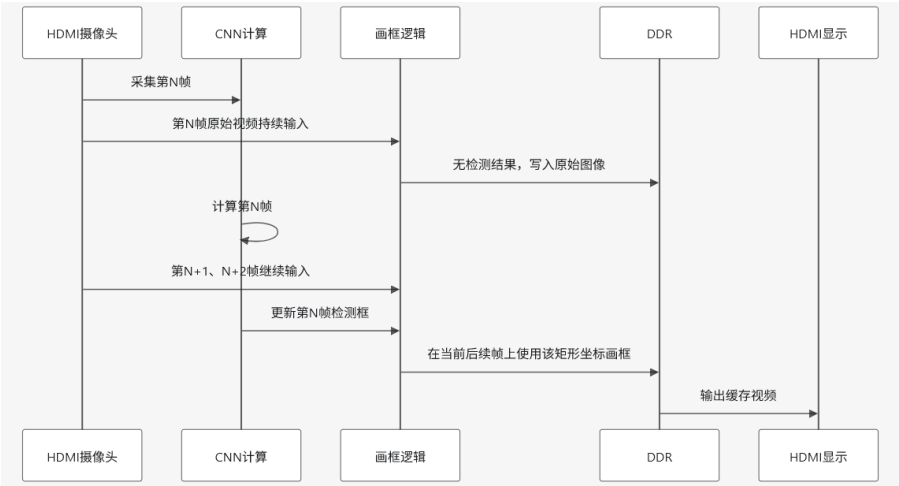


图 22 FPGA 工作流程图

模块算法介绍

1) 图像预处理模块

gamma_lookuptable

模块功能：对单个 8 位颜色通道执行 Gamma 查找表校正。使用一个 256 项 case 查找表同时将像素有效信号延迟到与 Gamma 输出数据对齐。

信号	位宽	说明
video_clk	1	HDMI 输入像素时钟
video_data	8	单个颜色通道数据
video_de	1	输入像素有效
gamma_data	8	Gamma 校正后的通道数据
gamma_de	1	输出数据有效

cnn_downscale_stream

模块功能：将 1920×1080 RGB565 视频流转换成 128×128×3 的 INT8 CNN 输入张量。模块首先将整个输入存储填充为固定 Padding 量化值，然后通过 dsmap_col.mem 和 dsmap_row.mem 判断当前原图像素是否属于采样点，将 1920×1080 图像缩放为 128×72；选中的 RGB565 像素被扩展成近似 8 位 R、G、B 通道，通过 quant8 函数完成定点乘法、移位和 INT8 限幅，并写入上下各保留 28 行填充区域的 CHW 存储中，全部有效像素写入完成后产生 frame_done，若 RGB 三个通道尚未写完就收到下一个采样像素，则置位 overflow。

模块流程图：

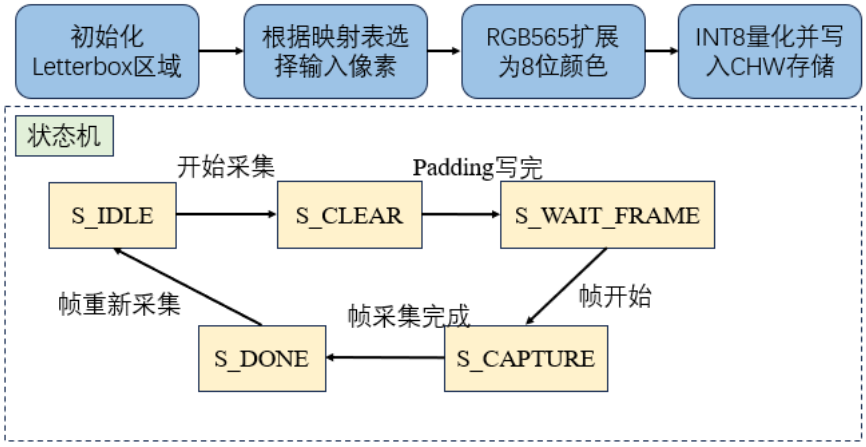


图 23 下采样模块流程图

主要状态机：

状态	功能
S_IDLE	等待帧采集启动
S_CLEAR	将整个输入张量写为 Padding 值
S_WAIT_FRAME	等待有效输入帧
S_CAPTURE	选择、量化并写入 RGB 像素
S_DONE	当前输入张量采集完成

输入输出接口：

信号	位宽	说明
clk	1	HDMI 输入像素时钟
rst_n	1	低有效复位
capture_enable	1	允许采集新图像
vsync	1	输入帧同步
de	1	输入像素有效
rgb565	16	Gamma 处理后的 RGB565
cur_x/cur_y	12+12	当前原图像素坐标
out_we	1	CNN 输入存储写使能
out_addr	17	CNN 输入存储地址
out_data	8	INT8 图像数据
frame_done	1	128×128 输入采集完成
overflow	1	RGB 三通道写入不及时导致溢出

2) CNN 控制模块

plate_cnn_accel

模块功能：该模块是整个车牌 CNN 算法的封装模块，负责连接 cnn_downscale_stream、cnn_top、box_decode、box_scale。并控制“采集—推理—解码—映射—更新结果”的完整周期。模块在 capture_enable 有效时允许预处理模块采集一帧，采集完成后暂停新图像采集并启动 cnn_top；CNN 完成后锁存最佳网格、目标响应和 tx/ty/tw/th 参数，再启动 box_decode 生成 128 空间矩形，使用 box_scale 映射到 1920×1080，并在 S_EXPAND 状态向外扩展矩形边界、执行原图限幅和目标阈值判断，最后更新 box_x1/y1/x2/y2、box_valid 和 box_update，然后重新允许采集下一帧。

模块流程图：

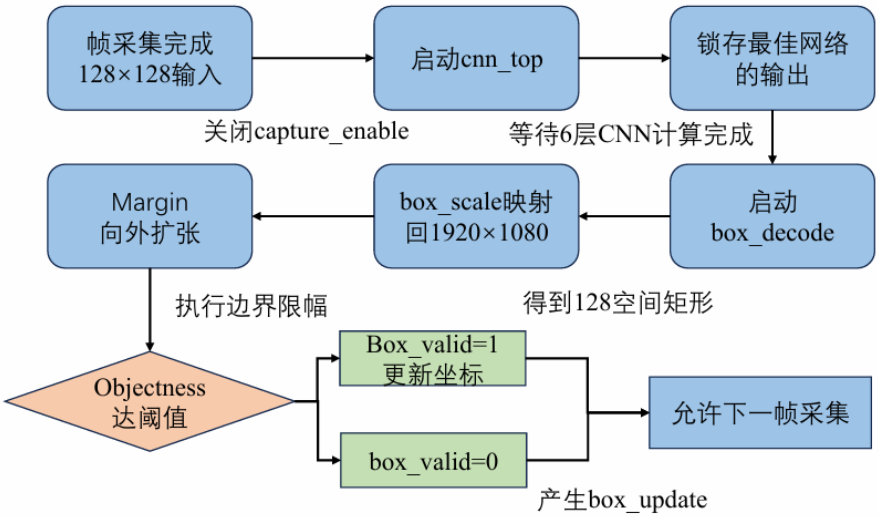


图 24 CNN 外层控制模块流程图

主要状态机：

状态	功能
S_IDLE	等待 128×128 输入采集完成
S_REARM	CNN 关闭时重新允许采集
S_START	启动 CNN
S_WAIT	等待六层 CNN 完成
S_DECODE	等待矩形参数解码
S_SCALE	锁存原图坐标
S_EXPAND	Margin 扩展并更新最终结果

输入输出接口：

信号	位宽	说明
clk/rst_n	1	时钟和复位
vsync/de	1	输入视频控制信号
rgb565	16	CNN 预处理图像输入
cur_x/cur_y	12+12	当前原图坐标
box_x1/box_y1	12+12	矩形左上角
box_x2/box_y2	12+12	矩形右下角
p0～p3	每坐标 12 位	由矩形组合出的四个角
box_valid	1	当前检测结果有效
box_update	1	检测结果更新脉冲
capture_done	1	图像采集完成

3) CNN 计算模块

cnn_top（CNN 推理顶层）

模块功能：六层 CNN 的网络级调度和存储管理模块，负责接收 128×128×3 INT8 输入、加载权重和 Bias、配置每一层卷积参数、控制共享卷积引擎以及启动最终 Head 搜索。模块内部设置两个 65536 字节的特征存储 bufA 和 bufB，各层计算结果在二者之间 Ping-Pong 切换；网络参数根据当前 layer 从 net_config.vh 中读取，状态机依次启动 conv_engine 完成 0～5 层计算，前五层结果写入 INT8 特征 Buffer，最后一层的 8×8×5 原始累加结果写入 320 项的 head_mem，全部层完成后启动 argmax_box，最终输出最大响应网格、objectness 以及对应的四个矩形参数。

Ping-Pong 特征存储示意：

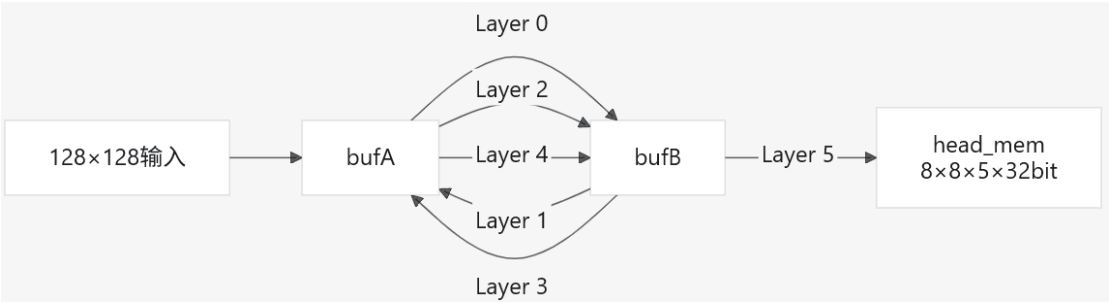


图 25 特征存储示意图

六层网络配置参数：当前层号 layer 通过 net_config.vh 查询配置。

参数	含义
cfg_IC/IH/IW	输入通道、高、宽
cfg_OC/OH/OW	输出通道、高、宽
cfg_KH/KW	卷积核尺寸
cfg_STRIDE	卷积步长
cfg_PAD	Padding
cfg_M0/SHIFT	Requant 参数
cfg_RELU	ReLU 使能
cfg_ISHEAD	当前层是否为 Head
cfg_WBASE	权重起始地址
cfg_BBASE	Bias 起始地址

模块流程图：

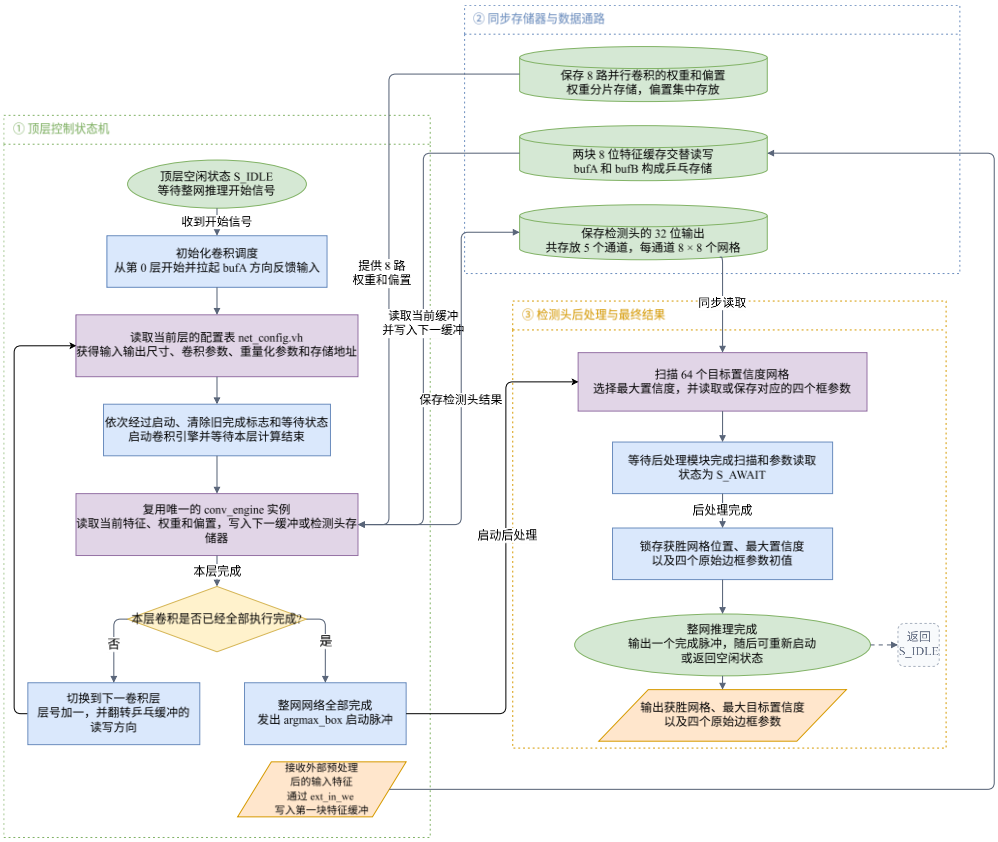


图 26 CNN 推理顶层模块流程图

主要状态机：

状态	功能
S_IDLE	等待 CNN 启动
S_LSTART	向卷积引擎发送启动脉冲
S_LCLEAR	等待卷积引擎离开上次完成状态
S_LWAIT	等待当前层计算完成
S_NEXT	判断进入下一层或 Head 搜索
S_ASTART	启动 argmax_box
S_AWAIT	等待最大网格搜索完成
S_DONE	输出 done 后返回空闲

输入输出接口：

接口	位宽	功能
clk	1	CNN 工作时钟
rst_n	1	低有效复位
start	1	启动一次推理
ext_in_we	1	外部输入写使能
ext_in_addr	17	CNN 输入地址
ext_in_data	8 signed	CNN INT8 输入数据
done	1	CNN 推理完成
win_row	3	最佳检测网格行号
win_col	3	最佳检测网格列号
obj_acc	32 signed	最佳目标置信度累加结果
box0	32 signed	目标框参数 tx
box1	32 signed	目标框参数 ty
box2	32 signed	目标框参数 tw
box3	32 signed	目标框参数 th

conv_engine

模块功能：六层网络共用的实际卷积计算核心，负责完成输入特征读取、卷积窗口遍历、Padding 判断、8 路并行乘法、32 位累加、Bias 相加、Requant、ReLU 和结果写回。模块按照输出通道组、输出坐标、输入通道和卷积核坐标逐层循环，每次读取一个 INT8 输入值并同时与 8 个输出通道的权重相乘，分别累加到 acc0～acc7；当前输出像素的全部输入通道和卷积核元素计算完成后，模块加载对应 Bias，通过多个状态将宽位定点乘法拆分、合并、舍入和移位，再进行 ReLU 与 INT8 限幅，普通卷积层通过 fout_* 接口写回 8 位特征，检测 Head 层通过 hout_* 接口输出 32 位“卷积累加结果+Bias”，跳过 Requant、ReLU 和 INT8 限幅。卷积计算呈循环结构，关系为：

```
输出通道组 oc_base
  └─ 输出坐标 oy、ox
       └─ 输入通道 ic
            └─ 卷积核 ky、kx
                 └─ 8 路乘法和累加
for oc_base = 0; oc_base < OC; oc_base += 8
    for oy
        for ox
            acc[0:7] = 0
            for ic
                for ky
                    for kx
                        input = feature[ic][iy][ix]
                        acc[0:7] += input × weight[0:7]
            output[0:7] = requant(acc[0:7] + bias[0:7])
```

Padding 处理：

根据输出位置、步长和 Padding 计算输入坐标：

```
input_y = output_y×STRIDE + ky-PAD
input_x = output_x×STRIDE + kx-PAD
如果越界：input_value = 0，否则通过 fin_addr 读取输入特征。
```

8 路并行乘法：

S_MUL 状态并行计算 8 路 INT8 乘法，并将结果保存为 16 位 mul0～mul7：

```
mul0 = w_data0 × in_val;
...
mul7 = w_data7 × in_val;
```

当 ic、ky、kx 全部到达最后一个元素，说明当前输出像素的卷积完成，进入 Bias 和 Requant 阶段。

Requant 处理：

卷积引擎内部使用多个状态完成宽位乘法和量化：

状态	功能
S_BIAS	加载 Bias 并形成量化输入
S_RQMUL_LO	计算乘法低部分
S_RQMUL_HI	计算乘法高部分
S_RQSUM	合并定点乘法结果
S_RQROUND	加舍入偏置
S_RQSHIFT	算术右移
S_CLAMP	ReLU 和 INT8 饱和
S_WRITE	输出写回

量化关系近似为：

a = acc + bias
当 SHIFT > 0 时：
q_tmp = ((a × M0) + 2^(SHIFT-1)) >>> SHIFT
当 SHIFT = 0 时：
q_tmp = a × M0
启用 ReLU 时：
q_tmp < 0 → q_tmp = 0
最终饱和范围：
q = clamp(q_tmp, -127, 127)

普通层与 Head 层输出：

普通层：
a = acc + bias
q = saturate[-127,127]((a × M0 + 2^(SHIFT-1)) >>> SHIFT)
若 RELU=1，则所有负数置 0
Head 层：
hout = acc + bias
不执行 M0 乘法、右移、ReLU 或 INT8 限幅

主要函数：

pad_row_base_fn

计算 Padding 情况下输入特征的初始行地址。

sx_coord

将 13 位有符号坐标扩展成 19 位，用于有符号地址计算。

模块流程图：

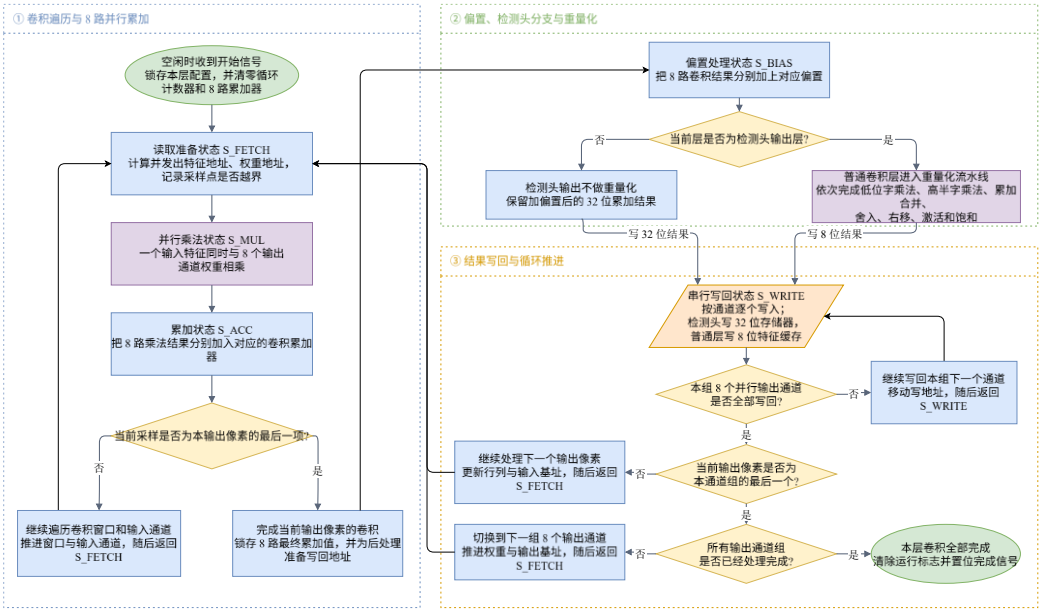


图 27 卷积计算引擎流程图

输入接口：

信号	位宽	说明
IC/IH/IW	11	输入张量尺寸
OC/OH/OW	11	输出张量尺寸
KH/KW	4	卷积核尺寸
STRIDE	4	步长
PAD	4	Padding
M0	32	Requant 乘数
SHIFT	6	Requant 右移
RELU	1	ReLU 使能
IS_HEAD	1	Head 层标志
W_BASE	18	权重起始地址
B_BASE	10	Bias 起始地址

特征接口：

信号	方向	说明
fin_addr	输出	输入特征读取地址
fin_data	输入	INT8 输入特征
fout_addr	输出	普通层输出地址
fout_data	输出	INT8 输出特征
fout_we	输出	普通层写使能

权重与 Bias 接口：

信号	方向	说明
w_addr	输出	8 路权重读取地址
w_data0～7	输入	8 路 INT8 权重
b_addr0～7	输出	8 路 Bias 地址
b_data0～7	输入	8 路 32 位 Bias

Head 接口：

信号	说明
hout_addr	Head 存储写地址
hout_data	32 位 Head 结果
hout_we	Head 写使能

控制接口：

信号	说明
start	启动当前卷积层
done	当前卷积层完成

argmax_box

模块功能：负责在 8×8×5 检测 Head 中寻找 objectness 最大的候选网格，并读取该网格对应的 tx/ty/tw/th 参数。模块先依次读取 head_mem 地址 0～63 的 64 个 objectness，将当前值与已记录最大值比较并保存最佳网格编号；扫描结束后，根据 Head 按通道连续存储的布局，分别读取 64+best_cell、128+best_cell、192+best_cell 和 256+best_cell，得到四个矩形参数，并将最佳网格编号拆分为 win_row 和 win_col，最终产生 done。

具体实现逻辑：

网格扫描：idx = 0～63

读取 head_mem[idx]

与 best 比较

更大则更新 best 和 best_cell

参数回读：

扫描完成后，按照四个阶段读取：

```
bphase=0 → box0=tx
bphase=1 → box1=ty
bphase=2 → box2=tw
bphase=3 → box3=th
最佳网格坐标：
win_row = best_cell[5:3];
win_col = best_cell[2:0];
```

模块流程图：

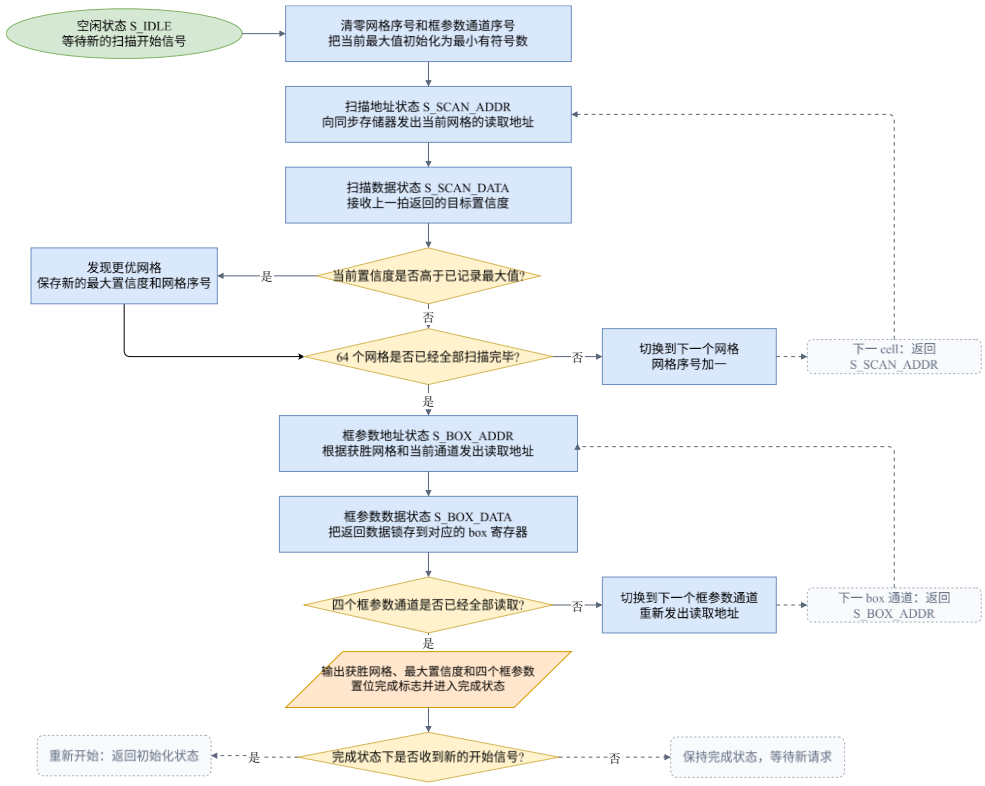


图 28 argmax_box 模块流程图

主要状态机：

状态	功能
S_IDLE	等待启动
S_SCAN_ADDR	设置 objectness 读取地址
S_SCAN_DATA	接收并比较响应值
S_BOX_ADDR	设置参数读取地址
S_BOX_DATA	接收 tx/ty/tw/th
S_DONE	输出完成

输入输出接口：

信号	位宽	说明
clk/rst_n	1	时钟和复位
start	1	启动搜索
h_data	32	Head 存储返回数据
h_addr	10	Head 存储读取地址
done	1	搜索完成
win_row/win_col	3+3	最佳网格
obj_acc	32	最大 objectness
box0～box3	32	tx、ty、tw、th

4) 检测结果解码

box_decode

模块功能：负责将最佳 8×8 网格坐标和 CNN 输出的 tx/ty/tw/th 转换成 128×128 空间中的矩形边界。模块首先通过 clip_acc 将 32 位累加结果限制到安全范围，再乘以定点解码系数 DEC_M，经过舍入和右移 DEC_S 得到 0～255 的 Sigmoid LUT 索引；LUT 输出用于计算最佳网格内的中心偏移以及矩形宽高，其中每个 8×8 网格在 128 空间中对应 16×16 区域，最后按照 x1=cx-w/2、y1=cy-h/2、x2=cx+w/2、y2=cy+h/2 生成矩形，并通过 clip128 将坐标限制在 0～128。

具体实现逻辑：

定点缩放：

将网络输出限制为 20 位，然后乘解码系数：

```
prod_tx = clip_acc(acc_tx)×DEC_M;
prod_ty = clip_acc(acc_ty)×DEC_M;
prod_tw = clip_acc(acc_tw)×DEC_M;
prod_th = clip_acc(acc_th)×DEC_M;
```

Sigmoid LUT：

乘法结果右移 DEC_S，加 128 形成 LUT 索引：

```
index = round(product / 2^DEC_S)+128
```

索引限制为 0～255，然后读取 sigmoid_lut.mem。

生成中心和宽高：

每个 8×8 网格在 128 空间中对应 16×16 区域：

```
cx = win_col×16 + sigmoid(tx);
cy = win_row×16 + sigmoid(ty);
```

宽高由 tw/th 的 LUT 结果产生：

```
bw = decode(tw)
bh = decode(th)
```

生成矩形边界：

```
x1 = cx-bw/2
y1 = cy-bh/2
x2 = cx+bw/2
y2 = cy+bh/2
```

最后通过 clip128 限制到 0～128。

主要函数：

clip_acc：将 32 位网络输出限制到 20 位范围。

lut_index：定点缩放并生成 0～255 的 LUT 索引。

clip128：将坐标限制到 128 空间。

模块流程图：

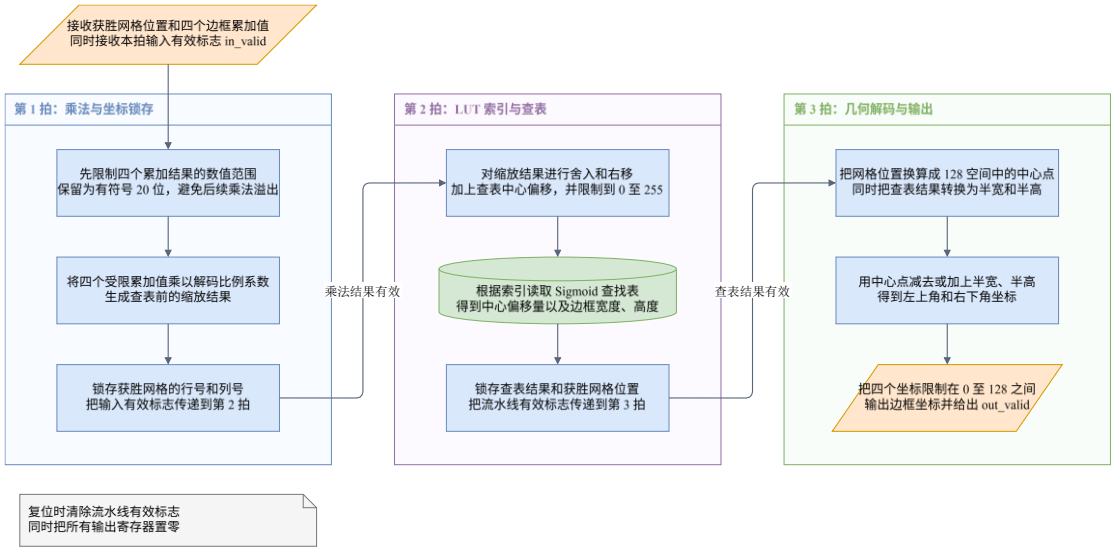


图 29 网格解码流程图

输入输出接口：

信号	位宽	说明
clk/rst_n	1	时钟和复位
in_valid	1	输入参数有效
win_row/win_col	3+3	最佳网格
acc_tx/acc_ty	32	中心偏移
acc_tw/acc_th	32	矩形宽高

box_scale

模块功能：负责将 128×128 Letterbox 空间中的矩形坐标恢复到 1920×1080 原始图像空间。由于原图被缩放成 128×72 后在上下各填充 28 行，因此 X 坐标直接乘以 1920/128=15，Y 坐标先减去 28 去除 Letterbox，再乘以 1080/72=15；模块通过 mapx 和 mapy 两个组合函数完成缩放和边界限幅，最终输出 12 位的原图左上角和右下角坐标。

实现逻辑：

X 方向没有 Padding：X = x×1920/128 = x×15

Y 方向先去除 28 行 Letterbox：

Y = (y-28)×1080/72

= (y-28)×15

输出再限制到原图边界。

模块流程图：

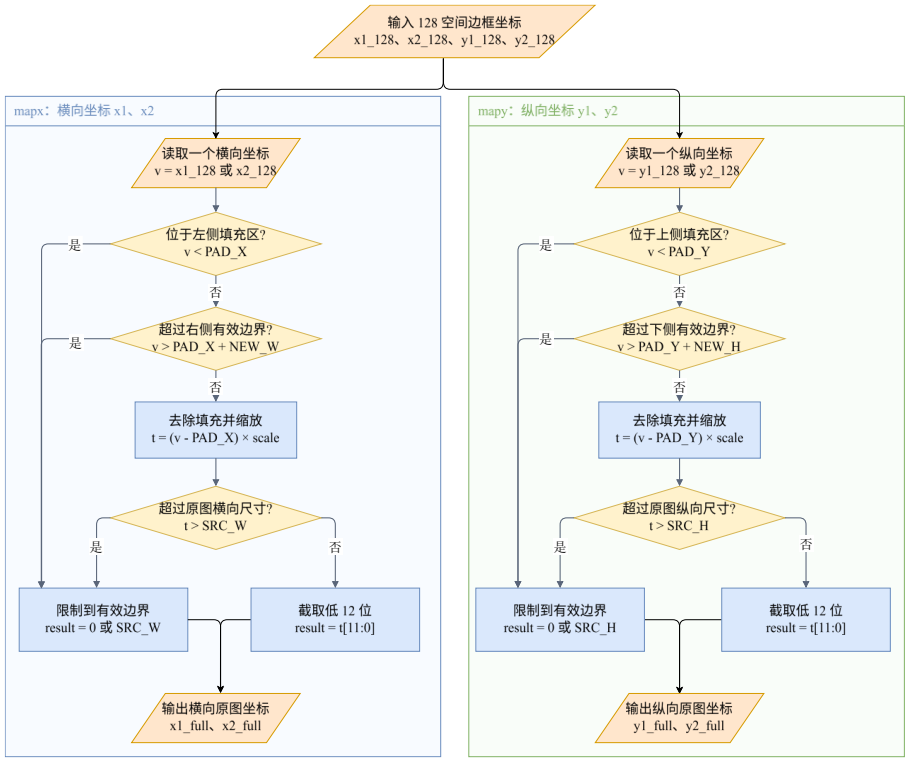


图 30 反映射模块流程图

输入输出接口：

信号	位宽	说明
x1_128/y1_128	8	128 空间左上角
x2_128/y2_128	8	128 空间右下角
x1_full/y1_full	12	原图左上角
x2_full/y2_full	12	原图右下角

5) UART 模块

plate_uart_tx

模块功能：当检测框更新时，将矩形四个角点坐标打包并通过 UART 发送。

实现逻辑：

触发条件：

cnn_box_update && cnn_valid，坐标由矩形生成：

TL=(x1,y1)

TR=(x2,y1)

BL=(x1,y2)

BR=(x2,y2)

UART 状态机依次发送数据包中的每个字节，每个字节按照：

输入输出接口：

信号	说明
clk/rst_n	UART 时钟和复位
send_trig	发送触发
p0_tl_x/y	左上角
p0_tr_x/y	右上角
p0_bl_x/y	左下角
p0_br_x/y	右下角
uart_tx	串口数据
tx_busy	UART 正在发送

2. 320×320 的 23 层 CNN

一、 总体架构变化

相较于第一个（128×128 的 6 层 CNN），两个网络版本的主要差异如下：第一个检测系统输出轴对齐矩形框，第二代可输出任意透视四边形的四个真实角点，两个版本网络的核心差异如下：

项目	128×128	320×320
检测类型	水平矩形框	任意透视四边形
网络输入分辨率	128×128	320×320
网络层数	6 层普通卷积	23 层（含深度/逐点/特征融合）
网络结构	全普通卷积	标准卷积 + Depthwise + Pointwise + 融合
Head 输出	5 通道（objectness + 矩形参数）	4 分支：Center(1ch)、Coarse(8ch)、Corner(4ch)、Offset(8ch)
画框位置	DDR 写入前覆盖原图	DDR 读出后叠加，无损原始帧
结果对齐	使用最近一次检测结果	按 frame_id 精确匹配
帧缓存	简单双帧计数逻辑	双 Bank Ping-Pong + 4 槽帧环管理
预处理	映射表抽样、128×72 填充	6×6 盒式下采样、320×180 、Letterbox
卷积引擎	单一普通卷积	三种模式（Standard / Depthwise / Pointwise）
坐标校验	仅置信度	凸性 + Shoelace 面积 + 边界 + 顺序

表 7 两版输入神经网络对比

二、 CNN 网络结构变化

1) 128×128 输入的 CNN

6 层普通卷积，配置在 net_config.vh 中：

层	操作	输入尺寸	输出尺寸
0	3×3，Stride=2	128×128×3	64×64×16
1	3×3，Stride=2	64×64×16	32×32×32
2	3×3，Stride=1	32×32×32	32×32×32
3	3×3，Stride=2	32×32×32	16×16×64
4	3×3，Stride=2	16×16×64	8×8×64
5	1×1 Head	8×8×64	8×8×5

表 8 神经网络架构

Head 每个网格输出 5 个量：objectness、tx、ty、tw、th。argmax_box 先在 8×8 网格的 objectness 通道中寻找最大响应位置，再读取该位置的 4 个框参数；box_decode 将 tx/ty 解码为框中心 cx/cy，将 tw/th 解码为框宽高 bw/bh，随后计算矩形左上角(x1,y1)和右下角(x2,y2)。四个角点 p0~p3 再由这两个对角点组合得到，只能输出轴对齐矩形。

2) 320×320 输入的 CNN

23 层轻量化结构，定义在 cnn_defs.vh 中，由 cnn_scheduler.v 逐层调度执行。主干尺寸变化如下：

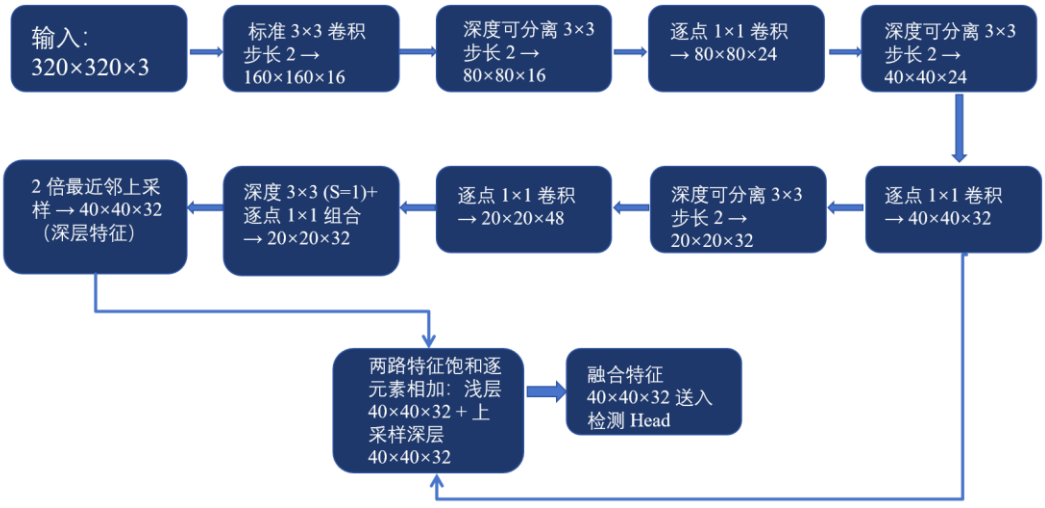


图 31 320 输入神经网络主干结构

最终检测 Head 为 4 分支解耦结构：

Head 分支	输出尺寸	作用
Center	40×40×1	车牌中心响应
Coarse	40×40×8	四个角点的粗坐标（每角点 x/y 各一通道）
Corner	40×40×4	四个角点的置信度
Offset	40×40×8	四个角点的精细偏移（每角点 x/y 各一通道）

四分支均在 40×40 网格上操作，直接预测透视四边形的四个真实角点，不再依赖矩形拼合。

三、 预处理逻辑变化

1) 128×128 输入

旧预处理模块 cnn_downscale_stream.v 处理链：

1920×1080 RGB888 → Gamma LUT → RGB565 → 映射表抽样 → 128×72 RGB → 上下填充 28 行 → 128×128 INT8 CHW

通过 dsmap_col.mem 和 dsmap_row.mem 选择输入像素。

输出为 CHW 排布，每个选中像素需连续写 3 个通道。

先清空整个输入张量为 padding 值，若新像素到达时上一个像素未写完，则 stream_overflow 报错。

2) 320×320 输入

预处理模块 cnn_letterbox_stream.v 处理链：

1920×1080 RGB888 → 6×6 盒式下采样 → 320×180 → Letterbox（上下各 70 行 0）→ 320×320 INT8 RGB 流

输入从 RGB565 改为直接使用 RGB888。

不再依赖行列映射表 .mem，完全固定几何变换。

输出为紧凑 RGB 字节流，每像素三字节连续输出，非 CHW 排布。

具有独立流式握手（output_ready 反压）和错误检测。

后接 cnn_input_pingpong_store 双 Bank 存储，可在采集下一帧时计算上一帧。

整体流程图：

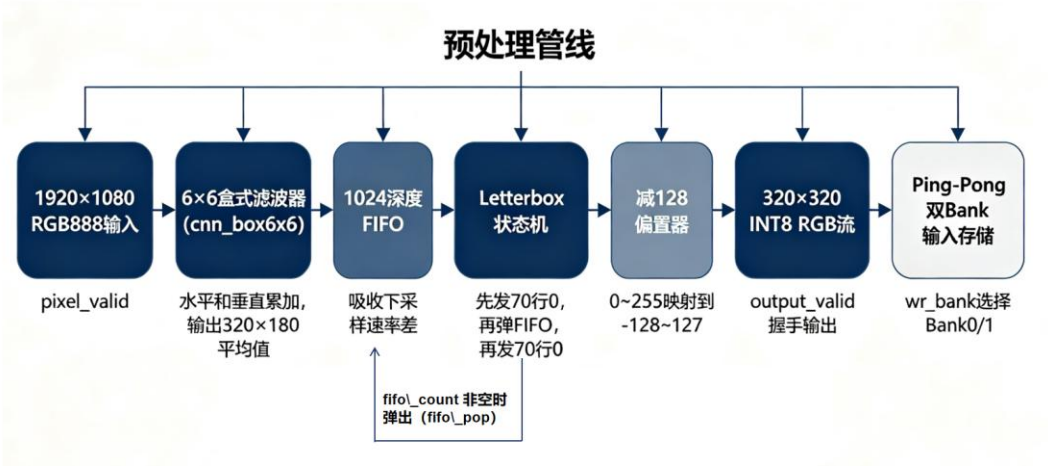


图 32 320 输入神经网络流程图

输入输出接口：

端口	方向	位宽	说明
pixclk_in	输入	1	视频像素时钟
frame_start	输入	1	帧同步脉冲
pixel_valid	输入	1	输入像素有效
pixel_r/g/b	输入	8	RGB888 分量
output_ready	输入	1	下游反压（来自 CNN 引擎）
output_valid	输出	1	输出 INT8 像素有效
output_r/g/b	输出	8	减 128 后的 INT8 值
frame_done	输出	1	一帧发送完成
sticky_error	输出	1	错误标志

关键代码及讲解：

1）6×6 下采样的除法舍入（cnn_box6x6.v）

```
wire [13:0] rounded_r = (vertical_complete_r_q + 14'd18) / 14'd36;
```

+18 为四舍五入偏置（36 的一半），除以 36 用组合除法器实现。最大输入 9180+18=9198，14 位宽足够，商在 0~255 之间。

2）FIFO 空满控制（cnn_letterbox_stream.v）

```
case ({fifo_push, fifo_pop})
    2'b10: fifo_count <= fifo_count + 11'd1;
    2'b01: fifo_count <= fifo_count - 11'd1;
    default: fifo_count <= fifo_count;
endcase
```

fifo_count 同时用于空判断（fifo_count != 0）和满判断（fifo_count != 1024）。若 FIFO 满时 box_valid 仍然拉高的话，则锁存 sticky_error。

3）减 128 偏置映射为 INT8（输出逻辑）

```
output_r <= fifo[read_pointer][23:16] - 8'd128;
output_g <= fifo[read_pointer][15:8]  - 8'd128;
output_b <= fifo[read_pointer][7:0]   - 8'd128;
```

将 0~255 的 RGB 均值映射到-128~127，与后续 CNN 的 INT8 有符号数格式匹配。减 128 操作在输出路径上组合完成，无额外延迟。

改进：128×128 输入映射表抽样后，每个选中像素需用 3 拍依次写入 CHW 三个通道；若下一个选中像素在 pending 尚未清空时到达，就会触发 overflow。320×320 采用固定 6×6 窗口下采样和可反压 FIFO，数据产生速率可预测。

四、 CNN 计算引擎变化

1) 128×128 输入

引擎 conv_engine.v 仅支持普通卷积，6 层共享同一计算引擎，特征使用简单 ping-pong 存储，量化由独立 requant_unit.v 完成，Head 特征单独写入 head_mem。

2) 320×320 输入

引擎 cnn_conv_engine.v 支持三种执行模式，由 mode 端口选择：

- **Mode 0（Standard）**：标准卷积（3×3 或 1×1）
- **Mode 1（Depthwise）**：深度卷积，每个输入通道独立卷积
- **Mode 2（Pointwise）**：逐点卷积（1×1）

内部子模块包括：

- 32 路 INT8 MAC 阵列（cnn_mac32 + cnn_mul8）
- 权重 Lane Bank（32 个独立小 RAM，按输出通道横向展开）
- Depthwise 行缓存（18 行×256 位）
- Pointwise 快速通路（三种子策略）
- Standard STEM 流式通路
- Bias 读取 ROM
- 8 级 Requant 流水线（cnn_requant）
- ReLU 和 INT8 饱和
- 32 字节特征打包输出

模块流程图：

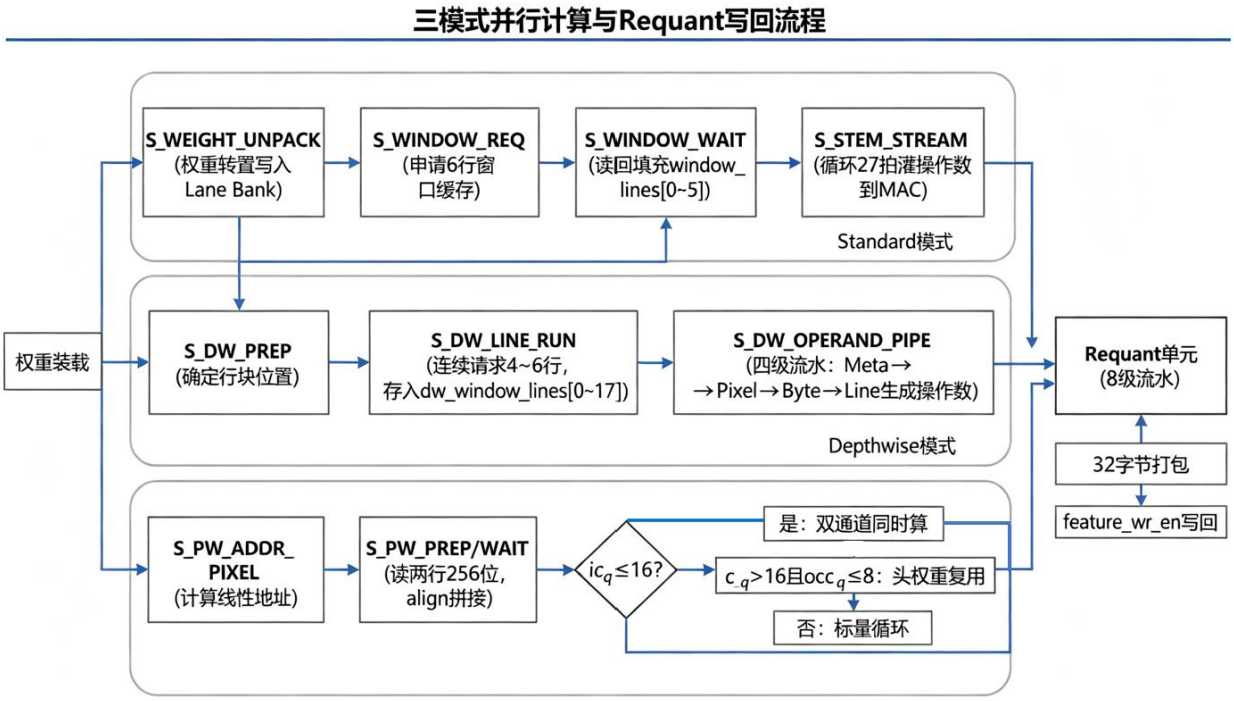


图 33 320 输入计算引擎流程图

输入输出接口：（cnn_conv_engine.v——仅列模式控制与存储接口）

端口	方向	位宽	说明
mode	输入	2	0:Standard, 1:Depthwise, 2:Pointwise
input_h/w/c, output_h/w/c	输入	9	当前层张量尺寸
kernel, stride, padding	输入	4/2/2	卷积超参数
groups, relu	输入	9/1	分组数和 ReLU 使能
feature_rd_*	输出/输入	19/256/1	特征存储读接口
feature_wr_*	输出	19/256/1	特征存储写接口
weight_rd_*	输出	10/1	权重存储读接口
busy, layer_done	输出	1	引擎状态
out_byte_valid/address/value	输出	1/19/8	逐字节 INT8 输出流

关键代码及讲解：

1）状态机根据 mode 选择操作数源

```
case (mode_q)
    MODE_STANDARD: if (stem_operand_push) begin
        mac_launch_a <= stem_operand_a;
        mac_launch_b <= stem_operand_b;
        mac_launch_valid <= 1;
    end
    MODE_DEPTHWISE: if (dw_operand_push) begin
        mac_launch_a <= dw_operand_a;
        mac_launch_b <= dw_operand_b;
        mac_launch_valid <= 1;
    end
    MODE_POINTWISE: if (pw_operand_push) begin
        mac_launch_a <= pw_operand_a;
        mac_launch_b <= pw_operand_b;
        mac_launch_valid <= 1;
    end
    default: mac_launch_valid <= 0; endcase
```

三个模式的操作数生成逻辑完全独立，通过 mac_launch_valid 统一控制 MAC 输入时机。无额外切换延迟。

2）Depthwise 模式的行缓存独热写入

```
generate
    for (dw_commit_bank = 0; dw_commit_bank < 18; dw_commit_bank =
dw_commit_bank + 1) begin : g_dw_commit_bank
        always @(posedge clk) begin
            if (dw_commit_valid_q && dw_commit_select_q[dw_commit_bank])
begin
                dw_window_lines[dw_commit_bank] <= dw_commit_data_q;
                dw_window_bases[dw_commit_bank] <= dw_commit_line_q;
            end
        end
    end
endgenerate
```

dw_commit_select_q 为 18 位独热码，单时钟只写入一行。综合工具将其映射为分布式寄存器而非集中式 RAM，避免多端口写冲突。

3）Standard 模式的操作数填充

```
for (index = 0; index < 32; index = index + 1) begin
if (index < 16 || pair_has_second) begin
stem_operand_a[index*8 +: 8] <= (index < 16) ? stem_cached_scalar0 :
stem_cached_scalar1;
stem_operand_b[index*8 +: 8] <= weight_lane_read_flat[index*8 +: 8];
end else begin
stem_operand_a[index*8 +: 8] <= 0;
stem_operand_b[index*8 +: 8] <= 0;
end
end
```

前 16 个 lane 使用第一个水平像素，后 16 个 lane 使用第二个水平像素（pair_has_second 为真时），一个 MAC 周期同时计算两个水平位置的部分和。

4）逐点卷积双通道优化（利用 MAC 半部分和）

```
if (mac_half_sums_valid) begin
    depth_acc[pw_pair_received*2] <= mac_lower_half_sum;
    if (pw_pair_received * 2 + 1 < oc_q)
        depth_acc[pw_pair_received*2+1] <= mac_upper_half_sum;
    pw_pair_received <= pw_pair_received + 1;
    if ((pw_pair_received + 1) * 2 >= oc_q) pw_run_phase <= 2;
end
```

当 ic_q≤16 时，利用 MAC 的 out_lower_half_sum（前 16 路）和 out_upper_half_sum（后 16 路）同时计算两个输出通道，MAC 运算次数减半。

改进：128×128 单一模式无法高效处理 Depthwise 和 Pointwise 层，新版通过状态机分级和专用行缓存/快速通道，使三种模式共享同一套 MAC 阵列，硬件复用率大幅提升。

五、 特征融合逻辑（新增）

1) 128×128 输入

无多尺度特征融合，各层特征独立使用，深层语义信息与浅层空间位置信息未结合。

2) 320×320 输入

调度器 cnn_scheduler.v 在第 17 层（20×20×32 深层特征）输出后执行 FUSION 状态：

1. 最近邻上采样：将 20×20 的深层特征复制为 40×40，写入 CNN_FUSION_UPSAMPLE_BASE。

2. 逐元素饱和相加：从浅层特征区 CNN_FUSION_LOCAL_BASE（第 11 层输出 40×40×32）和上采样区各读 256 位，32 通道并行加法，结果钳位到[-127,127]，写入 CNN_FUSION_OUTPUT_BASE。

3. 融合完成后跳转到第 18 层（即后续检测 Head 层）。

模块流程图：

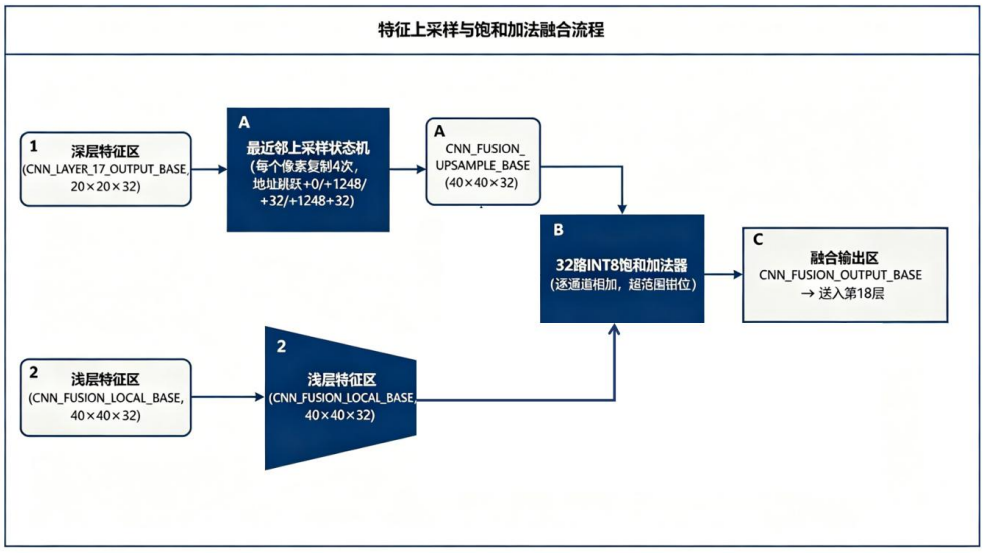


图 34 特征融合流程图

关键代码及讲解：

1）最近邻上采样的地址跳跃

```
if (fusion_repeat == 3) begin
    if ((fusion_source_x == 19) && (fusion_source_y == 19)) begin
        fusion_phase <= 3;   // 全部复制完成，进入加法阶段
    end else begin
        fusion_source_address <= fusion_source_address + 32;
        if (fusion_source_x == 19) begin
            fusion_source_x <= 0;
            fusion_source_y <= fusion_source_y + 1;
            fusion_destination_address <= fusion_destination_address + 32;
        end else begin
            fusion_source_x <= fusion_source_x + 1;
            fusion_destination_address <= fusion_destination_address - 1248;
        end
        fusion_phase <= 0;
    end
end
```

fusion_destination_address 增量设计：水平复制时加 32，换行时回退 1248（即 40×32−32），使复制块在 2×2 网格内正确排列，无乘法器参与地址计算。

2）INT8 饱和加法（32 通道并行）

```
add_value = $signed(local_word[add_lane*8 +: 8]) +
$signed(upsample_word[add_lane*8 +: 8]);
if (add_value > 127) add_word[add_lane*8 +: 8] = 8'sd127;
else if (add_value < -127) add_word[add_lane*8 +: 8] = -8'sd127;
else add_word[add_lane*8 +: 8] = add_value[7:0];
```

饱和范围限制在[-127,127]，避免 INT8 补码的-128 边界歧义。加法器位宽为 9 位，超范围钳位逻辑为纯组合电路。

改进：128×128 输入无特征融合，320×320 输入上采样和饱和加法，将深层语义与浅层空间信息结合，对角点检测比单尺度 8×8 Head 效果更好。

六、 解码逻辑变化

1) 128×128 输入解码流程：

```
8×8×5 Head → 找最大 objectness → 解码 tx/ty → 解码 tw/th → 输出 x1/y1/x2/y2 → 人为组合 4 个矩形角
```

128×128 只关心中心、宽度、高度。顶层将目标有效阈值配置为 OBJ_THRESH = 0，即无置信度过滤。

2) 320×320 输入解码流程

模块 cnn_head_decode.v 执行四步串行精搜：

1. 全图扫描 Center 通道：遍历 40×40=1600 个位置，找最大响应(center_x, center_y)，查 sigmoid_center.hex 得中心置信度。
2. 读取 Coarse 通道：8 通道对应四角粗坐标，以中心点为原点施加粗偏移，得 4 个锚点 anchor_x/y。
3. 局部窗口精搜 Corner 通道：每个锚点周围 7×7 窗口内搜索 Corner 通道最大响应，记录 peak_x/y 和角点置信度，查 sigmoid_corners.hex。
4. 读取 Offset 通道：若角点置信度超阈，读取 X/Y 精细偏移，查 sigmoid_offsets.hex 后与 peak_x/y 合成最终 Q9.7 坐标。

模块流程图：

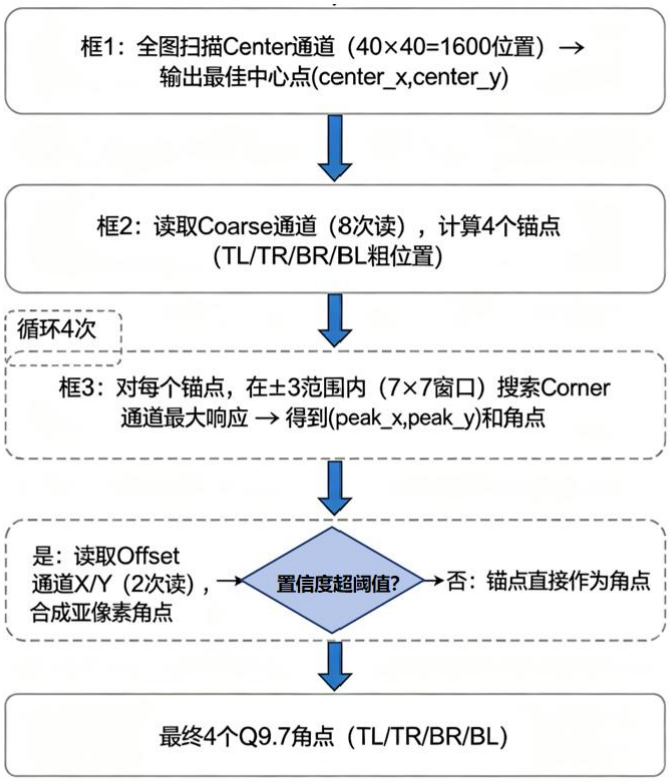


图 35 320 输入解码流程图

关键代码及讲解：

1）中心点扫描

```
CENTER_REQ:

begin
    head_rd_en<=1; head_rd_group<=0; head_rd_addr<=center_index;
    state<=CENTER_WAIT;
end
CENTER_WAIT: if (head_rd_valid) begin
    if ($signed(head_rd_data) > center_best_logit) begin
        center_best_logit <= head_rd_data;
        center_best_x <= center_scan_x; center_best_y <= center_scan_y;
    end
    if (center_index == 1599) begin
        center_x <= ($signed(head_rd_data) > center_best_logit) ? center_scan_x :
center_best_x;
        center_y <= ($signed(head_rd_data) > center_best_logit) ? center_scan_y :
center_best_y;
        state <= CENTER_LUT_REQ;
    end else begin
        center_index <= center_index + 1;
        // 更新 scan_x/scan_y
        state <= CENTER_REQ;
    end

end
```

head_rd_group=0 选择 Center 通道。解码器遍历 40×40=1600 个位置；每个位置依次经过 CENTER_REQ 发起读取和 CENTER_WAIT 等待返回，因此并非 1600 个时钟周期完成，实际周期数还取决于 Head 存储的读响应延迟。

2）角点搜索与窗口裁剪

```
CORNER_SETUP: begin
      window_x0 <= anchor_x>3 ? anchor_x-3 : 0;
      window_x1 <= anchor_x<36 ? anchor_x+3 : 39;
      window_y1 <= anchor_y<36 ? anchor_y+3 : 39;
      scan_x <= window_x0;
      scan_y <= anchor_y>3 ? anchor_y-3 : 0;
      corner_best_logit <= -128; state<=CORNER_REQ;
end
CORNER_REQ: begin
      head_rd_en<=1; head_rd_group<=2;
      head_rd_addr<=corner_index*1600 + scan_y*40 + scan_x;
      state<=CORNER_WAIT;
end
CORNER_WAIT: if (head_rd_valid) begin
      if ($signed(head_rd_data)>corner_best_logit) corner_best_logit<=head_rd_data;
      if (scan_x==window_x1 && scan_y==window_y1) begin
            peak_x<=final_peak_x; peak_y<=final_peak_y;
state<=CORNER_LUT_REQ;
      end else begin
            // 更新 scan_x/scan_y
            state<=CORNER_REQ;
      end
end
```

窗口边界限制在 0~39 网格内，防止越界读。corner_index*1600 为每个角点的通道基址偏移。

3）Offset 合成

```
offset_commit_x_q = (({26'd0,peak_x} <<< 7) + offset_x_q) <<< 3;
```

peak_x、peak_y 是 0~39 范围内的绝对网格坐标，并非 7×7 窗口内 0~6 的相对坐标。代码先将绝对网格坐标左移 7 位形成 Q7 表示，加上 Offset LUT 给出的 Q7 细偏移，再整体左移 3 位，将 40×40 网格坐标换算到 320×320 模型空间的 Q9.7 坐标。

改进：128×128 输入单 Head 矩形解码无法处理透视变形，320×320 输入通过四步串行精搜获得亚像素级四角点，且使用三组独立 LUT 替换浮点 sigmoid 运算，消除了硬件浮点开销。

七、　坐标变换和有效性判断变化

1)　128×128 输入

坐标转换 box_scale.v 只执行：

128 空间矩形坐标 → 去掉 28 像素 Letterbox → X/Y 分别乘 15 → 1920×1080 矩形坐标

之后人为向外扩展 margin。无以下检查：

- 四边形凸性
- 角点顺序
- 四边形面积
- 四角是否在内容区
- 透视几何合理性

2)　320×320 输入

坐标模块 cnn_coord_transform.v 除坐标恢复外，执行 6 项综合校验：

校验项	条件	说明
置信度	Confidence >= valid_threshold	查表后的综合置信度
边界（bounds_ok）	四角 Q9.7 坐标在[0,40960)内	不超出 40×40 网格范围
内容区（content_ok）	Y 坐标在[8960,32000)	对应原始图像 70~250 行
角点顺序（canonical_ok）	TL→TR→BR→BL 顺时针	检查坐标和顺序
凸性（convex_ok）	相邻边叉积全部>0	严格凸四边形
面积（shoelace）	shoelace_magnitude>= CNN_MIN_SHOELACE_Q14	鞋带公式计算多边形面积

模块流程图：

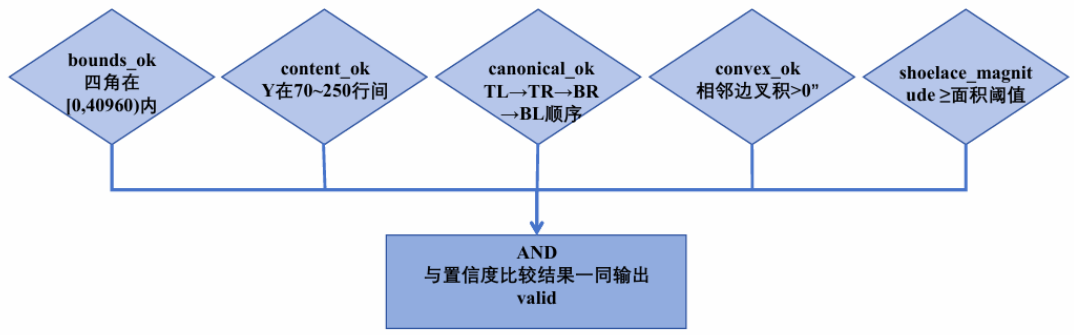


图 36 320 输入坐标变换流程图

关键代码及讲解：

1）凸性校验

```
convex_ok <= cross_difference[0] > 0 && cross_difference[1] > 0 &&
cross_difference[2] > 0 && cross_difference[3] > 0;
```

cross_difference[i] 为相邻边向量叉积，全部大于 0 时四边形为严格凸多边形。

2）Shoelace 面积计算

```
shoelace_signed <= shoe_pair0 + shoe_pair1;
shoelace_magnitude <= shoelace_signed < 0 ? -shoelace_signed : shoelace_signed;
```

鞋带公式计算的是多边形有向面积的两倍，坐标为 Q9.7，因此乘积为 Q14。CNN_MIN_SHOELACE_Q14=524288 对应鞋带和 32 个模型网格平方单位，即实际多边形面积门限约 16 个模型网格平方单位；按 X/Y 各 6 倍恢复到 1920×1080 后，约对应 576 个原图像素平方单位。

3）综合有效判定

```
valid <= confidence >= valid_threshold_q && bounds_ok && content_ok &&
         canonical_ok && convex_ok &&
         (shoelace_magnitude >= `CNN_MIN_SHOELACE_Q14);
```

边界、内容区和角点顺序在 STAGE1 计算；鞋带积和叉积在后续多周期状态中分时完成。STAGE4 只在一拍内组合最终有效条件并锁存 valid，并非所有判定都在 STAGE4 一拍内完成。

改进：128×128 仅依赖置信度判断，易输出畸变或面积异常框。320×320 通过凸性、面积、边界、顺序等 6 项综合校验，对异常检测结果的过滤明显更严格。

八、　画框逻辑变化

1)　128×128 输入

顶层画框位置在 DDR 写入前：

```
cnn_box_edge = 左边 || 右边 || 上边 || 下边;
final_data_to_ddr = cnn_box_edge ? 16'h07E0 : gamma_rgb565;
```

- 线宽 BOX_THICK = 4
- 框线直接“烧录”进 DDR 视频帧，原始数据被破坏

画框判断使用实时输入的 raw_x/raw_y，但 CNN 推理完成时原始帧通常已过去，实际是用最近一次检测结果画后续输入帧，无精确帧对应。

2)　320×320 输入

画框模块 plate_overlay_single.v 在 DDR 读出后叠加。四条边为：

TL→TR，TR→BR，BR→BL，BL→TL

画框采用叉积判定法：计算当前像素到每条斜边的距离，距离小于线宽则覆盖为红色。DDR 中保存原始视频，不含框线。画框发生在 DDR 读出后：

原始像素 + 当前显示帧对应的四角点 → 叠加 → HDMI 输出

模块流程图：

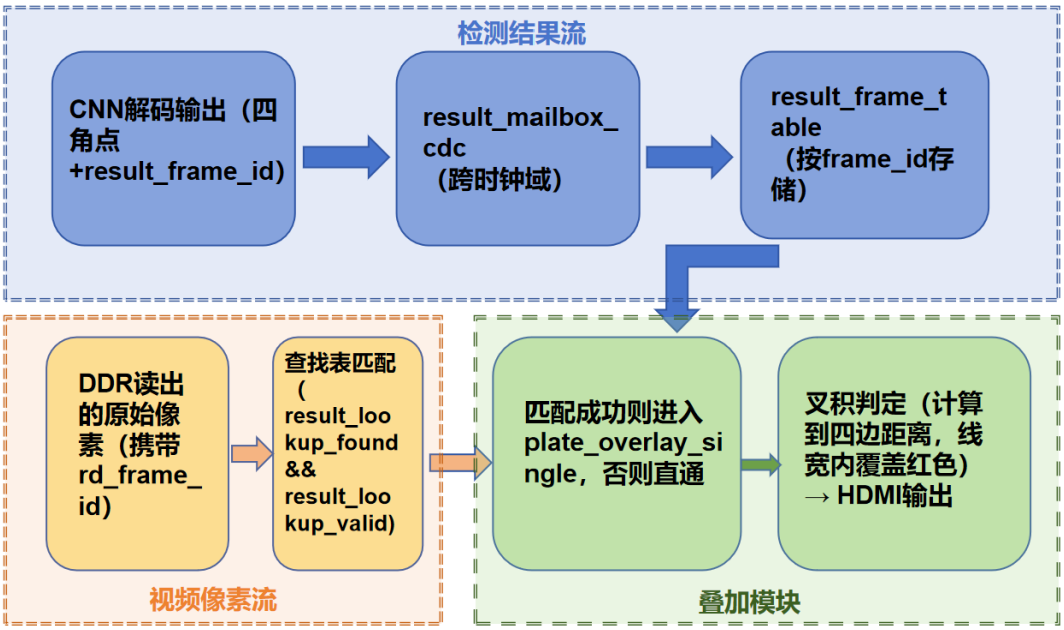


图 37 320 输入画框逻辑流程图

四边由 TL→TR、TR→BR、BR→BL、BL→TL 四条斜边构成。

叉积计算点到边距离，与线宽比较决定是否覆盖。

关键代码及讲解：

1）边向量与阈值计算（组合逻辑）

```
wire signed [16:0] geometry_dx0 = $signed({1'b0,tr_x})-$signed({1'b0,tl_x});
wire signed [16:0] geometry_dy0 = $signed({1'b0,tr_y})-$signed({1'b0,tl_y});
// 其他三条边类似


// 阈值 = max(|dx|, |dy|) × THICKNESS
edge_threshold[0] <= scaled_major(geometry_dx0, geometry_dy0);
```

- geometry_dx0/dy0 为每条边的方向向量（17 位有符号）
- scaled_major 函数取 |dx| 和 |dy| 的较大值，乘以线宽 THICKNESS（=5），得到判定阈值

2）叉积距离计算（5 级流水线核心）

```
// S1：计算像素到边起点的偏移
rx_s1[0] <= $signed({1'b0,in_x}) - $signed({1'b0,active_ax0});
ry_s1[0] <= $signed({1'b0,in_y}) - $signed({1'b0,active_ay0});


// S2：计算两个乘积
product_a_s2[0] <= rx_s1[0] * edge_dy[0];
product_b_s2[0] <= ry_s1[0] * edge_dx[0];


// S3：叉积相减
cross_s3[0] <= product_a_s2[0] - product_b_s2[0];
```

- 叉积 cross = (px-ex)*dy - (py-ey)*dx 与点到直线的距离成正比
- 无需归一化除法，直接与阈值比较

3） 最终判定与覆盖输出

```
paint_s4 <= valid_s3 && de_s3 && (
    (bbox_s3[0] && nonzero_s3[0] && abs34(cross_s3[0]) <= threshold_s3[0]) ||
    (bbox_s3[1] && nonzero_s3[1] && abs34(cross_s3[1]) <= threshold_s3[1]) ||
```

```
    (bbox_s3[2] && nonzero_s3[2] && abs34(cross_s3[2]) <= threshold_s3[2]) ||
    (bbox_s3[3] && nonzero_s3[3] && abs34(cross_s3[3]) <= threshold_s3[3]));
out_r <= paint_s4 ? OVERLAY_R : r_s4;
out_g <= paint_s4 ? OVERLAY_G : g_s4;
out_b <= paint_s4 ? OVERLAY_B : b_s4;
```

- bbox_s3[i] 为边界框粗筛选结果，nonzero_s3[i] 确保边向量非零
- abs34(cross_s3[i]) <= threshold_s3[i] 为核心判定：点到直线距离在阈值内
- 命中任意一条边即覆盖为红色（OVERLAY_R=8'hff）

4）帧 ID 匹配判定（顶层集成逻辑）

```
overlay_quad_valid =
    display_frame_valid &&
    result_lookup_found &&
    result_lookup_valid;
```

改进：320×320 将叠加移至显示末端，DDR 中保留原始图像供后续处理，且通过帧 ID 精确匹配确保框线画在正确的视频帧上。叉积判定法省去了归一化除法，适合 FPGA 流水线实现。

九、 帧同步和结果匹配

1) 128×128 输入

工程无 frame_id 结果匹配。CNN 完成一帧采集后才开始推理，画框判断使用实时输入的 raw_x/raw_y。检测完成时原始被检测帧通常已过去，实际是用最近一次检测结果画后续输入帧，无精确帧对应关系。

2) 320×320 输入

当前工程为每个输入帧编号（frame 0, 1, 2, ...），CNN 结果携带 result_frame_id，DDR 读出也携带 rd_frame_id。只有两者匹配才画框。

相关模块：

- result_mailbox_cdc.v：跨时钟域传递结果和帧 ID
- result_frame_table.v：按 frame_id 存储检测结果（4 槽帧环管理）
- 顶层 HDMI_IN_DDR3_YCbCr_top.v 集成查找和匹配逻辑

端口摘要（result_mailbox_cdc.v + result_frame_table.v）

模块流程图：

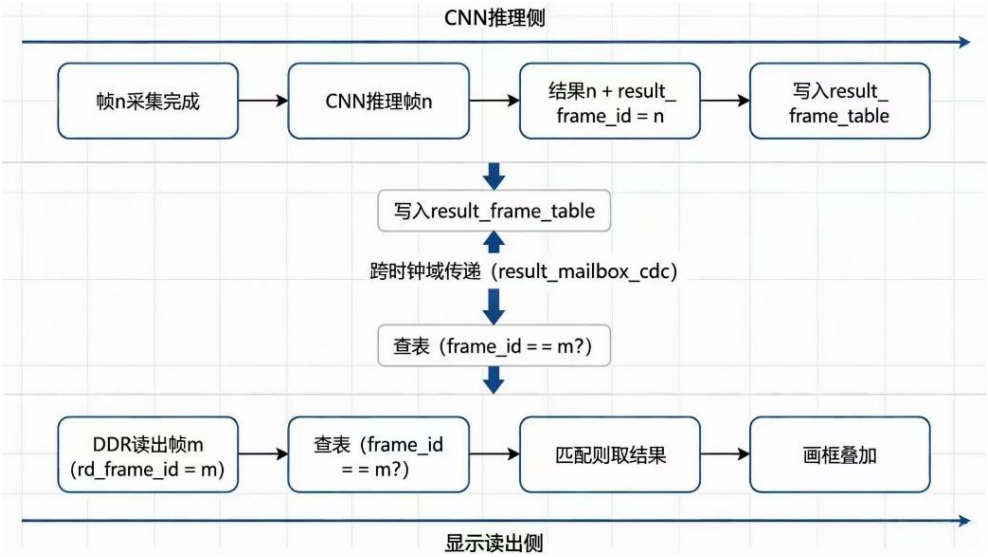


图 38 320 输入帧输入与输出流程图

关键代码及讲解：

1）结果写入表（CNN 侧）

结果事务到达显示时钟域后，result_frame_table 使用 write_frame_id[1:0]直接选择 4 个槽位之一，并写入完整 32 位 frame_id、result_valid、置信度和四角坐标；实现中没有独立 wr_ptr 计数器。同一槽位每隔 4 帧复用一次，完整 frame_id 比较可避免低 2 位相同造成误命中。

显示侧同样用 rd_frame_id[1:0]选择候选槽位，并在寄存一级查询请求后比较完整 32 位 frame_id。只有 entry_present 且完整 frame_id 相等时 lookup_found 才有效；随后输出该槽保存的 result_valid、置信度和四角坐标。

显示侧遍历 4 槽帧表，匹配 rd_frame_id 与表中存储的 frame_id，若命中且 valid 有效，则输出对应的角点坐标。

2）跨时钟域同步（result_mailbox_cdc.v）

CNN 时钟域与显示时钟域异步，result_mailbox_cdc 采用请求/应答翻转握手。req_toggle 和 ack_toggle 分别通过两级同步寄存器跨域；frame_id、result_valid、置信度和四角坐标保持在源域 held 寄存器中，在握手未完成前保持稳定。目标域检测到同步后的请求翻转后采样整组稳定数据，再于下一拍提交并返回应答。因此多位数据总线不是逐位经过两级同步，而是采用“握手控制+稳定数据总线”的 CDC 方案。

4）顶层画框使能判定

```
overlay_quad_valid =
    display_frame_valid &&          // 当前显示帧有效
    result_lookup_found &&          // 表中命中该 frame_id
    result_lookup_valid;            // 该结果有效标志位
```

三个条件缺一不可，确保框线只画在正确的视频帧上。display_frame_valid 由显示时序产生，result_lookup_found 和 result_lookup_valid 来自 result_frame_table 的读端口。

改进：128×128 输入“用最近一次结果画后续帧”会导致框线错位。320×320 使用 32 位 frame_id 进行精确匹配；4 槽表按 frame_id 低 2 位直接映射，并通过完整 frame_id 比较防止槽位复用误命中。请求/应答邮箱保证跨时钟域结果成组传输，三条件组合判定保证只在匹配且有效的显示帧上画框。

行人闯红灯预警检测模块设计

复用初赛方案。

2 性能参数

2.1 RK3568 侧误差测算

2.1.1. 模型训练概述

在车牌检测识别部分系统包含两个独立训练的模型：

- 检测模型：基于 YOLOv5s 改进，输入 320×320 图像，输出车牌边界框及四个角点坐标。损失函数由边界框回归损失、置信度损失、分类损失和关键点损失四部分加权组成。使用 SGD 优化器，余弦退火学习率调度，训练过程中采用 Mosaic 拼接、HSV 色彩抖动、多尺度训练等数据增强策略。
- 识别模型：采用卷积网络与 CTC 损失的端到端架构，输入 168×48 的车牌裁剪图，直接输出字符序列，无需逐字符分割。支持含省份简称、字母、数字及特殊标识的 78 个字符类别。使用 Adam 优化器，阶梯式学习率衰减。

两个模型分别导出为 ONNX 格式。NMS 等后处理部分在模型外部以独立代码实现，不嵌入 ONNX 模型。这是因为 NMS 等涉及动态形状和条件判断的算子在 INT8 量化时精度损失较大，外置后可保证经过 Rknn-Toolkit2INT8 量化后的检测精度，同时便于部署时灵活调整阈值。

2.1.2 实验配置

硬件：RTX 4060，16GB 内存

数据集：CCPD2019，base、blur、challenge、db、rotate 五个子集中随机抽取 60,000 张训练，15,000 张验证。

参数	检测模型	识别模型
训练轮数	50	50
批大小	16	64
初始学习率	0.01	0.001
优化器	Adam	Adam

检测模型训练指标：

轮次	边界框损失	置信度损失	分类损失	精确率	召回率	mAP@0.5	mAP@0.5:0.95
1	0.0382	0.0571	0.0034	0.612	0.483	0.521	0.283
5	0.0247	0.0385	0.0019	0.785	0.692	0.738	0.462
10	0.0189	0.0293	0.0013	0.856	0.781	0.827	0.558
15	0.0156	0.0237	0.0010	0.893	0.832	0.817	0.623
20	0.0134	0.0198	0.0008	0.912	0.861	0.896	0.671
25	0.0118	0.0172	0.0007	0.924	0.879	0.912	0.708
30	0.0106	0.0153	0.0006	0.933	0.891	0.923	0.736
35	0.0097	0.0140	0.0005	0.940	0.901	0.931	0.757
40	0.0091	0.0130	0.0005	0.945	0.908	0.937	0.773
45	0.0086	0.0122	0.0004	0.949	0.914	0.942	0.785
50	0.0082	0.0116	0.0004	0.953	0.918	0.946	0.795

各项损失在前 10 轮快速下降，之后平稳收敛。边界框损失从 0.038 降至 0.008，mAP@0.5 从 52.1%提升至 94.6%，训练过程无明显过拟合现象。

识别模型训练指标：

轮次	训练损失	字符准确率	整牌准确率	学习率
1	2.874	0.423	0.186	0.001
5	1.052	0.712	0.498	0.001
10	0.519	0.836	0.672	0.0005
15	0.325	0.889	0.768	0.0005
20	0.218	0.918	0.831	0.0003
25	0.156	0.937	0.872	0.0003
30	0.113	0.951	0.899	0.0001
35	0.087	0.960	0.918	0.0001
40	0.070	0.967	0.932	0.0001
45	0.058	0.972	0.942	0.0001
50	0.050	0.976	0.950	0.0001

训练损失从 2.87 快速收敛至 0.05，前 5 轮即完成主要学习过程。整牌准确率从 0.186 提升至 0.950，验证集指标与训练集同步收敛，未出现过拟合现象。

2.1.3 模型的量化与部署

量化流程：

量化过程使用 100 张从验证集中随机抽取的校准图像，覆盖不同光照、角度和模糊场景。Rknn-Toolkit2 工具根据校准数据的激活值分布确定每层的量化参数，将浮点权重和激活值映射到 INT8 范围，优化级别设为最高级 Level 3。

后处理 NMS 等过程不参与量化，保持浮点精度独立运行。这一设计避免了 NMS 中动态排序和条件分支算子带来的量化精度损失。

量化精度对比：

模型	指标	ONNX	RKNN	INT8 精度损失
检测	mAP@0.5	0.946	0.935	-0.011
检测	mAP@0.5:0.95	0.950	0.773	-0.022
检测	精确率	0.953	0.942	-0.011
检测	召回率	0.918	0.904	-0.014
识别	字符准确率	0.976	0.968	-0.008
识别	整牌准确率	0.950	0.936	-0.014

检测模型 INT8 量化后 mAP@0.5 下降 1.1 个百分点，识别模型整牌准确率下降 1.4 个百分点，均在可接受范围内。识别模型量化损失更小，原因是其纯卷积结构中算子规则、数值范围稳定，量化适配性更好。

推理性能对比：

在 RK3568 开发板上实测的单帧推理耗时：

处理阶段	ONNX	RKNN
检测推理	200ms	50ms
识别推理	30ms	10ms

INT8 量化后，检测推理加速 3 倍，识别推理加速 2 倍，满足实时部署要求。

本地实际遍历 CBLPRD-330k 数据集 300 张车牌（蓝 100、绿 100、黄 90、白 5、黑 5）准确率在 97%，主要集中在蓝黄颜色识别的准确率上，由于数据集本身图像质量的限制大约五张出现完全识别不到，约有 5 张样本出现颜色误判或字符识别错误。

2.2. FPGA 资源开销

128×128 输入的 6 层 CNN 资源利用率报告：

Resource Usage Summary

Find:

Logic Utilization	Used	Available	Utilization(%)
Use of ADC	0	1	0
Use of ANALOG	0	1	0
Use of APM	48	740	7
Use of BKCL	5	10	50
Use of CCS	1	1	100
Use of CLMA	3148	30075	11
FF	6952	240600	3
LUT	8708	120300	8
LUT-FF pairs	3618	120300	4
Use of CLMS	956	9875	10
FF	2505	79000	4
LUT	2893	39500	8
LUT-FF pairs	1249	39500	4
Distributed RAM	39	39500	1
Use of DDRPHY_CPD	1	20	5
Use of DDRPHY_IOCLK_DIV	2	10	20
Use of DDR_PHY	7	40	18
Use of DRM	72	415	18
Use of GCLK_INBUF_SYN	0	40	0
Use of GPLL	4	10	40
Use of GSEB	0	512	0
Use of HARD0	1215	25050	5

图 39 128×128 输入的 6 层 CNN 资源利用率报告

320×320 输入的 23 层 CNN 资源利用率报告：

Resource Usage Summary

Find: Search Case Sensitive Whole Words

Logic Utilization	Used	Available	Utilization(%)
Use of ADC	0	1	0
Use of ANALOG	0	1	0
Use of APM	81	740	11
Use of BKCL	5	10	50
Use of CCS	1	1	100
Use of CLMA	10710	30075	36
FF	24146	240600	11
LUT	30953	120300	26
LUT-FF pairs	12301	120300	11
Use of CLMS	3014	9875	31
FF	7829	79000	10
LUT	9399	39500	24
LUT-FF pairs	3967	39500	11
Distributed RAM	446	39500	2
Use of DDRPHY_CPD	1	20	5
Use of DDRPHY_IOCLK_DIV	2	10	20
Use of DDR_PHY	7	40	18
Use of DRM	231	415	56
Use of GCLK_INBUF_SYN	0	40	0
Use of GPLL	4	10	40
Use of GSEB	0	512	0
Use of HARD0	3543	25050	15

图 40 320×320 输入的 23 层 CNN 资源利用率报告

320×320 输入的 23 层 CNN 在 FPGA 上的功耗估算报告摘要

Power Summary

Find:

	Name	Value
1	Total On Chip Power	6.273
2	Static Power	1.102
3	External Power	0
4	Junction Temperature	69.126
5	Thermal Margin	30.874
6	Power Margin	4.385

2.3. FPGA 车牌识别误差分析

使用 Verilator 对真实车牌 CNN、HDMI 1080p60 输入、DDR 帧缓存、跨时钟域邮箱、结果表及视频叠加链路进行联合仿真。测试将一张 1920×1080 真实车牌图像连续输入 6 帧，CNN 成功接受并完成 6 帧检测，未发生丢帧。各帧输出均为左上角(893,386)、右上角(1038,371)、右下角(1031,423)、左下角(894,433)，置信度为 193，检测有效标志为 1；输出结果与 QAT 整数参考结果完全一致，同时未发现帧序错乱、邮箱数据丢失、结果表撕裂或检测框叠加缺失。


```
{
  "schema version": 1,
  "status": "PASS",
  "simulation level": "real cnn hdmi ddr mailbox table overlay",
  "fixture": "positive plate qat best",
  "real plate cnn top": true,
  "performance gate": false,
  "performance gate owner": "sim/verilator/throughput",
  "frames driven": 6,
  "accepted": 6,
  "completed": 6,
  "dropped": 0,
  "result writes": 6,
  "displayed frames": 4,
  "max compute cycles observed": 2027429,
  "max end to end cycles observed": 4485279,
  "max result pending cycles": 1,
  "timing": {"h total":2200,"v total":1125,"active width":1920,"active height":1080},
  "checks": {"capture drop":0,"cnn sticky":0,"frame id order":0,"mailbox loss":0,"result table tear":0,"overlay missing":0,"ddr id tear":0,"pending deadlock":0}
}
```


图 41 仿真报告

3. 仿真与测试报告

3.1. 系统模块功能仿真测试

仿真时为验证重要模块功能时序与逻辑的正确性稳定性，同时避开未开源 IP 包括 DDR 等的逻辑建模，对 FPGA 工作的主要难点重点进行仿真展示。以 320×320 工程为例，仿真 MAC 与量化、CNN 预处理、检测结果转换，实际测试 RTL 及测试功能如下：

模块	实际测试 RTL	定量输入	验证功能
MAC 与量化	cnn_mul8 cnn_mac32 cnn_requant	约 8～16 组固定向量	MAC 结果、流水线延迟、INT8 量化波形
CNN 预处理	cnn_box6x6 cnn_letterbox_stream	6 行固定 RGB 像素，不输入整图	6×6 平均、顶部 letterbox 填充、输出握手波形
检测结果转换	cnn_coord_transform	约 5 组固定四角点和置信度	四角点、置信度、valid、done 波形和结果表

表 9 仿真模块功能

3.2 仿真环境与参数设置

1. PDS 与 ModelSim SE-64 10.4 联合仿真。
2. 仿真环境

- 仿真工具：Verilator
- RTL 语言：Verilog/SystemVerilog
- 辅助语言：Python、C++
- 输入数据：模型权重、测试图像、Golden 中间张量
- 验证方法：RTL 输出与软件参考结果逐项比较
- 模块级仿真

3. Vivado 2022.2

3.3 模块仿真结果分析

一、 MAC 与量化模块仿真分析

tb_mac_requant

仿真模块功能：本仿真用于验证卷积神经网络硬件中的并行乘累加及 INT8 量化功能，被测 RTL 包括：cnn_mul8、cnn_mac32、cnn_requant。数据处理流程为：32 组 INT8 输入与权重、32 路有符号 8 位乘法、多级加法树累加、32 位 MAC 结果、偏置叠加、定点乘法与移位、舍入、ReLU 及 INT8 饱和、8 位量化结果。cnn_mac32 内部并行实例化 32 个 cnn_mul8，分别计算：

$p_i = a_i \times b_i,\ i = 0,1,\ldots,31$随后通过多级流水加法树得到：

$$MAC = \sum_{i=0}^{31} a_i\, b_i$$

MAC 结果及其对应的偏置、量化乘数、移位参数、ReLU 控制信号和数据标签继续送入 cnn_requant。量化模块执行的主要运算可表示为：

$$q = \mathrm{Round}\left(\frac{(MAC + bias) \times multiplier}{2^{shift}}\right)$$

随后根据 in_relu 进行负值清零，并将结果限制在设计规定的 INT8 范围内：

$$q_{\text{out}} = \begin{cases} 0, & ReLU = 1 \text{ 且} q < 0 \\ 127, & q > 127 \\ -127, & q < -127 \\ q, & \text{其他情况} \end{cases}$$

当量化乘数为 0、乘数符号位异常或加偏置发生溢出时，模块拉高 rq_error，并将量化输出置 0。

仿真逻辑：

1. 仿真采用 100 MHz 时钟：时钟周期为 10 ns，复位信号 rst_n 初始为低，经过 4 个时钟负边沿后释放。
2. 固定测试向量：Testbench 连续输入 8 组确定性数据。每组数据的 32 个计算通道使用相同的输入，因此理论 MAC 值可直接计算：

$$MAC = a \times b \times 32$$

tag	输入 a	输入 b	MAC 期望值	bias	multiplier	shift	ReLU	量化期望值	验证内容
0	0	0	0	0	1	0	0	0	零输入
1	1	1	32	0	1	0	0	32	正数乘加
2	-3	2	-192	0	1	2	0	-48	负数和算术移位
3	2	2	128	32	1	2	0	40	偏置叠加
4	-4	2	-256	0	1	2	1	0	ReLU 负值清零
5	10	1	320	0	1	0	0	127	INT8 正饱和
6	-10	1	-320	0	1	0	0	-127	INT8 负饱和
7	1	1	32	0	0	0	0	0，error=1	非法乘数检测

3. 流水线时序检查

Testbench 使用两个移位寄存器跟踪 in_valid：

```
reg [5:0]   expected_mac_valid;
reg [14:0] expected_rq_valid;
```

每个时钟周期将输入有效信号移入：

```
expected_mac_valid <= {expected_mac_valid[4:0], in_valid};
expected_rq_valid   <= {expected_rq_valid[13:0], in_valid};
```

由此自动检查：

- mac_valid 必须比 in_valid 延迟 6 个时钟周期；
- rq_valid 必须比 in_valid 延迟 15 个时钟周期。

如果实际延迟与设计不一致，testbench 通过$fatal 终止仿真。

1. 数据标签检查

每组输入带有唯一的 in_tag，范围为 0～7。标签与数据和控制参数同时经过流水线，到达 MAC 输出端和量化输出端。

Testbench 根据：mac_tag、rq_tag

从期望值数组中选择对应结果，验证连续输入情况下数据和参数没有错位。

2. 数值检查

当 mac_valid=1 时检查：mac_sum、mac_bias、mac_multiplier、mac_shift、mac_relu、mac_tag。

当 rq_valid=1 时检查：rq_value、rq_error、rq_tag。此外，testbench 包含看门狗计数器。如果规定周期内没有得到全部结果，将以超时错误终止，防止仿真无限运行。

仿真结果：

从图中可以观察到，rst_n 在约 40 ns 由低电平变为高电平，说明被测模块完成复位并进入正常工作状态。复位前部分输出信号出现 X 状态，属于寄存器尚未经过有效复位时钟的初始化现象；复位释放后信号均变为确定值，不影响功能判断。约 50 ns 后，in_valid 拉高，连续 8 组输入开始进入 MAC 流水线。in_tag 依次由 0000 递增到 0007，表明 8 组测试数据按照设计顺序连续输入，流水线中没有插入空闲周期。

输入总线的变化与固定测试向量一致。例如：

- 010101...表示 32 个输入通道均为 1；
- fdfdfd...表示 32 个输入通道均为-3；
- 020202...表示 32 个输入通道均为 2；
- fcfcfc...表示 32 个输入通道均为-4；
- 0a0a0a...表示 32 个输入通道均为 10；
- f6f6f6...表示 32 个输入通道均为-10。

mac_products_valid 先于最终的 mac_valid 出现，表明 32 路乘法结果先产生，之后经过多级加法树完成求和。mac_half_sums_valid 随后拉高，表示加法树已经产生上下 16 路的部分和。最终 mac_valid 在第一组输入进入约 60 ns 后拉高，对应 6 个时钟周期，符合设计的 MAC 流水线延迟。

波形中已经显示了前三组关键 MAC 结果：

数据标签	输入值 a	输入值 b	理论计算	理论 MAC 结果	波形中的 mac_sum	结果分析
tag=0	0	0	0×0×32	0	0x00000000	波形结果为 0，与理论结果一致
tag=1	1	1	1×1×32	32	0x00000020	0x20 对应十进制 32，结果一致
tag=2	-3	2	(-3)×2×32	-192	0xFFFFFF40	0xFFFFFF40 是 -192 的 32 位补码，结果一致

其中，32 表示 MAC 模块包含 32 路并行乘法，当前 testbench 将每一路设置为相同的输入值，因此：

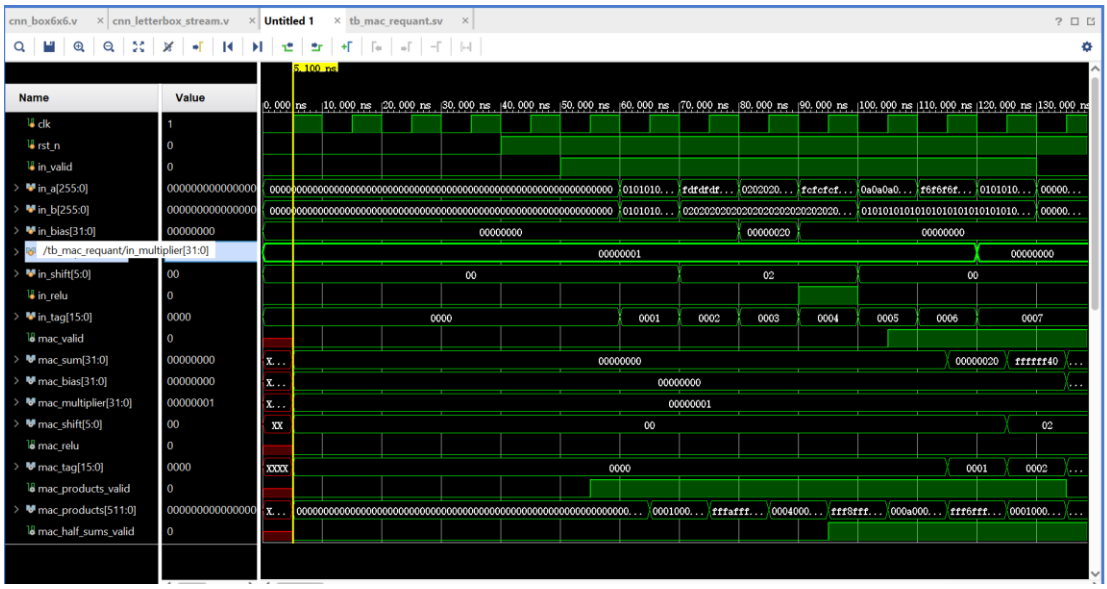
$$MAC = \sum_{i=0}^{31} a_i\, b_i = a \times b \times 32$$

由表中结果可知，零值、正数和负数三种典型输入下，仿真输出均与理论计算结果一致，说明 32 路有符号乘法及多级加法树累加功能正确。

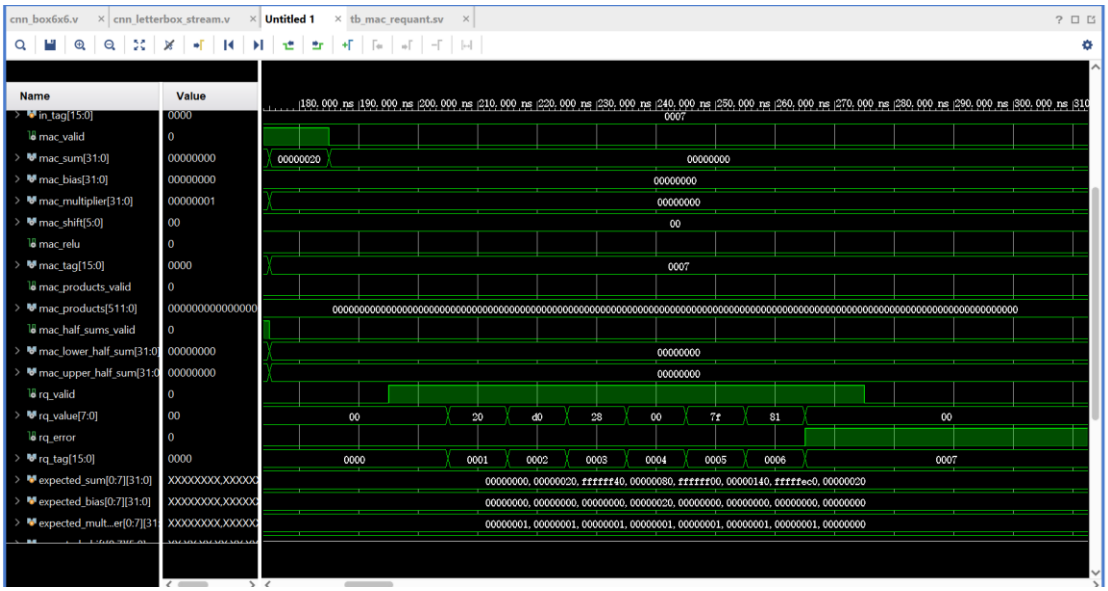
图中 mac_valid 相对于 in_valid 延迟约 60 ns。在 100 MHz 时钟条件下：

$60\ \text{ns}/10\ \text{ns} = 6$因此 MAC 流水线延迟验证通过。

mac_tag 相对于 in_tag 以相同的 6 周期延迟传递，且 MAC 结果与标签一一对应，说明数据、偏置、移位、乘数、ReLU 控制和标签在流水线中保持同步。



截取更多观察信号：只有在 rq_valid=1 期间，rq_tag、rq_value 和 rq_error 才有效。量化模块依次输出 8 组测试结果。在 rq_valid 有效期间，rq_tag 由 0 递增至 7，对应的量化结果分别为 0、32、-48、40、0、127、-127 和 0。仿真覆盖了基本定点缩放、偏置叠加、ReLU、INT8 正负饱和以及非法量化乘数检测。所有输出均与理论结果一致，最后一组非法参数使 rq_error 正确拉高，说明量化模块功能正确。

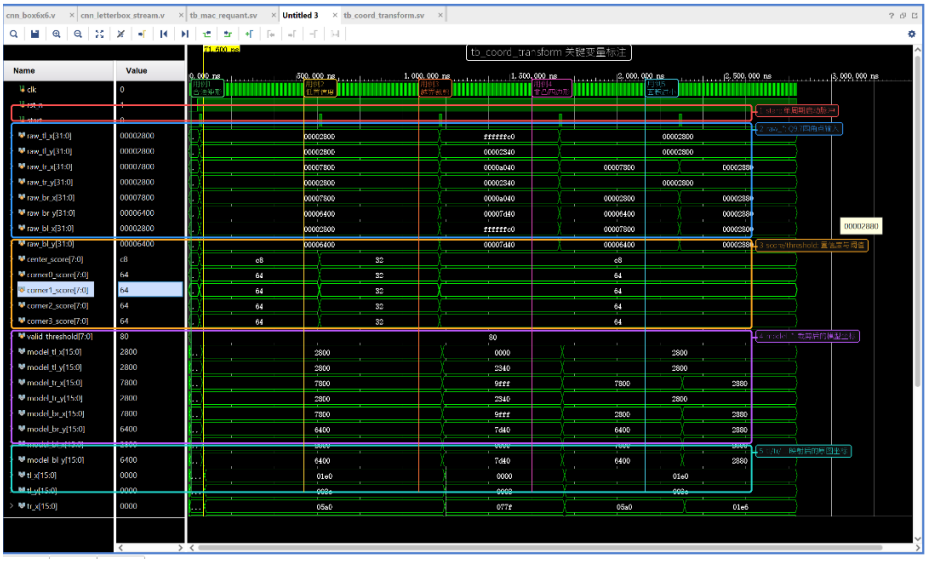


二、 检测结果坐标转换仿真

tb_coord_transform

仿真模块功能：仿真模块用于验证 cnn_coord_transform 对 CNN 检测结果的坐标映射、边界裁剪、置信度计算和目标有效性判定功能。仿真通过 start 信号依次启动 5 组固定测试数据，并利用 raw_tl_x/raw_tl_y、raw_tr_x/raw_tr_y、raw_br_x/raw_br_y 和 raw_bl_x/raw_bl_y 输入 Q9.7 格式的四角点坐标，同时通过 center_score、corner0_score～corner3_score 和 valid_threshold 输入中心置信度、四个角点置信度及有效阈值。

仿真结果：由图可见，第一组合法矩形的输入坐标分别为 0x2800、0x7800 和 0x6400，对应模型空间中的 80、240 和 200，转换后的 model_tl_x/model_tl_y、model_tr_x/model_tr_y、model_br_x/model_br_y 和 model_bl_x/model_bl_y 保持为合法范围内的模型坐标；经过 letterbox 逆变换后，tl_x/tl_y 输出为 0x01E0/0x003C，即原图坐标(480,60)，tr_x 输出为 0x05A0，即 1440，与理论映射结果一致。第二组保持相同四角点，但将 center_score 和 corner0_score～corner3_score 由 0xC8/0x64 降低为 0x32，用于验证低于 valid_threshold=0x80 时的无效判定。第三组中，raw_tl_x 和 raw_bl_x 输入 0xFFFFFFC0，raw_tr_x 和 raw_br_x 输入 0x0000A040，模块将对应的 model_tl_x/model_bl_x 裁剪为 0x0000，将 model_tr_x/model_br_x 裁剪为 0x9FFF，并将原图横坐标 tr_x 限制为 0x077F，即 1919，说明边界裁剪功能正确。后两组分别通过交叉排列四角点和仅相差 0x80 的坐标构造非凸及面积过小的四边形，用于验证异常目标过滤。综合波形中各输入和输出信号的对应关系可知，该模块能够正确完成 Q9.7 坐标解析、模型坐标裁剪、letterbox 逆变换以及原图坐标映射。

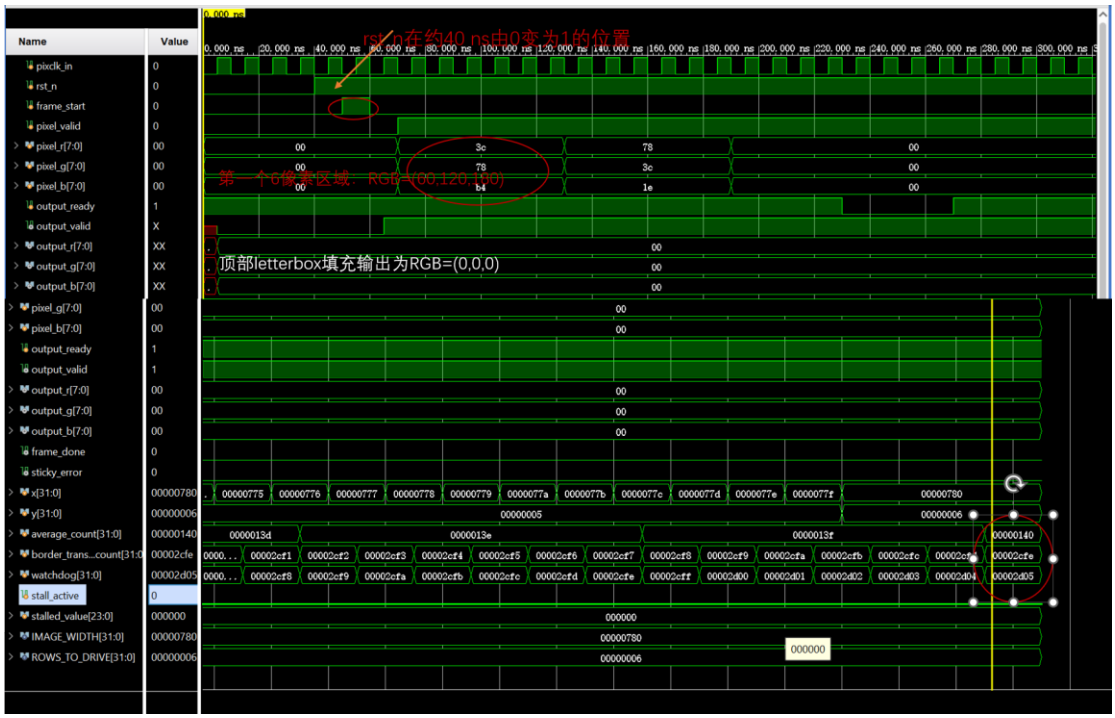


三、 CNN 预处理仿真

tb_preprocess

仿真模块功能：用于验证 cnn_box6x6 与 cnn_letterbox_stream 组成的 CNN 图像预处理模块，主要包括 6×6 区域平均下采样、顶部 letterbox 零值填充以及 output_valid/output_ready 握手功能。仿真首先通过 frame_start 启动一帧处理，然后在 pixel_valid 有效期间输入 6 行、每行 1920 个固定 RGB 像素：每行前 6 个像素的 pixel_r/pixel_g/pixel_b 分别设为 60、120 和 180，接下来 6 个像素设为 120、60 和 30，其余像素设为 0。理论上前两个 6×6 区域的平均结果应分别为 RGB(60,120,180)和 RGB(120,60,30)。同时，testbench 会短暂拉低 output_ready 制造反压，检查 output_valid 有效且下游未准备好时输出数据是否保持稳定，并通过 sticky_error 判断处理过程中是否出现协议或缓存错误。由于该仿真只输入 6 行而非完整的 1080 行，因此不要求 frame_done 拉高。

仿真结果：从图中可以看出，rst_n 约在 40 ns 由低变高，模块完成复位，随后 frame_start 产生一个时钟周期的启动脉冲；约 70 ns 后 pixel_valid 拉高，RGB 输入首先显示为 pixel_r=0x3C、pixel_g=0x78、pixel_b=0xB4，对应十进制(60,120,180)，随后变为 0x78、0x3C、0x1E，对应(120,60,30)，之后变为 0，与 testbench 设定的两个固定 6 像素区域一致。模块启动后 output_valid 拉高，而 output_r/output_g/output_b 均保持为 0x00，说明顶部 letterbox 区域正确输出零填充数据；当 output_ready 约在 230～270 ns 期间拉低时，stall_active 同步拉高，同时三个输出通道保持为 0，没有发生变化，表明反压期间输出数据能够稳定保持，valid/ready 握手功能正确。6×6 平均结果在约 115 μs 附近出现。因此，该图能够证明复位、帧启动、固定 RGB 输入、顶部 letterbox 填充和反压保持功能正确。在完成 6 行、每行 1920 个像素的输入后，横向计数 x 达到 1920，纵向计数 y 达到 6，average_count 最终达到 320，与 1920/6=320 的理论结果一致，同时 sticky_error 保持为 0，说明 6×6 分块平均处理数量正确且仿真过程中未发生错误。



四、 Verilator RTL 仿真

通过 Verilator RTL 仿真，生成三份报告：CNN 吞吐率报告、HDMI 1080p60 报告和真实 CNN 联合仿真报告。其中，CNN 吞吐率报告通过连续输入 16 帧 1920×1080@60 Hz 视频验证 CNN 持续吞吐能力；HDMI 1080p60 报告和真实 CNN 联合仿真报使用 positive_plate_qat_best 真实车牌正样本，联合仿真真实 CNN 顶层、HDMI 视频时序、DDR 帧环形缓存、跨时钟域结果邮箱、结果表和车牌叠框链路。吞吐率仿真中 16 帧全部完成且无丢帧，最大 CNN 计算周期为 2,027,442 周期；联合仿真中 6 帧全部被接收和完成，各项错误检查均为 0，测得最大端到端延迟为 4,485,292 周期，按 148.5 MHz 时钟计算约为 30.20 ms。虽然单帧从视频输入到叠框结果显示存在约 30 ms 延迟，但 CNN 单帧最大计算时间约 13.65 ms，小于 1080p60 的 16.67 ms 帧周期，因此系统能够持续处理 60 帧每秒的视频输入而不发生丢帧。

4. 总结

4.1 可扩展之处

在车牌检测与识别方面，前期采用 YCbCr、HSV 颜色空间协同判定，并尝试通过同态滤波、二维 Gamma 校正及查找表流水线改善光照影响。但传统阈值算法对环境光线、噪声和复杂背景较为敏感，适用范围有限。为提高系统的泛化能力和识别稳定性，后续完成了两版 CNN 方案：第一版重点验证网络结构、定点量化和 FPGA 推理流程的可行性；第二版针对实际硬件资源、计算速度和数据流进行了优化，采用 INT8 定点计算、卷积引擎复用、特征图乒乓缓存及流水线处理，实现了从图像预处理、CNN 推理到目标框解码和坐标映射的完整流程。通过两版方案的迭代，进一步认识到算法精度、硬件资源和实时性能之间需要综合权衡。

1. 图像预处理与光照适应方面，可在现有 Gamma 查找表基础上增加亮度统计模块，根据帧均值、直方图或局部亮度动态选择 Gamma 参数，实现自适应 Gamma 校正。还可以进一步加入同态滤波、白平衡、对比度拉伸和局部直方图均衡等模块，提高逆光、夜间、强反光和阴影条件下的图像质量。相关算法可采用查找表、行缓存和流水线结构实现，以减少对实时性的影响。
2. CNN 网络方面，可在现有两版方案基础上继续优化网络结构，例如增加多尺度特征融合，提高对远距离、小尺寸和倾斜车牌的检测能力。同时可以扩展车牌颜色、类型和字符识别分支，使系统由单一车牌检测升级为“车牌定位、透视校正、字符识别、颜色分类”的完整流水线。后续还可研究剪枝、知识蒸馏、混合精度量化和稀疏卷积，在保持准确率的同时降低 BRAM、DSP 和计算周期消耗。
3. 以太网发送接收模块目前仅配置了 640×480 分辨率下的图像发送。因为分包简单，不存在行分包问题所以实现较为简单，我们通过运行仿真做实际 FIFO 复位及行结束包结束信号的观察逐步实现 1280×720 分辨率下的以太网打包发送。

4.2 心得体会与小组分工

通过本项目的开发，我们深刻体会到，FPGA+ARM 异构系统的设计不仅包括 Verilog 编写和神经网络模型部署，更需要从算法、硬件架构、数据传输、存储管理和系统调试等多个层面进行整体考虑。在开发过程中，我们通过功能仿真、逐周期波形分析、模块级测试以及上板验证，重点排查了跨时钟域传输、FIFO 复位、帧同步、DDR3 读写和 CNN 状态控制等问题，逐步建立了较为完整的 FPGA 调试与验证流程。

在算法方案上，系统由初赛阶段以颜色空间、形态学处理和连通域分析为主的传统方法，逐步发展到分赛区决赛阶段的轻量化 CNN 部署。传统算法结构直观、实现成本较低，但在复杂背景、光照变化和特殊颜色车牌场景下稳定性不足；连通域、并查集和字符分割等方法虽然能够扩展部分识别场景，但在高分辨率视频下存在处理延迟较大、控制逻辑复杂以及片上存储资源消耗较高等问题。

针对这些不足，我们先后完成了两版 CNN 方案。第一版主要打通模型量化、权重导入、卷积计算和检测结果输出流程，验证轻量级神经网络在 FPGA 上部署的可行性；第二版进一步结合硬件资源和实时处理需求，对卷积引擎、INT8 定点计算、特征图乒乓缓存、图像缩放预处理以及目标框解码进行了优化，并将 CNN 检测结果与 HDMI 视频、DDR3 帧缓存和 UART 坐标输出相结合，形成了较完整的实时车牌检测数据链路。

通过两版 CNN 的设计和迭代，我们更加直观地认识到 FPGA 片上 BRAM、DSP、逻辑单元和存储带宽对网络结构及并行度的限制。提高计算并行度能够缩短推理时间，但会增加计算资源和存储带宽需求；过度复用硬件资源虽然能够降低资源占用，却可能导致推理周期增加。因此，模型精度、资源消耗、处理延迟和系统吞吐率之间必须进行综合权衡。通过 FPGA、ARM 与 GPU 异构计算方案的开发和比较，我们也进一步理解了不同计算平台的特点：FPGA 适合完成视频采集、预处理、确定性数据流和低延迟推理；ARM 适合承担系统控制、参数配置、通信协议和任务调度；GPU 则更适合模型训练、算法验证及高算力推理。项目实践表明，实时性与智能化并不是单纯依靠增加算法复杂度实现的，而是需要通过软硬件合理分工、网络轻量化、数据流优化和系统级协同设计取得平衡。这也是本项目开发过程中最重要的认识和收获。

团队分工明确、协作紧密：一名队员负责 RK3568 端检测与识别模型的训练、量化和部署，完成 PC 端上位机开发及 RK3568 与 FPGA 之间的通信实现；一名队员负责 FPGA 车牌检测 CNN 的网络结构设计、数据集处理、模型训练、量化验证和部署参数导出；一名队员负责 CNN 网络的 FPGA 全流程部署，完成 INT8 卷积加速、量化、多尺度特征融合、四角点解码、存储调度及视频系统集成，并联合开展网络结构与硬件资源优化。团队共同完成模型精度、硬件资源、计算延迟和系统吞吐率之间的协同优化。

5 参考文献

[1]深圳市紫光同创电子有限公司. Logos2 系列 FPGA 器件数据手册：DS04001 V1.5[S]. 2020.

[2] OmniVision Technologies Inc. *OV5640 datasheet: 1/4-Inch color CMOS QSXGA (5 Megapixel) image sensor with OmniBSI™ technology* (version 2.01) [EB/OL]. (2011-01) [2024-11-20]. https://www.ovt.com/sensors/OV5640.

[3] 深圳市小眼睛科技有限公司. MES-FMC-LPC-IO 子卡硬件使用手册[M]. v1.1. 深圳: 深圳市小眼睛科技有限公司, 2025.

[4]深圳市小眼睛科技有限公司. MES2L676-200HP 硬件使用手册[M]. v1.2. 深圳: 深圳市小眼睛科技有限公司, 2024.

[5] S. Chen, Q. Song, H. Guo, and X. Li, "Design of license plate recognition system based on FPGA," in *2022 2nd Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS)*, Shenyang, China, 2022, pp. 227-231. doi: 10.1109/ACCTCS53867.2022.00054.

[6]正点原子仓库地址：https://github.com/alientek-openedv/Products.git

6 AI 使用说明

本团队在系统的研发与文档撰写过程中，严格遵守学术诚信与比赛规范。为了提高研发效率和突破技术瓶颈，我们在部分环节合理地借助了 AI 工具的辅助。特此声明本作品中 AI 工具的具体使用情况如下：

一、 创意与方案辅助

使用的 AI 工具：ChatGPT / Gemini

1. 查找表数据的离线生成

涉及文件： sigmoid_center.hex、sigmoid_corners.hex、sigmoid_offsets.hex

320×320 网络的检测 Head 需要输出角点置信度与亚像素偏移，算法端依赖 Sigmoid 激活函数。FPGA 硬件无法直接计算指数运算，因此采用查找表（LUT）

方案：将 Sigmoid 函数离散化为 256 个采样点，预先计算定点值并导出为.hex 文件，供 FPGA 的 BRAM ROM 初始化使用。

AI 辅助编写了 Python 脚本，用于在离线环境下批量计算各采样点的 8 位定点量化值，并输出为十六进制格式。

说明： 量化位宽（8 位输入/8 位输出）和采样点数（256 点）由团队成员根据硬件资源与推理精度要求确定。AI 仅协助编写数据生成脚本，**不涉及任何**硬件设计。FPGA 端的 BRAM 例化和查表控制逻辑由团队成员手工 RTL 实现。

2. Q9.7 定点数格式的数学分析

涉及场景： 坐标解码中的位宽分配与数值范围分析

在 320×320 网络的坐标解码过程中，粗坐标、局部峰值、精细偏移三者需要合成为最终的 Q9.7 格式角点坐标。设计初期，团队对整数部分 9 位是否能覆盖 1920×1080 的显示范围、小数部分 7 位的精度是否满足亚像素定位需求存在疑虑。AI 帮助团队分析了 Q 格式定点的数值范围与精度：9 位整数部分可覆盖 0~511，足以覆盖 1920×1080；7 位小数部分精度为 1/128≈0.0078 像素，满足亚像素级定位需求。基于该数学分析结论，团队成员独立完成了 Verilog 中的位移拼接逻辑和符号扩展编码。

说明： AI 提供的帮助仅限于定点数格式的数值范围分析，属于数学层面的理论推演。所有的硬件实现细节（位拼接、符号扩展、流水线划分）均由团队成员自主完成。

3. 色彩阈值调试工具（**初赛阶段，已废弃**）

涉及文件： Python GUI 调节器脚本

在初赛阶段的颜色检测方案中，团队需要确定 YCbCr 色彩空间中车牌颜色的有效阈值范围。AI 辅助编写了一款轻量级 Python GUI 调节器，通过滑动条实时调节各通道阈值并同步显示二值化效果。该工具加速了阈值标定过程，最终确定的参数被固化在 FPGA 的色彩状态机中。

说明： 该工具仅用于参数可视化标定，不涉及任何硬件逻辑。

二、 代码调试与验证辅助

使用的 AI 工具：ChatGPT / Claude

1. 仿真波形分析与问题定位辅助

在模块级仿真验证过程中，某些信号行为与预期不符时，团队需要从仿真波形中抓取关键信号分析时序关系。AI 辅助分析了波形中特定信号跳变的因果关系，帮助团队理解信号之间的时序依赖。

说明： 问题定位和代码修改均由团队成员完成，AI 仅辅助解释波形信号之间的因果关系。所有的调试结论由团队成员基于对设计意图的理解独立做出。

2. Testbench 通用框架模板

在编写仿真测试激励时，AI 辅助生成了部分 Testbench 的通用框架代码（时钟生成、复位控制、模块例化等重复性模板）。具体的信号赋值、时序控制、预期值判断和覆盖率规划均由团队成员根据模块功能需求独立编写。

说明： AI 生成的仅为通用的空壳模板，不包含任何与 CNN 功能相关的测试逻辑。所有测试用例的设计均源于团队成员对模块行为的理解。

三、 文档撰写与语言润色

使用的 AI 工具：ChatGPT /豆包大模型

团队成员在撰写技术报告初稿时，表述较为口语化。我们将核心逻辑要点交由 AI 润色，AI 帮助规范了专业术语，使复杂硬件时序逻辑的描述更加符合学术严谨性要求。

说明： 所有技术内容、架构分析、设计思路均为团队原创，AI 仅对表达方式进行规范化调整，不产生任何技术结论。

四、 AI 生成内容占比声明

内容类别	AI 辅助占比	团队原创占比
RTL 代码（Verilog .v 文件）	0%	100%
硬件架构设计与状态机	0%	100%
模型训练与量化参数提取	0%	100%
综合约束与布局布线	0%	100%
上板调试与验证	0%	100%
技术文档正文	约 8%（语言润色）	92%
设计思路与技术结论	5-10%	90-95%
查找表数据文件（.hex）	约 10%（批量生成脚本）	90%
测试数据与实验分析	5%	95%
Python 调试工具	约 10%（框架代码）	90%
Testbench 通用模板	约 8%（时钟/复位框架）	92%

表 10AI 生成内容占比说明